

Sistema de Integração Segura e Análise Automática de CCTV com Câmaras em Redes Não Controladas

Gustavo Caiano

**Dissertação para obtenção do Grau de Mestre em
Engenharia Informática, Área de Especialização em
Engenharia de Software**

**Orientador: Paulo Baltarejo Sousa
Supervisor: Francisco Loureiro**

Porto, 8 de junho de 2026

Declaração de Integridade

Declaro ter conduzido este trabalho académico com integridade.

Não plagiei ou apliquei qualquer forma de uso indevido de informações ou falsificação de resultados ao longo do processo que levou à sua elaboração.

Portanto, o trabalho apresentado neste documento é original e de minha autoria, não tendo sido utilizado anteriormente para nenhum outro fim. As exceções estão explicitamente reconhecidas na secção “Considerações Éticas” do primeiro capítulo. Esta secção também declara como as ferramentas de IA foram utilizadas e para que finalidade.

Declaro ainda que tenho pleno conhecimento do Código de Conduta Ética do P.PORTO.

ISEP, Porto, 8 de junho de 2026

Resumo

O presente trabalho de dissertação propõe o desenvolvimento de uma plataforma de software concebida para centralizar e organizar sistemas de videovigilância heterogêneos, geograficamente dispersos e situados em redes não controladas. Esta investigação surge em resposta a desafios críticos identificados na segurança pública e privada, nomeadamente a fragmentação tecnológica, as vulnerabilidades de segurança inerentes às comunicações Peer-to-Peer (P2P) e a ineficiência operacional enfrentada por entidades como os Órgãos de Polícia Criminal (OPCs) na análise forense de vídeo. A plataforma proposta, designada Flexible Universal Stream Engine (FUSE), pretende responder a estes desafios através de uma abordagem unificada de integração segura e análise automática de vídeo.

A arquitetura proposta integra uma camada de abstração de hardware que normaliza a comunicação com câmaras de diversos fabricantes através do protocolo Real Time Streaming Protocol (RTSP), garantindo a interoperabilidade. As suas comunicações são asseguradas através de túneis Virtual Private Network (VPN) que protegem a integridade e confidencialidade dos dados em redes não controladas, complementadas por um modelo de autorização baseado em permissões granulares e verificações contextuais sobre utilizadores, processos, câmaras e ações. Adicionalmente, o sistema incorpora um módulo de visão computacional progressivo, estruturado em três fases: deteção de atividade para filtragem temporal, classificação de objetos (e.g., humanos, veículos) e extração de atributos específicos, visando a automatização da análise forense e a redução significativa da carga de trabalho manual.

A metodologia adotada segue o paradigma de "Design and Creation", com a validação da solução a ser realizada através de um Minimum Viable Product (MVP) em cenários simulados e reais. Os resultados esperados centram-se na demonstração da eficácia da integração segura de dispositivos, na robustez da proteção de dados e na otimização dos processos de investigação criminal, sempre em estrita observância dos princípios éticos e regulamentares, como o EU AI Act.

Palavras-chave: Videovigilância, Integração CCTV, Automatização, Deteção de Objetos, Software Seguro, IoT Communication

Abstract

This dissertation proposes the development of a software platform designed to centralize and organize heterogeneous, geographically dispersed video surveillance systems located in uncontrolled networks. This research emerges in response to critical challenges identified in public and private security, namely technological fragmentation, security vulnerabilities inherent to P2P communications, and the operational inefficiency faced by entities such as Law Enforcement Agencies (LEAs) in forensic video analysis. The proposed platform, designated as Flexible Universal Stream Engine (FUSE), aims to address these challenges through a unified approach to secure integration and automated video analysis.

The proposed architecture integrates a hardware abstraction layer that normalizes communication with cameras from various manufacturers via the RTSP protocol, ensuring interoperability. Communication security is ensured through VPN tunnels that protect data integrity and confidentiality across uncontrolled networks, complemented by an authorization model based on granular permissions and contextual checks over users, investigation cases, cameras, and actions. Additionally, the system incorporates a progressive computer vision module structured in three phases: activity detection for temporal filtering, object classification (e.g., humans, vehicles), and specific attribute extraction, aiming to automate forensic analysis and significantly reduce manual workload.

The adopted methodology follows the "Design and Creation" paradigm, with the solution's validation being conducted through an MVP in both simulated and real-world scenarios. The expected results focus on demonstrating the effectiveness of secure device integration, the robustness of data protection, and the optimization of criminal investigation processes, always in strict compliance with ethical and regulatory principles such as the EU AI Act.

Agradecimentos

Agradeço ao meu orientador, Professor Paulo Baltarejo Sousa, pela disponibilidade, orientação e apoio prestado ao longo do desenvolvimento deste trabalho. Agradeço igualmente à entidade acolhedora, StabilityBubble, Lda., por disponibilizar os recursos e os meios profissionais que tornaram o desenvolvimento do projeto mais acessível. Este agradecimento é dirigido em particular ao Ernesto Sousa, ao Luís Landeiro e ao Francisco Loureiro pelo acompanhamento constante e partilha de conhecimento e ligações com o contexto real, que contribuíram para aproximar a componente teórica da prática.

Índice

Lista de Figuras	xv
Lista de Tabelas	xvii
Lista de Excertos de Código	xix
1 Introdução	1
1.1 Contexto	1
1.2 Problema	1
1.3 Pergunta de Investigação	3
1.4 Objetivos	4
1.4.1 Requisitos da Entidade	5
1.5 Metodologia	5
1.6 Considerações Éticas	7
1.6.1 Utilização de Artificial Intelligence (AI)	8
1.7 Planeamento	8
1.8 Estrutura do Documento	10
2 Estudo do Estado da Arte	11
2.1 Processo de Investigação	11
2.2 LRRQ1 - Arquiteturas de Abstração e Interoperabilidade IoT	12
2.2.1 Processo de Pesquisa	12
2.2.2 Discussão	14
Padrões Arquiteturais	15
Abstração de Media e Normalização de Streaming	15
Escalabilidade e Adaptabilidade à Rede	15
Análise Crítica	16
2.3 LRRQ2 - Protocolos de Comunicação Segura e VPN	16
2.3.1 Processo de Pesquisa	16
2.3.2 Discussão	19
O Desafio da Conectividade em Redes Não Controladas	19
Limitações dos Protocolos Legacy e Comparação TCP vs. UDP	19
A Ascensão do WireGuard e Túneis de Alta Performance	19
Análise Crítica	20
2.4 Estado da Arte em Visão Computacional	20
2.4.1 Processamento em Edge vs Centralizado	21
2.4.2 Detecção de Movimento e Filtragem Temporal (Fase 1)	22
2.4.3 Detecção e Classificação de Objetos (Fase 2)	22
2.4.4 Modelos de Visão-Linguagem (Vision-Language Models (VLMs)) e Identificação Alvo (Fase 3)	22
2.4.5 Armazenamento e Pesquisa Forense	23

2.5	Conclusão do Estado da Arte	23
3	Análise e Desenho	25
3.1	Análise	25
3.1.1	Âmbito do MVP	25
3.1.2	Modelo de Domínio	26
	Gestão de Processos e Controlo de Acessos	27
	Dispositivos e Gravação de Vídeo	27
	Análise Automática e Triagem	27
3.1.3	Requisitos	28
	Requisitos Funcionais	28
	Requisitos Não Funcionais	28
3.2	Arquitetura da Solução	30
3.2.1	Conexão Segura aos Dispositivos	31
3.2.2	Componentes da Solução	31
3.2.3	Infraestrutura e Conectividade	32
3.2.4	Interações entre Componentes	34
	Processo de Streaming	34
	Processo de Controlo da Câmara	35
	Processo de Detecção	37
3.3	Rastreabilidade das Decisões Arquiteturais	38
3.4	Síntese	38
4	Implementação	41
4.1	Ambiente Tecnológico e Implantação do MVP	41
4.1.1	Tecnologias Utilizadas	41
	Fundamentação de Escolhas	42
4.1.2	Implementação com Docker Compose	42
4.2	Orquestração Central e Modelo Operacional	44
4.2.1	Concretização do Modelo de Domínio	44
4.2.2	Controlo de Acessos na Aplicação	45
	Filtragem por Processo	45
	<i>Roles</i> e Permissões	46
4.2.3	Recursos Filament e Operação da Plataforma	47
4.2.4	Serviços Aplicacionais, Configuração e Integração	48
	<i>Actions</i> Reutilizáveis	48
	Camada de Clientes de Integração	49
	Configuração Aplicacional	50
4.2.5	Processamento Assíncrono e Trabalhos em Segundo Plano	51
4.3	Streaming, Gravação e Operação Remota	52
4.3.1	Configuração Funcional do <i>media-parser</i>	52
4.3.2	<i>Proxy</i> HLS pelo Nginx	53
4.3.3	Operação Remota PTZ	54
4.4	Segurança e Rastreabilidade	54
4.4.1	Conectividade Segura com Tailscale	55
4.4.2	Proteção do <i>Streaming</i> HTTP Live Streaming (HLS)	55
4.4.3	Autenticação de Chamadas entre Componentes	56
4.4.4	Auditoria e Rastreabilidade	57
4.5	Detetor Automático e Pipeline de Análise	57

4.5.1	Ambiente Tecnológico e Implantação do Serviço	57
	Tecnologias Utilizadas	57
	Fundamentação de Escolhas	58
	Implantação com Docker Compose	58
4.5.2	Pipeline de Análise	59
4.5.3	Contrato de Resultado	62
4.6	Persistência dos Eventos e Artefactos de Análise	64
4.6.1	artefactos que vao ser visuais	64
4.7	Compromissos e Limitações da Implementação	64
4.8	Síntese	64
5	Validação, Avaliação e Resultados	65
5.1	Plano de Avaliação	65
5.2	Cenário Experimental	65
5.3	Validação Funcional	65
5.4	Avaliação de Segurança e Dependências	65
5.4.1	Software Composition Analysis	65
5.5	Avaliação da Conectividade e Streaming	65
5.6	Avaliação do Controlo PTZ	65
5.7	Avaliação do Detetor	65
5.8	Avaliação Qualitativa	65
5.9	Discussão dos Resultados	65
5.10	Síntese	65
6	Considerações Finais	67
6.1	Conclusões	67
6.2	Trabalho Futuro	67
	Bibliografia	69
	Apêndice A Questionário e Entrevistas	75
A.1	Síntese de Resultados	75
A.2	Questionário (ficheiro de respostas)	75
	Apêndice B Project Charter	77
B.1	Project Charter (documento completo)	77
	Apêndice C Work Breakdown Structure	79
	Apêndice D Identificação de Stakeholders e Gestão de Riscos	81
D.1	Identificação de Stakeholders	81
D.2	Gestão de Riscos	82

Lista de Figuras

1.1	Cenário atual de acesso remoto a sistemas Closed Circuit Television (CCTV), destacando a dependência de servidores P2P e a exposição insegura via <i>firewall</i> .	2
1.2	Diagrama de "Research Process" de B. J. Oates [7], aplicado ao contexto do projeto.	6
1.3	Gantt Chart do projeto.	9
2.1	Fluxo PRISMA para a LRRQ1	14
2.2	Fluxo PRISMA para a LRRQ2	18
3.1	Modelo de Domínio Conceptual	26
3.2	Cenário de conectividade segura proposto para o FUSE com Tailscale. . . .	31
3.3	Vista híbrida da infraestrutura e conectividade da solução	33
3.4	Vista de Processo de Streaming	35
3.5	Vista do processo de controlo da câmara	36
3.6	Vista do processo de deteção automática	37
4.1	Pipeline de análise implementada no detetor automático.	60
C.1	WBS do projeto.	80

Lista de Tabelas

2.1	Questões de Pesquisa Orientadas à Revisão Literária	11
2.2	Modelo PICOCS para a LRRQ1	13
2.3	Modelo PICOCS para a LRRQ2	17
3.1	Requisitos funcionais do MVP	28
3.2	Requisitos não funcionais do MVP	29
3.3	Critérios de validação dos requisitos não funcionais	30
4.1	Mapeamento entre componentes conceptuais e recursos Docker	42
4.2	Principais recursos Filament da aplicação central	47
D.1	Identificação de Stakeholders com atribuição de Poder e Interesse	82
D.2	Identificação e gestão de riscos associados ao projeto	83

Lista de Excertos de Código

4.1	Excerto simplificado da configuração Docker Compose do servidor central (yaml)	43
4.2	Excerto demonstrativo da filtragem por processo e câmara (php)	45
4.3	Excerto do funcionamento do filtro <i>relatedToUser</i> (php)	46
4.4	Páginas definidas no recurso de Câmara (php)	47
4.5	Action reutilizável para iniciar ou parar gravação (php)	49
4.6	Excerto simplificado de alteração de gravação para o cliente <i>media-parser</i> (php)	50
4.7	Configuração e construção do <i>endpoint</i> do <i>media-parser</i> (php)	51
4.8	Excerto simplificado de origem de tarefa após criação de uma Câmara (php)	51
4.9	Excerto simplificado da configuração funcional do MediaMTX (yaml)	52
4.10	<i>Proxy</i> funcional de pedidos aplicacionais e HLS no Nginx (nginx)	53
4.11	Excerto simplificado de encaminhamento de controlo PTZ (python)	54
4.12	Validação simplificada do acesso HLS no Nginx (nginx)	56
4.13	Aplicação do <i>trait</i> de auditoria na classe base dos modelos (php)	57
4.14	Excerto simplificado da implantação do serviço de deteção (yaml)	59
4.15	Representação simplificada da orquestração interna da <i>pipeline</i> de análise (python)	62
4.16	Exemplo simplificado da estrutura de eventos devolvida pelo detetor (json)	63

Capítulo 1

Introdução

Este capítulo introduz o âmbito e a motivação do presente trabalho, contextualizando o desenvolvimento da plataforma proposta face aos desafios atuais da videovigilância e investigação criminal. Ao longo das secções seguintes, é definido o problema de investigação, formuladas as questões centrais e estabelecidos os objetivos técnico-científicos. Por fim, são detalhadas a metodologia de investigação adotada, as considerações éticas fundamentais, o planeamento do projeto e a organização estrutural da dissertação.

1.1 Contexto

No panorama atual dos sistemas de videovigilância verifica-se uma grande dependência de hardware, como Network Video Recorders (NVRs) para agregar várias câmaras e disponibilizar streaming da sua imagem. A maioria destes dispositivos também possibilita o *playback* e exportação de vídeos gravados, sendo que alguns controlam ainda os acessos de utilizadores. A deteção de movimentos ou objetos é uma *feature* de apenas alguns modelos apesar de que, ultimamente, se têm verificado um grande crescimento e aposta tecnológica nesta área [1]. Assim, os NVRs, embora raramente reúnam individualmente todas estas funcionalidades num dispositivo só [2], são a peça central do processo de videovigilância dentro de uma rede controlada.

O presente trabalho decorre em contexto empresarial, sendo acolhido pela empresa StabilityBubble, Lda. A proposta não se configura apenas como um exercício académico teórico, mas sim como a resposta a lacunas de mercado identificadas pela empresa. Assim, esta dissertação visa conciliar o rigor da investigação científica com as necessidades práticas e operacionais do setor, beneficiando da infraestrutura e *know-how* da entidade de acolhimento.

1.2 Problema

Em organizações com instalações geograficamente distribuídas, é frequente que as câmaras Closed Circuit Television (CCTV) estejam instaladas em redes distintas, muitas vezes fora do controlo direto da entidade que pretende monitorizá-las. Nestes cenários [3], existe a impossibilidade destas câmaras serem inseridas todas numa determinada Local Area Network (LAN) privada. Por este motivo, torna-se necessário o acesso via Internet, o que origina desafios técnicos significativos, tais como:

- Incompatibilidade entre câmaras de diferentes fornecedores, obrigando frequentemente à utilização de mais do que um software, visto que nem todos os equipamentos são compatíveis com a mesma aplicação.
- Dependência de infraestruturas de terceiros para contornar restrições de rede local (LAN), recorrendo-se tipicamente a conexões Peer-to-Peer (P2P). Estas ligações dependem de servidores *relay* do fabricante, sobre os quais não existe controlo nem garantia de proteção dos dados [4].
- Aumento da superfície de ataque da rede quando se opta pela abertura direta de portas no *router* ou *firewall* local (*port forwarding*) como alternativa ao P2P, expondo os equipamentos diretamente à Internet.
- Falta de confidencialidade e integridade, uma vez que a comunicação entre a aplicação e a câmara é frequentemente não cifrada, expondo a *stream* de vídeo e as credenciais a potenciais interceções.
- Falta de controlo de acessos e gestão de utilizadores, pois a maior parte das aplicações de acesso remoto a CCTV prioriza o acesso intuitivo e rápido em detrimento de políticas rigorosas de segurança [5].

A Figura 1.1 ilustra os dois paradigmas arquiteturais que ocorrem com mais frequência para o acesso remoto.

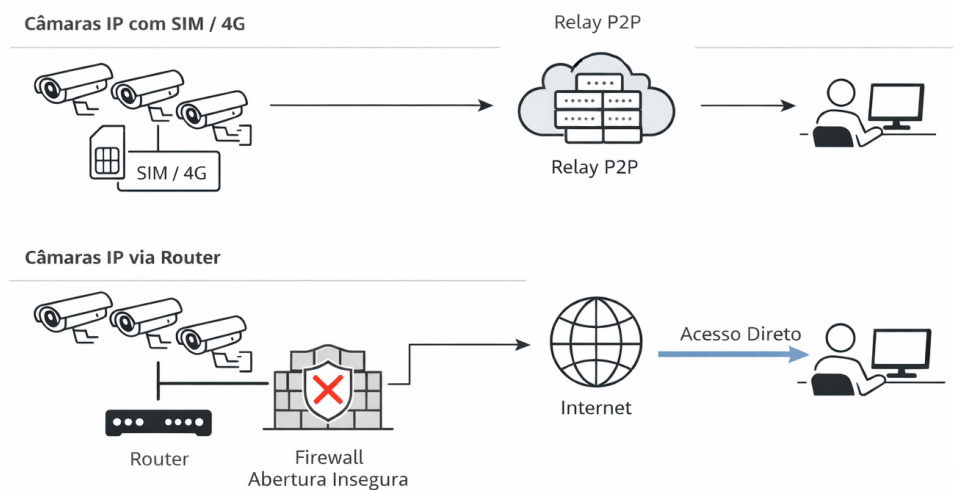


Figura 1.1: Cenário atual de acesso remoto a sistemas CCTV, destacando a dependência de servidores P2P e a exposição insegura via *firewall*.

No primeiro, as conexões são estabelecidas através de um servidor intermediário do fabricante dos dispositivos. No segundo, as conexões são estabelecidas diretamente entre o cliente e a câmara, tendo que o acesso à mesma ser exposto publicamente à Internet. Os cenários

descritos evidenciam as falhas de segurança e a dependência de infraestruturas de terceiros que caracterizam o cenário atual.

Estes desafios arquiteturais e de segurança assumem uma gravidade particular no âmbito da segurança pública. Quando os Órgãos de Polícia Criminal (OPCs) recorrem à instalação de meios técnicos de videovigilância para monitorização de suspeitos, deparam-se frequentemente com a necessidade de operar em redes externas não controladas. Nestes cenários críticos, a dependência de plataformas heterogêneas e as vulnerabilidades do acesso remoto comprometem não só a segurança operacional, mas também a integridade da própria cadeia de custódia da prova.

A estas barreiras técnicas soma-se um desafio operacional igualmente limitante. O processo atual de análise forense do vídeo recolhido assenta frequentemente na revisão humana através da visualização integral contínua das gravações efetuadas, exigindo muitas vezes a divisão temporal das horas de vídeo por vários agentes para tentar agilizar a tarefa. A evidência exploratória recolhida através de um questionário conduzido junto de núcleos e esquadras de investigação criminal da Guarda Nacional Republicana (GNR) e da Polícia de Segurança Pública (PSP) (consultável no Apêndice A) valida a existência destas dificuldades práticas: os inquiridos relatam o uso generalizado de equipamentos móveis (4G) em redes externas e acessos remotos diretos inseguros, lidando rotineiramente com a dependência de múltiplas plataformas incompatíveis e com restrições institucionais para a instalação de software. Por conseguinte, este processo de investigação torna-se demorado e altamente intensivo, recorrendo a uma enorme quantidade de recursos humanos e tempo, que são valiosos e, infelizmente, escassos [6].

Estes dois eixos — técnico-arquitetural e operacional — convergem na ausência de uma camada de software intermédia capaz de abstrair a diversidade dos dispositivos, assegurar comunicações controladas e disponibilizar mecanismos comuns de gestão, auditoria e análise automática de possível prova. É neste contexto de múltiplos desafios que surge a proposta deste trabalho: o desenvolvimento do Flexible Universal Stream Engine (FUSE). O FUSE é idealizado como uma plataforma de software, *self-hosted* e *web-served*, que centraliza e organiza sistemas de videovigilância heterogêneos e geograficamente dispersos, com um foco integrado na segurança dos dados, na automatização da análise de vídeo, e na manutenção de registos de auditoria e gestão de utilizadores.

1.3 Pergunta de Investigação

Considerando os desafios técnicos e operacionais expostos, e com o intuito de resolver as falhas de segurança e falta de eficiência nos processos atuais, a proposta desta plataforma levanta a seguinte pergunta principal de investigação:

RQ1: De que forma pode ser desenhada/projetada uma solução de software que permita aceder seguramente a câmaras CCTV, localizadas em redes externas não controladas, caracterizadas por uma elevada diversidade de arquiteturas, padrões tecnológicos e origem de fabrico?

Decompondo esta pergunta para uma melhor e mais estruturada compreensão, a investigação será guiada pelas seguintes sub-perguntas operacionais:

RQ1.1: Como pode ser desenhada uma camada de abstração de software que normalize as funcionalidades (visualização, controlo, gravação) de câmaras de diferentes interfaces e especificações técnicas distintas, garantindo a extensibilidade futura do sistema?

RQ1.2: Que mecanismos de rede e protocolos de comunicação podem ser utilizados para garantir a confidencialidade e integridade da comunicação com câmaras localizadas em redes não fidedignas, sem introduzir vulnerabilidades na rede de destino e agregação das várias *streams*?

RQ1.3: Como podem ser integrados modelos de visão computacional para a automatização da detecção de eventos, validando a utilização da plataforma proposta para otimização de processos de investigação criminal?

Estas questões irão ser respondidas ao longo do decorrer do documento. A resposta à questão central de investigação (RQ1) será também materializada através da implementação e validação experimental da plataforma FUSE. Todo o processo encontra-se detalhado na Seção 1.5 referente à Metodologia. No entanto, para fundamentar as decisões técnicas necessárias na implementação do Minimum Viable Product (MVP) analisar-se-ão as sub-questões operacionais, no Capítulo 2.

1.4 Objetivos

De forma a operacionalizar a resposta às questões de investigação formuladas e garantir a criação de um MVP, foram definidos os seguintes objetivos para o desenvolvimento e validação do FUSE:

- Desenvolver uma arquitetura de software extensível que, através de uma camada de abstração, normalize a comunicação com câmaras de diferentes fornecedores. O sistema deverá receber e suportar o *streaming* e centralizar as funcionalidades de visualização em tempo real, gravação, e *playback* numa interface unificada.
- Implementar um modelo de comunicação seguro, Secured By Design, que utilize túneis Virtual Private Network (VPN) para isolar e criptografar a comunicação entre as câmaras externas e o servidor de implementação da aplicação.
- Desenvolver um sistema de controlo de acessos baseado em permissões granulares e verificações contextuais, permitindo gerir ações por utilizador, processo, câmara e recurso, bem como garantir a auditoria das operações relevantes.
- Integrar um módulo de análise de vídeo para a detecção automatizada de eventos complexos, implementando uma *pipeline* de processamento progressivo em três fases:
 - Fase 1 (Detecção de Atividade): Identificação de movimento e atividade relevante nos *streams* de vídeo para filtrar segmentos de interesse.
 - Fase 2 (Classificação de Objetos): Análise dos segmentos filtrados para detectar e classificar objetos de categorias pré-definidas (e.g., humanos, veículos, animais).
 - Fase 3 (Extração de Atributos): Análise aprofundada dos objetos classificados para extrair características específicas e customizáveis, como matrículas e cores de veículos, ou atributos de vestuário e acessórios de pessoas, que servirão de base para alertar ao OPC de determinado evento complexo.
- Validar a viabilidade da arquitetura através de um MVP que demonstre a integração bem-sucedida de, no mínimo, duas câmaras tecnologicamente diferentes, a segurança da comunicação e a eficácia da detecção de eventos num cenário simulado, pelo menos até à Fase 2 anteriormente mencionada.

- Desenvolver a plataforma com uma arquitetura *self-hosted*, garantindo que o armazenamento e processamento de dados sensíveis ocorrem exclusivamente na infraestrutura da entidade utilizadora, eliminando a dependência de serviços cloud de terceiros para a persistência de provas.

Quanto aos principais contributos deste trabalho, espera-se que do ponto de vista técnico-científico surja uma arquitetura referência para integração segura e inteligente de sistemas CCTV distribuídos, principalmente quando há a presença de interoperabilidade entre redes não controladas e controladas. Por outro lado, do ponto de vista social e operacional, tenciona-se validar a ferramenta para que esta possa vir a aumentar significativamente a eficiência e eficácia da investigação criminal dos OPCs, otimizando a alocação de recursos e reduzindo o tempo de análise manual do vídeo.

1.4.1 Requisitos da Entidade

Para além dos objetivos anteriormente definidos, a entidade de acolhimento estabelece um conjunto de requisitos tecnológicos para o desenvolvimento do projeto.

Em primeiro lugar, a aplicação principal do FUSE, excluindo eventuais módulos independentes que possam justificar a utilização de outras ferramentas, deverá ser desenvolvida sobre a stack tecnológica Laravel + Filament. Esta opção permite alinhar o desenvolvimento com o ecossistema técnico já dominado pela entidade, acelerando a implementação do MVP e facilitando a futura manutenção, extensão e integração da plataforma.

Adicionalmente, sempre que tecnicamente viável deverá ser privilegiada a utilização de soluções *open-source*. Este princípio aplica-se à escolha de bibliotecas, dependências e componentes de infraestrutura, reduzindo a dependência de soluções proprietárias e aumentando a auditabilidade do sistema.

1.5 Metodologia

A metodologia adotada para a realização deste trabalho académico baseia-se no modelo de “Research Process” proposto por B. J. Oates no livro “Researching Information Systems and Computing” [7], ilustrado na Figura 1.2.

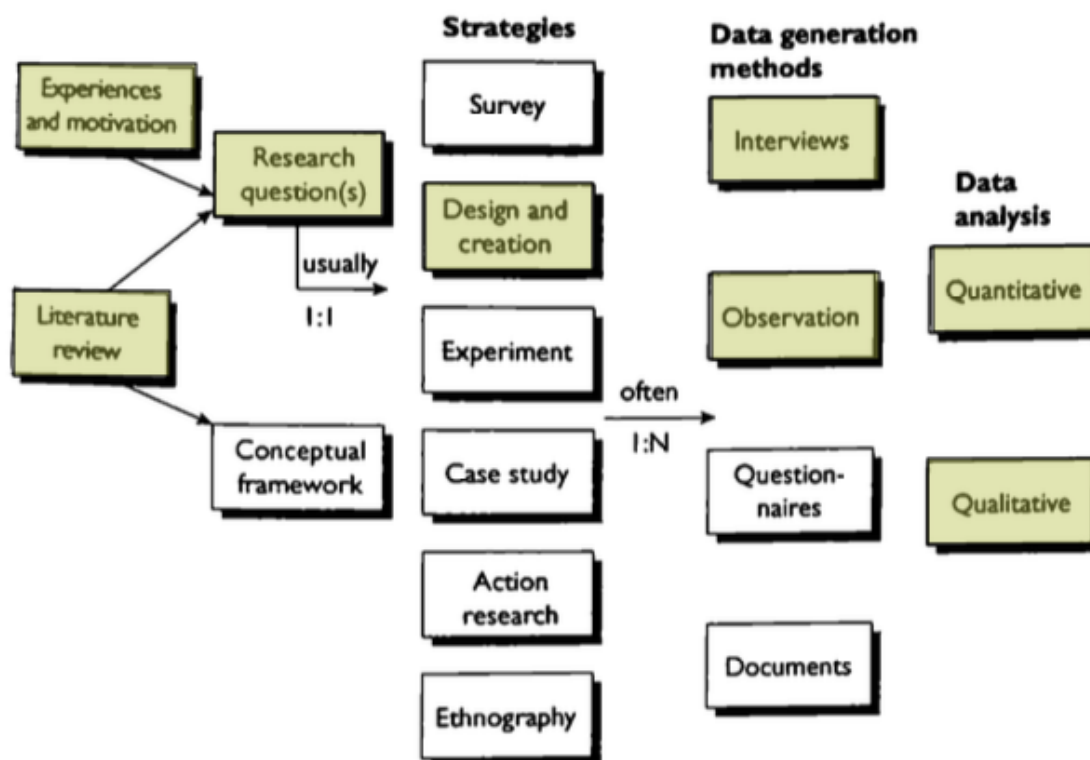


Figura 1.2: Diagrama de "Research Process" de B. J. Oates [7], aplicado ao contexto do projeto.

O ponto de partida da investigação enquadra-se em "**Experiences and motivation**", dado que a génese deste projeto deriva diretamente da observação da ineficiência e limitações técnicas enfrentadas pelos OPCs e gestores de segurança na recolha de imagens de CCTV. Através de feedback adquirido através de questionários e entrevistas com Núcleos (GNR) e Esquadras (PSP) de Investigação Criminal (Apêndice A), estes problemas são confirmados e registados. Complementarmente, é realizada uma "**Literature review**" preliminar para compreender o estado da arte em protocolos de comunicação e visão computacional. Esta análise permitiu identificar a ausência de soluções integradas que garantam simultaneamente segurança em redes não controladas e inteligência analítica avançada. A junção desta necessidade prática (experiência) com a identificação desta lacuna teórica (literatura) resultou na formalização das "**Research questions**" apresentadas anteriormente.

A estratégia central adotada para este trabalho é a de "**Design and Creation**". Esta abordagem é a mais adequada para projetos de Engenharia de Software cujo objetivo primário é o desenvolvimento de um protótipo que visa resolver muitos dos problemas práticos observados e identificados em determinado cenário, como forma de validação de futura implementação ou desenvolvimento de algo mais avançado. Dentro desta estratégia inserem-se quatro principais passos, nomeadamente Design, Desenvolvimento, Testes e Conclusões.

Relativamente aos resultados e à sua avaliação, será usado o método de "**Interviews**" com utilizadores finais, sendo estes OPCs ou encarregados de segurança de uma outra entidade, de modo a recolher feedback e validação do funcionamento do protótipo e respetiva utilidade do mesmo. Em adição, será também usado o método de "**Observation**", onde se poderá

obter e quantificar resultados e taxas de correspondência na análise automática de eventos e objetos.

Quanto à análise de dados, optou-se por uma abordagem mista que integra os dois métodos disponíveis: “**Quantitative**” e “**Qualitative**”. A análise quantitativa incidirá sobre as métricas técnicas recolhidas durante os testes do protótipo, tais como as taxas de precisão e alerta na deteção e classificação de objetos (Fase 2 e 3 da pipeline), a latência do streaming via túnel VPN, os tempos de processamento da análise de vídeo, entre outros. Já a análise qualitativa será aplicada à interpretação do feedback dos utilizadores e das observações de cenário real, avaliando o impacto operacional da ferramenta, a eficácia percebida na redução do tempo de investigação e a adequação da interface aos processos de trabalho dos OPCs.

1.6 Considerações Éticas

A realização deste trabalho rege-se pelos princípios de integridade, responsabilidade e rigor científico descritos no Código de Boas Práticas e de Conduta do P.PORTO [8].

Do ponto de vista profissional, o trabalho enquadra-se nos princípios estruturantes da engenharia de software, alinhados com referências internacionais como o Software Engineering Code of Ethics and Professional Practice, desenvolvido em conjunto pela Association for Computing Machinery (ACM) e pela IEEE Computer Society (IEEE-CS) [9]. A conformidade com estes princípios garante a qualidade e segurança dos sistemas desenvolvidos, a responsabilidade perante clientes e utilizadores, e a honestidade na comunicação de resultados, limitações e riscos associados às soluções tecnológicas.

O FUSE envolve uma *pipeline* de análise automática aplicada em contextos de videovigilância, potencialmente incidindo sobre a obtenção de dados pessoais identificáveis. Estes dados sensíveis apenas serão obtidos num contexto de validação do MVP em cenário real, que estará sempre salvaguardado por despacho judicial para fornecimento de meios técnicos e respetiva autorização de captação de imagens. Ainda assim, a *pipeline* é desenhada procurando alinhar-se com o EU AI Act [10], mitigando o risco de o sistema incorrer em “Unacceptable risks”.

Para além do enquadramento legal via despacho judicial para os testes em ambiente real, a privacidade a longo prazo é assegurada pelo design arquitetural da solução (Privacy by Design). Ao optar-se por um modelo *self-hosted* em detrimento de Software as a Service (SaaS), garante-se que a custódia das imagens permanece sempre sob o controlo exclusivo dos OPCs ou da entidade detentora, onde a plataforma é implementada.

O sistema não se destina, nem permite, práticas proibidas tais como:

- *Social scoring* ou manipulação comportamental;
- Recolha indiscriminada (*untargeted scraping*) de imagens de CCTV para criação de bases de dados de reconhecimento facial;
- Identificação biométrica remota em tempo real em espaços públicos para fins policiais (real-time remote biometric identification).

No âmbito do último ponto, apesar de o EU AI Act incidir sobretudo sobre a identificação biométrica remota em tempo real, a conformidade do FUSE é justificada pela natureza da análise proposta: esta incide sobre gravações de eventos e não sobre identificação biométrica em *streaming* contínuo.

A mitigação de utilizações indevidas é assegurada por três mecanismos complementares: em primeiro lugar, a utilização operacional por OPCs pressupõe a existência de despacho judicial associado a um processo concreto; em segundo lugar, a plataforma não disponibiliza funcionalidades de reconhecimento facial, identificação biométrica ou criação de bases de dados biométricas; por fim, as operações relevantes são sujeitas a controlo de acessos e registos de auditoria, permitindo rastrear ações realizadas por utilizadores autenticados.

A análise focada na "Fase 3" da *pipeline* de deteção automática restringe-se à identificação de características objetivas (e.g., cor de vestuário, tipo de veículo) para auxiliar a pesquisa forense, mantendo sempre o princípio da validação humana, onde a decisão final cabe ao agente humano e não ao algoritmo.

1.6.1 Utilização de Artificial Intelligence (AI)

No decorrer desta dissertação foram utilizadas ferramentas de AI como apoio ao processo de escrita, revisão e organização do trabalho. Todas as respostas geradas foram analisadas, editadas e validadas pelo autor, que assume integral responsabilidade pelo conteúdo final apresentado.

O *harness* mais utilizado foi o *opencode*, que permite orquestrar Large Language Models (LLMs) e disponibilizar-lhes, quando necessário, acesso controlado aos ficheiros do repositório, incluindo os ficheiros *.tex* da dissertação. Adicionalmente, foi utilizada uma *skill* de apoio à escrita académica, designada *academic-writing*, baseada numa *skill* pública disponibilizada num repositório GitHub [11], tendo sido apenas traduzida e adaptada para trabalho académico em português.

Foi também utilizada AI generativa para apoiar a criação da Figura 1.1, apresentada na introdução do problema. O objetivo desta utilização foi produzir uma representação explicativa e intuitiva do cenário atual de acesso remoto a sistemas CCTV. O *prompt* usado para esse efeito, assim como outros *prompts* relevantes utilizados ao longo do trabalho, encontram-se disponíveis no repositório público do projeto [12].

1.7 Planeamento

Esta secção descreve o planeamento deste projeto, estruturado para garantir a exequibilidade dos objetivos propostos dentro do prazo académico estipulado. A organização das atividades divide-se entre a fase de preparação (unidade curricular de Preparação para Dissertação (PREPD)) e a fase de execução e escrita da dissertação (unidade curricular de Dissertação do Mestrado em Engenharia Informática (DIMEI)).

A estruturação do projeto foi realizada com recurso a uma *Work Breakdown Structure* (Work Breakdown Structure (WBS)), detalhada no Apêndice C, que ditou a decomposição do trabalho em cinco fases incrementais:

- **Planeamento e Análise:** Definição do problema, estudo do estado da arte e produção de artefactos iniciais (ex. Project Charter e questionários aos OPCs).
- **Design:** Modelação arquitetural da solução (Vistas 4+1) e desenho do Modelo de Domínio.
- **Desenvolvimento:** Fase nuclear de implementação técnica, compreendendo os módulos de Comunicações Seguras, Camada de Abstração e Análise de Vídeo.

- **Testes:** Validação funcional e unitária da arquitetura, seguida de testes do protótipo em cenário real.
- **Conclusões:** Redação final do documento de dissertação e preparação da respetiva defesa pública.

O planeamento temporal encontra-se representado no diagrama de Gantt, Figura 1.3.

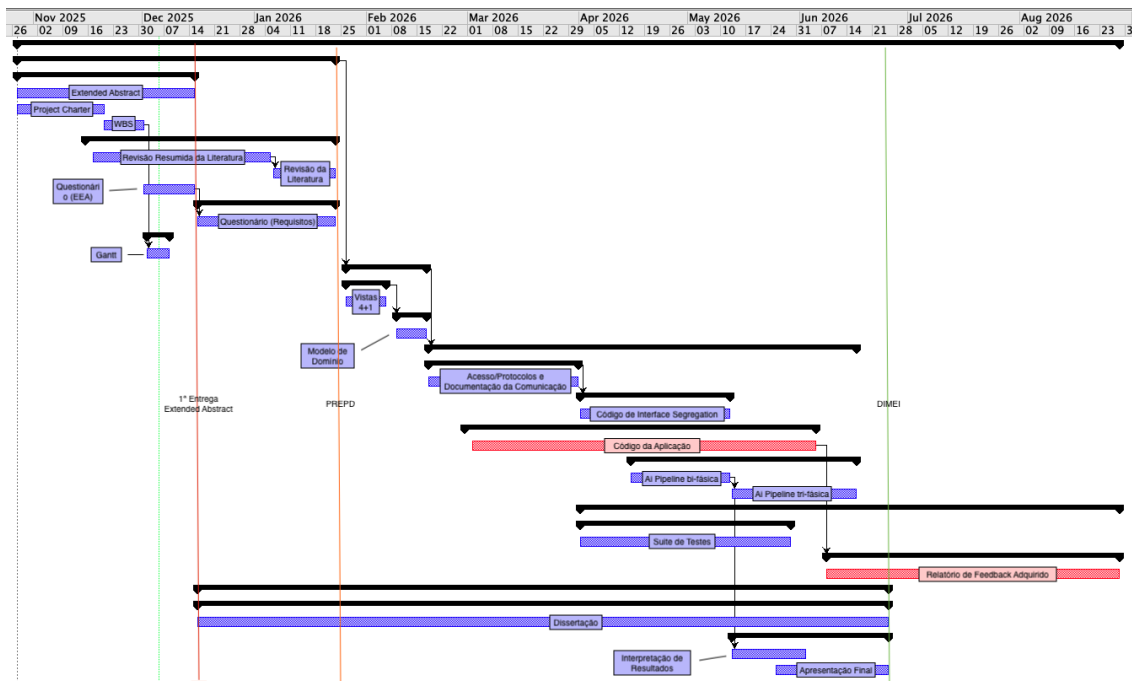


Figura 1.3: Gantt Chart do projeto.

A execução do cronograma estende-se por aproximadamente 220 dias úteis, iniciando-se em outubro de 2025 com a fase de **Planeamento e Análise (PREPD)**, cujo encerramento está estipulado para 22 de janeiro de 2026. A partir desta data, o projeto transita para a unidade curricular de DIMEI, arrancando com o **Design** arquitetural da solução.

A fase de **Desenvolvimento** representa o núcleo temporal do projeto (com uma duração estimada de 85 dias, entre fevereiro e junho de 2026). Esta fase acomoda, de forma parcialmente paralela, a implementação das Comunicações Seguras, da Camada de Abstração e do Módulo de Análise de Vídeo (desenvolvido numa abordagem bi-fásica e tri-fásica). Em simultâneo, a fase de **Testes** avança de forma contínua a partir de abril, garantindo a validação unitária e funcional iterativa dos componentes. A data-alvo para a entrega final da dissertação está fixada a 22 de junho de 2026.

Contudo, como ilustrado na Figura 1.3, existe trabalho planeado até ao final de agosto de 2026. Esta extensão temporal deve-se à complexidade da validação do protótipo em ambiente real (junto dos OPCs), que se espera exigir um período mais alargado do que o estipulado para o término formal da dissertação. O *feedback* adquirido até à data de entrega académica será anexado ao relatório final, enquanto a validação e otimização do sistema continuarão o seu curso natural pós-entrega. Por fim, a identificação dos *stakeholders* e a análise detalhada de riscos (matriz de probabilidade/impacto e mitigação) encontram-se documentadas no Apêndice D.

1.8 Estrutura do Documento

FALTA COISAS AQUIII ATENCAOOOOO

Este documento está organizado em capítulos que apresentam de forma estruturada o trabalho desenvolvido. O Capítulo 1 (Introdução) apresenta o contexto do problema, as questões de investigação, os objetivos, as considerações éticas, a metodologia adotada, o planeamento do projeto e a estrutura do documento.

O Capítulo 2 (Estudo do Estado da Arte) apresenta uma revisão sistemática da literatura sobre arquiteturas de abstração e interoperabilidade de dispositivos Internet of Things (IoT), protocolos de comunicação segura e visão computacional, respondendo às sub-questões de investigação operacionais.

O Capítulo 6 (Considerações Finais) resume os resultados desta fase de preparação e apresenta o plano para o trabalho futuro a desenvolver na dissertação.

O documento inclui ainda a Bibliografia com todas as referências consultadas e utilizadas, bem como os apêndices: o Apêndice A, que contém o link para download das respostas relativas ao questionário apresentado em formato de entrevista junto dos OPCs; o Apêndice B, que disponibiliza o Project Charter na sua versão integral; o Apêndice C, que detalha o WBS do projeto; e o Apêndice D, que apresenta a identificação de stakeholders e a análise de riscos.

Acrescenta algo do género, aqui, (uma subsection)

Relativamente às convenções de escrita adoptadas neste documento: os termos em **negrito** são utilizados para dar realce a conceitos-chave ou expressões de particular relevância; os termos em *itálico* identificam estrangeirismos em língua inglesa; os termos em `monospace` identificam identificadores de código, nomes de ficheiros, comandos e outros elementos técnicos concretos; as siglas e acrónimos são expandidos na sua primeira ocorrência e listados no glossário no final do documento.

Capítulo 2

Estudo do Estado da Arte

Este capítulo apresenta a revisão de literatura fundamentada no âmbito do projeto, analisando o estado atual das tecnologias críticas para o desenvolvimento do FUSE. O objetivo principal desta revisão é obter conhecimento para responder (total ou parcialmente) às sub-questões operacionais previamente identificadas, nomeadamente a RQ1.1 e a RQ1.2, estabelecendo uma base teórica sólida para as decisões de arquitetura e segurança.

Adicionalmente, e dada a natureza aplicada da RQ1.3, é conduzido um estudo técnico aprofundado sobre os paradigmas atuais de visão computacional. Este estudo visa não apenas identificar o estado da arte, mas também comparar padrões, *pipelines* de processamento e modelos de AI passíveis de integração na plataforma.

2.1 Processo de Investigação

O Processo de investigação foi guiado pelo mapeamento das duas primeiras sub-questões da presente dissertação em questões de pesquisa com foco na revisão literária, conforme descrito na Tabela 2.1.

Tabela 2.1: Questões de Pesquisa Orientadas à Revisão Literária

RQ ID	LRRQ ID	Description	Tópicos e Keywords de pesquisa
RQ1.1	LRRQ1	Quais são as arquiteturas de referência e padrões de design de software descritos na literatura para a abstração e interoperabilidade de dispositivos IoT heterogêneos?	IoT Interoperability, Hardware Abstraction Layer, Middleware patterns, Open Network Video Interface Forum (ONVIF) standardization, Heterogeneous device integration.
RQ1.2	LRRQ2	Qual o estado da arte em protocolos de comunicação segura e VPNs para acesso remoto e <i>streaming</i> ?	Secure Tunneling Protocols, VPN performance analysis, Streaming encryption, Network Address Translation (NAT) Traversal, Zero Trust Network Access (ZTNA).

Cada questão de pesquisa será enquadrada de acordo com o modelo Population, Intervention, Comparison, Outcomes, Context, Study (PICOCs) [13], garantindo o rigor na seleção

das fontes utilizadas. A informação será depois selecionada seguindo o fluxo Preferred Reporting Items for Systematic reviews and Meta-Analysis (PRISMA) [14]. O processo inclui a definição de keywords de pesquisa, a aplicação estrita de critérios de inclusão e exclusão, seguida de uma filtragem em etapas e, finalmente, uma discussão crítica sobre a aplicabilidade dos estudos selecionados para a arquitetura do FUSE.

Como fator inclusivo de seleção, selecionou-se, por exemplo, a data de publicação posterior a 2018. A escolha deste intervalo visa garantir que as arquiteturas, frameworks e algoritmos analisados representam o atual estado da arte, evitando a adoção de paradigmas que, embora válidos no passado, não refletem as necessidades de desempenho e escalabilidade dos sistemas modernos. Foram privilegiados artigos revistos por pares, normas técnicas e literatura técnica amplamente reconhecida. Excluíram-se estudos puramente teóricos sem aplicação prática ao domínio da videovigilância ou da integração de dispositivos, tal como trabalhos relativos a soluções proprietárias fechadas sem documentação técnica pública suficiente.

2.2 LRRQ1 - Arquiteturas de Abstração e Interoperabilidade IoT

Nesta secção, a literatura é revista de forma a responder à questão de investigação "Quais são as arquiteturas de referência e padrões de design de software descritos na literatura para a abstração e interoperabilidade de dispositivos IoT heterogêneos?".

2.2.1 Processo de Pesquisa

A questão de investigação foi estruturada de acordo com o modelo PICOCS apresentado na Tabela 2.2. Esta inclui os critérios de inclusão e exclusão (I/E) relacionados com cada parte da questão de investigação, bem como as possíveis *sub-strings* utilizadas para conduzir a pesquisa nas bases de dados científicas. Apenas estudos de 2018 em diante que contemplem arquiteturas de software ou *middleware* para integração de dispositivos heterogêneos foram considerados.

Tabela 2.2: Modelo PICOCS para a LRRQ1

PICOCS	Parte da RQ	I/E	Sub string
P	Estudos envolvendo dispositivos IoT heterogêneos e CCTV	I - Estudos que considerem heterogeneidade E - Anteriores a 2018	≥ 2018 ; IoT; Heterogeneous; CCTV
I	Arquiteturas de software, Middleware e Camadas de Abstração	I - Estudos sobre padrões de design e abstração E - Soluções proprietárias fechadas	Software Architecture; Middleware; Abstraction Layer; Design Patterns
C	—	—	—
O	Arquiteturas de referência e interoperabilidade	I - Estudos que propõem ou validam arquiteturas	Architecture; Interoperability; Framework
C	Engenharia de Software e Sistemas Distribuídos	I - Acadêmico I - Indústria	Software Engineering; Distributed Systems
S	Estudos de caso e Propostas de Arquitetura	I - Case Studies I - System Proposals E - Opiniões sem validação	Case study; Proposal; Prototype

A pesquisa foi conduzida em três bases de dados principais: IEEE Xplore (282 registros), ACM Digital Library (94 registros) e ScienceDirect (59 registros), totalizando 435 registros identificados através de pesquisa sistemática. Adicionalmente, foi identificado 1 registro [15] através de outras fontes, resultando num total de 436 registros na fase de identificação.

Após a remoção de duplicados (1 registro), foram analisados 435 registros na fase de *screening* através da leitura de títulos e resumos. Desta análise, 412 registros foram excluídos por não cumprirem os critérios de inclusão, nomeadamente por focarem-se puramente em algoritmos de visão computacional sem arquitetura de sistema, abordarem domínios como agricultura ou saúde sem componente de vídeo, ou simplesmente não se adequar ao tema de integração de dispositivos IoT ou de camadas de abstração em design de software. Os 23 registros restantes foram avaliados em texto completo, dos quais 12 foram excluídos por falta de detalhes de implementação, ausência de diagrama de arquitetura claro, ou por serem propostas teóricas sem validação prática. O processo resultou na inclusão final de 11 estudos na revisão qualitativa, conforme ilustrado na Figura 2.1.

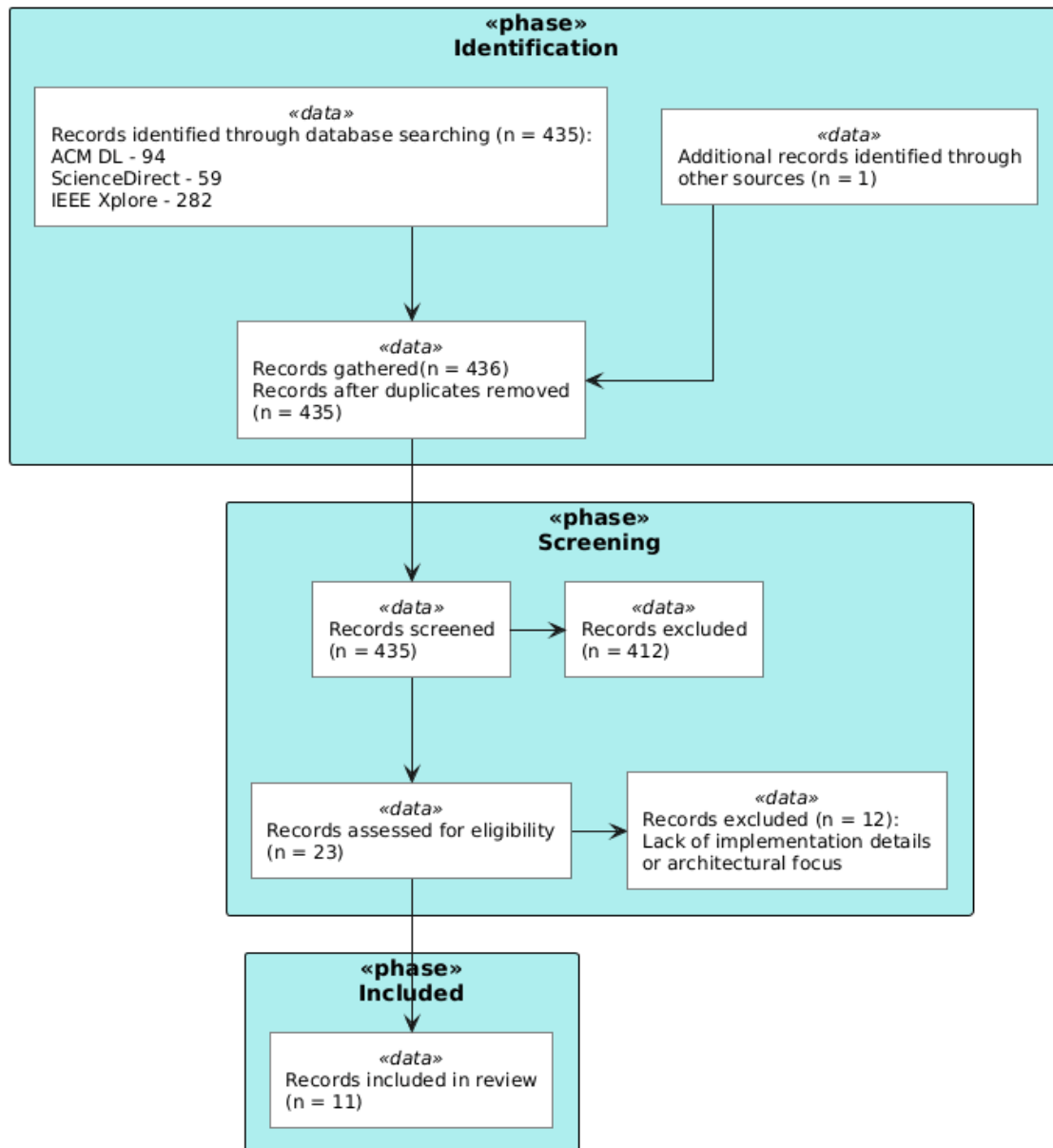


Figura 2.1: Fluxo PRISMA para a LRRQ1

O rigor na filtragem aplicada, reduzindo de 435 para 11 estudos incluídos, garantiu que apenas trabalhos com arquiteturas de sistema relevantes, detalhadas e validadas fossem analisados, assegurando a qualidade e relevância da revisão de literatura para fundamentar as decisões arquiteturais do FUSE.

2.2.2 Discussão

A análise dos 11 estudos incluídos revela uma convergência clara na literatura: a integração direta de dispositivos IoT heterogêneos sem uma camada intermediária resulta em sistemas monolíticos e de difícil manutenção. A diversidade de protocolos (Real Time Streaming Protocol (RTSP), Hypertext Transfer Protocol (HTTP), Message Queuing Telemetry Transport (MQTT)), algoritmos de codificação de vídeo (H.264, H.265, MJPEG) e especificidades de

fabricantes exige, invariavelmente a implementação de uma camada de abstração que atue como tradutor universal. Como observado no estudo [16], a ausência desta camada força o sistema central a lidar com a complexidade de hardware, violando princípios de desacoplamento. A literatura valida que a solução reside numa arquitetura que isole a lógica de negócio das particularidades dos dispositivos.

Padrões Arquiteturais

A evolução dos padrões arquiteturais na literatura aponta para um distanciamento de *gateways* simples em direção a microserviços inteligentes. Inicialmente, soluções como o *IoT M2B* [17] e *Smart Gateway* [16] validaram o uso de um ponto central para tradução de protocolos (ex: converter MQTT/Constrained Application Protocol (CoAP) para HTTP). No entanto, o trabalho de [17] identifica uma limitação crítica: a normalização para HTTP POST, embora eficaz para envio de leituras de um sensor, introduz um gargalo de desempenho inaceitável para *streaming* de vídeo contínuo.

A resposta a esta limitação encontra-se na adoção de microserviços com o padrão Mediator, como proposto no estudo [18]. Esta abordagem introduz uma Camada de Abstração de Dados dedicada que desacopla a lógica de serviço da comunicação com o dispositivo. Mais relevante para o conceito do FUSE é a introdução do padrão "Access Mapper" ou "Digital Twin" pelo trabalho [19]. Neste modelo, a aplicação central interage exclusivamente com um "Dispositivo Virtual" padronizado, tornando-se completamente agnóstica ao hardware físico. Este é o fundamento teórico para um componente da solução a desenvolver, ou seja, uma camada que encapsula a complexidade do driver proprietário e expõe uma interface limpa e uniforme.

Abstração de Media e Normalização de Streaming

A abstração em sistemas de videovigilância exige também a normalização do próprio *streaming* de vídeo. A conexão direta entre uma aplicação web e câmaras de vigilância apresenta desafios técnicos significativos devido à natureza dos fluxos de vídeo brutos e algoritmos de codificação de vídeo utilizados. A literatura, nomeadamente o estudo [20], suporta a implementação de uma camada intermédia responsável pelo tratamento do *input* e gestão do *streaming*, expondo-os posteriormente num formato padronizado e otimizado para consumo web (como HTTP Live Streaming (HLS) ou Web Real-Time Communication (WebRTC)).

Esta abordagem desacopla a aplicação cliente da complexidade de conexão direta às câmaras: o cliente consome sempre um fluxo normalizado a partir da camada de abstração. Embora em [20] e [21] se discuta também o processamento semântico destes fluxos em Edge (análise de vídeo), este tópico específico de automação e visão computacional será aprofundado na Secção 2.4. Para a LRRQ1, o contributo essencial é a arquitetura que garante a estabilidade e abstração do *streaming*.

Escalabilidade e Adaptabilidade à Rede

A viabilidade futura do sistema depende da sua capacidade de escalar para novos dispositivos e adaptar-se a redes instáveis. O estudo [16] demonstra a importância da descoberta automática de dispositivos, tecnicamente designada por *service discovery*, para detetar novas câmaras sem configuração manual, um requisito essencial para a usabilidade. No que toca à rede, o artigo [22] introduz o conceito vital de "Latency-Accuracy Trade-off", onde

o middleware ajusta dinamicamente parâmetros (como a resolução do vídeo) face à congestão da rede. A validação empírica desta topologia distribuída é fornecida pelo estudo [23], que comprova uma redução de latência de até 98% em arquiteturas "Edge Computing" comparativamente à Cloud pura para serviços críticos. Adicionalmente, o trabalho [24] demonstra as vantagens de uma arquitetura baseada em cadeias de módulos independentes. Esta modularidade permite que componentes distintos (como a o processamento e gestão do *streaming*, a transcodificação e a análise do mesmo) operem de forma independente e encadeada. Para o FUSE, isto significa que é possível atualizar ou substituir módulos individuais sem comprometer a estabilidade do sistema base.

Análise Crítica

Apesar da diversidade de soluções, a revisão identifica lacunas relevantes para o contexto do FUSE. Primeiro, a maioria das arquiteturas foca-se em sensores e transmissão de leituras dos mesmos, negligenciando os requisitos específicos de largura de banda do vídeo [17]. Adicionalmente, embora o estudo [23] comprove a baixa latência em Edge, a sua dependência de hardware compatível com Time-Sensitive Networking (TSN) torna a solução impraticável para ambientes domésticos com equipamentos de rede convencionais, exigindo abordagens de software mais flexíveis.

Segundo, embora existam padrões sintáticos, o estudo [25] alerta para a falta de "Interoperabilidade Pragmática", ou seja, a capacidade dos dispositivos comunicarem a intenção do dado (ex: urgência de um alerta) e não apenas o conteúdo. O trabalho de [15] valida a viabilidade técnica de usar plataformas open-source como o ZoneMinder em hardware modesto (Raspberry Pi), mas a ausência dessa camada semântica e pragmática limita a capacidade do sistema reagir contextualmente e expandir o leque de câmaras suportadas de forma escalável. Isto reforça a necessidade da arquitetura distribuída, eficiente e modular proposta para o FUSE.

Esta camada tem como responsabilidades críticas: (1) normalizar a comunicação com dispositivos heterogêneos para uma interface agnóstica [17, 16], (2) abstrair a complexidade do hardware físico através de representações virtuais [19], e (3) garantir a gestão unificada do *streaming* de vídeo [20]. A adoção de uma estrutura modular, em detrimento de uma abordagem monolítica rígida, assegura que o sistema possa acomodar novos protocolos e dispositivos futuramente [18], garantindo que a aplicação central permaneça desacoplada da volatilidade do hardware e assegurando a sua escalabilidade a longo prazo.

2.3 LRRQ2 - Protocolos de Comunicação Segura e VPN

Nesta secção, a literatura é revista de forma a responder à questão de investigação "Qual o estado da arte em protocolos de comunicação segura e VPNs para acesso remoto e *streaming*".

2.3.1 Processo de Pesquisa

A questão de investigação foi estruturada de acordo com o modelo PICOCS apresentado na Tabela 2.3. Esta inclui os critérios de inclusão e exclusão (I/E) relacionados com cada parte da questão de investigação, bem como as possíveis *sub-strings* utilizadas para conduzir a pesquisa nas bases de dados científicas. Apenas estudos de 2018 em diante que contemplem

protocolos de comunicação segura, VPNs ou mecanismos de túnel para acesso remoto e *streaming* foram considerados.

Tabela 2.3: Modelo PICOCS para a LRRQ2

PICOCS	Parte da RQ	I/E	Sub string
P	Estudos envolvendo acesso remoto a dispositivos em redes não controladas	I - Estudos que considerem redes não controladas E - Anteriores a 2018	>=2018; Remote access; Uncontrolled networks; Hostile networks
I	Protocolos de comunicação segura, VPNs e mecanismos de túnel	I - Estudos sobre protocolos de segurança e túneis E - Soluções proprietárias fechadas	Secure protocols; VPN; Tunneling; Encryption; Secure communication
C	—	—	—
O	Estado da arte em protocolos e desempenho de VPNs	I - Estudos que analisem ou comparem protocolos I - Análises de desempenho	Protocol comparison; VPN performance; Security analysis; Streaming protocols
C	Comunicações de rede e segurança de sistemas distribuídos	I - Acadêmico I - Indústria	Network security; Distributed systems; Secure streaming
S	Estudos de caso, análises comparativas e propostas de protocolos	I - Case Studies I - Comparative analysis E - Opiniões sem validação	Case study; Comparative study; Protocol proposal; Performance evaluation

A pesquisa foi conduzida em três bases de dados principais: IEEE Xplore (62 registros), ACM Digital Library (15 registros) e ScienceDirect (2 registros), totalizando 79 registros identificados através de pesquisa sistemática. Adicionalmente, foi identificado 1 registro através de outras fontes, o mesmo identificado para a outra questão [15], resultando num total de 80 registros na fase de identificação.

Após a verificação de duplicados, não foram encontrados registros duplicados, mantendo-se os 80 registros para análise na fase de *screening* através da leitura de títulos e resumos. Desta análise, 52 registros foram excluídos por não cumprirem os critérios de inclusão, nomeadamente por serem irrelevantes para o tema de VPNs, protocolos seguros ou por falta de foco em acesso remoto. Os 28 registros restantes foram avaliados em texto completo, dos quais 9 foram excluídos por tecnologia obsoleta, falta de detalhes de implementação ou por estarem fora do âmbito do estudo. O processo resultou na inclusão final de 19 estudos na revisão qualitativa, conforme ilustrado na Figura 2.2.

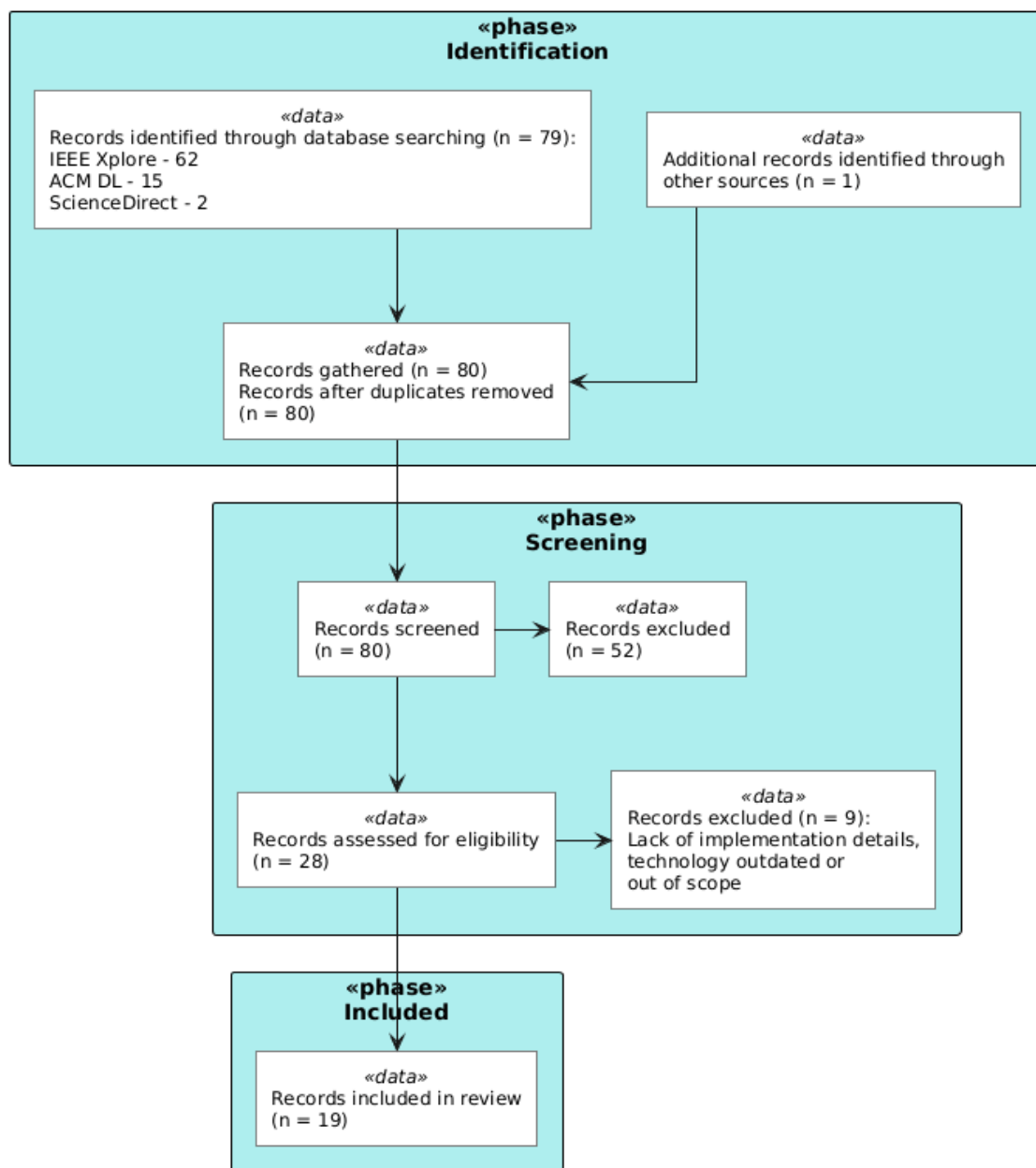


Figura 2.2: Fluxo PRISMA para a LRRQ2

O rigor na filtragem aplicada, reduzindo de 80 para 19 estudos incluídos, garantiu que apenas trabalhos com protocolos de comunicação segura relevantes, detalhados e validados fossem analisados. Comparativamente à LRRQ1, que identificou 435 registros resultando em 11 estudos incluídos (2,5% de taxa de aproveitamento), a presente questão apresentou uma taxa de aproveitamento significativamente superior (23,8%), indicando que a literatura sobre protocolos de comunicação segura e VPNs apresenta uma maior concentração de artigos com conteúdo interessante e utilizável para fundamentar as decisões de segurança e comunicação do FUSE.

2.3.2 Discussão

A análise dos 19 estudos incluídos permitiu traçar uma evolução clara nas tecnologias de acesso remoto seguro, partindo de soluções *legacy* e centralizadas para arquiteturas modernas, descentralizadas e baseadas em túneis de alta performance. Esta subsecção sintetiza as descobertas literárias, organizando-as em quatro vetores fundamentais para o desenho da solução FUSE.

O Desafio da Conectividade em Redes Não Controladas

Um consenso transversal na literatura é a impraticabilidade e insegurança da exposição direta de dispositivos IoT à Internet. O estudo [26] demonstra que o método tradicional de "Port Forwarding" expõe vulnerabilidades críticas de firmware, defendendo o uso de túneis como única defesa viável. Contudo, a criação destes túneis enfrenta barreiras estruturais nas redes modernas. A investigação [27] valida o problema do Carrier-Grade NAT (CGNAT) em redes móveis (4G), onde os Internet Service Providers (ISPs) não alocam endereços Internet Protocol (IP) públicos aos dispositivos, tornando impossível o acesso direto. A solução validada em [28] reside na inversão do modelo de conexão: é o dispositivo em Edge que deve iniciar o túnel para um ponto de encontro externo, ou utilizar redes P2P (como o Tailscale) para garantir a travessia de NAT sem configurações complexas de firewall.

Limitações dos Protocolos Legacy e Comparação TCP vs. UDP

A literatura estabelece protocolos como OpenVPN e IPSec como referências sólidas para a transmissão de dados de sensores [29, 30]. No entanto, a sua aplicação a fluxos de vídeo em tempo real apresenta limitações severas. O artigo [31] destaca o problema crítico do "TCP-over-TCP meltdown": o encapsulamento de tráfego de vídeo (que beneficia da natureza *fire-and-forget* do User Datagram Protocol (UDP)) dentro de túneis baseados em Transmission Control Protocol (TCP) (comuns em VPNs baseadas em Secure Sockets Layer (SSL)) provoca um ciclo vicioso de retransmissões redundantes, resultando em latência exponencial sob redes instáveis. Para além da performance, o estudo [32] alerta para a vulnerabilidade de segurança: o tráfego OpenVPN possui padrões de dados identificáveis, como tamanho de pacotes e tempos de resposta, permitindo que ISPs reconheçam e bloqueiem a conexão, mesmo quando encriptada.

A Ascensão do WireGuard e Túneis de Alta Performance

Como resposta às limitações supracitadas, o protocolo WireGuard emerge na literatura analisada como o novo estado da arte para comunicações seguras em dispositivos com recursos limitados. Os trabalhos [33] e [34] fornecem evidência empírica de que o WireGuard introduz uma latência mínima (na ordem dos microsegundos) e um *overhead* de processamento significativamente inferior aos antecessores. A sua eficiência em hardware *low-cost* (como Raspberry Pi) é validada no estudo [35], tornando-o ideal para a arquitetura Edge do FUSE. Mais relevante ainda, [15] valida a performance do WireGuard em Single Board Computer (SBC) para streaming de vídeo, embora utilize uma topologia de rede dependente de *port forwarding* que o FUSE pretende superar. O artigo [36] reforça a abordagem de evolução para arquiteturas descentralizadas, onde a comunicação ocorre diretamente entre o produtor e o consumidor de dados (P2P), demonstrando ser significativamente mais eficiente do que modelos que dependem de um servidor central, apresentando ganhos de desempenho na ordem de 3x a 5x.

Análise Crítica

A síntese dos estudos permite validar as decisões arquiteturais do FUSE por oposição às abordagens tradicionais. A literatura suporta a implementação de um "Secure Gateway" em Edge [37, 38] como ponto central de segurança. Embora estes estudos demonstrem o potencial desta arquitetura para a implementação de *pipelines* de Continuous Integration/Continuous Deployment (CI/CD) e atualizações de firmware (Firmware Over-the-Air (FOTA)), a presente dissertação foca-se exclusivamente na transmissão segura de vídeo, considerando a gestão complexa de frota um tema adjacente que, pela sua complexidade, supera o âmbito deste trabalho, apesar de tecnicamente viável na arquitetura proposta. Contrariando a intuição de que a encriptação degrada a performance, o estudo [39] sugere que o encapsulamento em túneis modernos pode, de facto, melhorar a qualidade de experiência em redes instáveis.

Relativamente ao risco de deteção por Deep Packet Inspection (DPI) levantado em [32], o FUSE assume uma posição pragmática: embora a ofuscação total seja desejável, esta impõe frequentemente penalizações severas ao débito de dados. Assim, o projeto opta pelo WireGuard para priorizar a performance e a fluidez do vídeo em detrimento de técnicas de ofuscação pesada, assumindo um compromisso calculado entre latência mínima e furtividade absoluta.

Importa referir que quatro dos estudos selecionados na filtragem PRISMA não foram aprofundados na discussão principal, servindo antes como "Baseline da Indústria" que contextualiza a inovação do FUSE. Enquanto a literatura tradicional valida abordagens baseadas em *fog computing* com *blockchain* [40] ou arquiteturas centralizadas clássicas (IPSec/DMVPN) [41], o estado da arte aponta para soluções mais leves e descentralizadas. Similarmente, os estudos [42] e [43] documentam o uso de OpenVPN para sistemas *legacy* (SCADA) e a sua otimização, reforçando a prevalência histórica destas soluções que o presente projeto opta por superar através da adoção integral do WireGuard.

Concretamente, a arquitetura do FUSE materializa-se na disponibilização de cada câmara (ou conjunto de câmaras) em conjunto com um **SBC**, especificamente um Raspberry Pi. Este dispositivo atuará como um gateway de rede, controlando o tráfego e estabelecendo o encaminhamento através de uma VPN gerida pelo **Tailscale**. Embora o Tailscale utilize um servidor de coordenação centralizado (SaaS) como prova de conceito, a arquitetura modular permite a transição futura para implementações *self-hosted* (como o Headscale) para garantir a total posse e confidencialidade dos dados. Desta forma, o servidor central consegue agregar e controlar o *streaming* de vídeo de todas as câmaras dispersas, garantindo a segurança sem expor os dispositivos diretamente à rede pública.

2.4 Estado da Arte em Visão Computacional

Relativamente à terceira sub-questão, a RQ1.3, referente à automatização da análise de vídeo, optou-se por uma abordagem de Revisão Narrativa e Exploratória, como descrita por Maria J. Grant e Andrew Booth em [44] como "State-of-the-art review". Esta modalidade metodológica caracteriza-se pela sua flexibilidade na análise crítica da literatura atual, permitindo identificar conceitos-chave, padrões arquiteturais e soluções técnicas emergentes sem a rigidez protocolar de uma revisão sistemática e formal.

Esta opção justifica-se pela vertiginosa evolução dos modelos de AI e pela necessidade de analisar não apenas literatura académica clássica, mas também documentação técnica de

modelos recentes e *benchmarks* da indústria. A análise encontra-se segmentada em três domínios fundamentais que correspondem às fases da *pipeline* de processamento proposta para o FUSE:

1. Detecção de Movimento (Fase 1);
2. Detecção de Objetos (Fase 2);
3. Visual-Language Models - Vision-Language Models (VLMs) (Fase 3).

Apesar da natureza exploratória, esta pesquisa manterá o rigor científico na seleção de fontes, priorizando publicações de menos de 1 ano para os domínios de evolução rápida (Fase 2 e Fase 3), dado o exponencial e recente crescimento tecnológico da área, e repositórios open-source com forte validação comunitária. A pesquisa foi orientada pelas seguintes *keywords*, agrupadas por domínio:

- **Fase 1 (Pré-processamento):** Background Subtraction, Motion Detection Algorithms, Frame Differencing efficiency, Video Activity Detection.
- **Fase 2 (Classificação):** Real-time Object Detection, You Only Look Once (YOLO) architecture, One-stage detectors, Convolutional Neural Network (CNN) inference optimization, Edge AI.
- **Fase 3 (Extração de Atributos):** Vision-Language Models (VLMs), Multimodal AI, Zero-Shot Learning, Open-vocabulary detection, Visual Question Answering.

2.4.1 Processamento em Edge vs Centralizado

A arquitetura de sistemas de videovigilância inteligentes tem oscilado historicamente entre o processamento em Edge (Edge Computing) e na nuvem/servidor central (Cloud Computing). Embora a tendência recente favoreça o *Edge AI* (execução de modelos de AI diretamente no dispositivo em Edge) para reduzir a latência, a literatura mais atual revela também barreiras intransponíveis para a aplicação de modelos de AI avançados em hardware *legacy* ou de baixo custo.

O estudo [45] demonstra extensivamente que a execução de modelos de visão computacional em Edge exige recursos de memória consideráveis. Mesmo modelos computacionalmente eficientes como o MobileNet requerem mais de 1GB de memória para processar imagens de resolução média (512x512) a 15 Frames Per Second (FPS), o que inviabiliza a sua utilização em câmaras IP comuns ou dispositivos IoT sem hardware dedicado. Complementarmente, o trabalho [46] validou experimentalmente que dispositivos Edge populares (como Raspberry Pi), mesmo utilizando algoritmos de baixa complexidade, introduzem latências de reconhecimento na ordem dos 10.5 segundos, um valor inaceitável para aplicações de segurança em tempo real.

Em contrapartida, o estudo [47] propõe e valida uma arquitetura centralizada baseada em *Cloud/Server*, demonstrando que esta abordagem não só permite a execução de modelos mais complexos, como resulta numa redução de 38% no custo total de propriedade (Total Cost of Ownership (TCO)) e num aumento de 2.5x na capacidade de processamento face a sistemas tradicionais descentralizados. Estes dados fundamentam a decisão arquitetural do FUSE de rejeitar o processamento pesado em Edge, optando por uma centralização inteligente.

2.4.2 Detecção de Movimento e Filtragem Temporal (Fase 1)

Dada a inviabilidade de processar continuamente streams de vídeo de alta resolução com modelos de Deep Learning computacionalmente exigentes, a primeira fase da *pipeline* impõe-se como um mecanismo de filtragem eficiente. A literatura valida a estratégia de "Event-Driven Surveillance", onde algoritmos de baixa complexidade computacional de detecção de movimento atuam como "portão" de entrada para o sistema. Por outras palavras, apenas ser processado determinado vídeo correspondente a uma porção de uma *stream* que resultou da detecção de movimento na mesma.

O estudo [48], embora de 2019, permanece relevante dada a maturidade das técnicas de filtragem de movimento, demonstrando que uma arquitetura orientada a eventos, utilizando algoritmos simples de subtração de fundo ou diferenciação de frames, seja em Edge ou no servidor, permite reduzir o consumo de Central Processing Unit (CPU) em cerca de 60% e poupar mais de 99% em largura de banda e armazenamento em comparação com a transmissão contínua. Esta abordagem, chamada de "Temporal Pruning", também defendida pelo estudo mais recente [45] como estado da arte em eficiência, valida a Fase 1 do FUSE: o uso de métodos clássicos e computacionalmente baratos para eliminar frames estáticos, garantindo que os recursos do servidor, nomeadamente a(s) Graphics Processing Unit (GPU), são alocados exclusivamente a segmentos de vídeo com atividade relevante.

2.4.3 Detecção e Classificação de Objetos (Fase 2)

Para a análise dos segmentos de vídeo filtrados, a necessidade de processamento em tempo real exige modelos de detecção de objetos que maximizem o compromisso entre velocidade de inferência e precisão. A família de modelos YOLO mantém-se como a referência indisputada neste domínio [49].

Estudos comparativos recentes, como o estudo [50], destacam o **YOLOv11**, lançado no final de 2024, como a evolução ideal para ambientes de servidor de alto desempenho. Na sua avaliação, o YOLOv11 superou significativamente as versões anteriores (v8 e v9), atingindo tempos de inferência de apenas 7.7ms por imagem (quase o dobro da rapidez do YOLOv8, que registou 15.9ms) mantendo a maior precisão média (mAP@0.5¹ de 93,4%). Estes resultados são suportados pelos relatórios técnicos oficiais da Ultralytics [51], que indicam que o YOLOv11m atinge um novo estado da arte ao oferecer maior precisão com 22% menos parâmetros que o seu predecessor. Esta redução de complexidade computacional é crítica para a escalabilidade do FUSE, permitindo processar múltiplos streams simultâneos num único servidor.

2.4.4 Modelos de Visão-Linguagem (VLMs) e Identificação Alvo (Fase 3)

A detecção de objetos clássica (Fase 2) identifica a presença de entidades genéricas (e.g., "carro vermelho"), mas é insuficiente para confirmar atributos específicos definidos pelo utilizador (e.g., "matrícula 43-RD-45"). A fronteira da investigação situa-se na integração de *Vision-Language Models* (VLMs) como uma camada de verificação semântica em cascata.

O estudo [52] argumenta que a simples detecção falha em distinguir instâncias específicas devido a oclusões ou semelhanças visuais. Para resolver esta limitação, o FUSE adota uma

¹mAP@0.5 (*mean Average Precision at IoU 0.5*): Métrica que avalia a precisão média do modelo considerando uma detecção válida apenas quando a sobreposição (*Intersection over Union*) com o objeto real é superior a 50%.

arquitetura onde a Fase 3 atua sobre os recortes da Fase 2 para validar atributos textuais ou visuais complexos. O trabalho [53] valida a viabilidade desta abordagem ao demonstrar que modelos da família **Qwen2.5-VL**, equipados com mecanismos de raciocínio, conseguem realizar Optical Character Recognition (OCR) e compreensão de contexto com precisão superior a métodos tradicionais, permitindo, por exemplo, ler uma matrícula ou identificar um logótipo específico.

Complementarmente, o estudo [54] reforça a necessidade de modelos generativos robustos (como o Florence-2) para tarefas de *Grounding*, ou seja, localizar com precisão o atributo alvo dentro da imagem. Esta capacidade de "raciocinar" sobre o conteúdo visual valida a necessidade de hardware de servidor robusto, uma vez que a execução destes modelos exige memória e capacidade de computação muito superiores às de um detetor convencional, sendo inviável em dispositivos Edge.

2.4.5 Armazenamento e Pesquisa Forense

Embora o âmbito do MVP do FUSE privilegie a deteção e notificação de eventos, a revisão da literatura permitiu identificar estratégias emergentes para o armazenamento de dados que potenciam a utilidade forense a longo prazo. A simples acumulação de metadados, sem uma estratégia de indexação semântica, limita a capacidade futura de realizar pesquisas complexas sobre o histórico de eventos.

Neste contexto, a investigação em [55] e [56] aponta para a insuficiência das bases de dados relacionais clássicas face à complexidade dos dados de vídeo modernos. Estes trabalhos validam, respetivamente, o uso de Grafos e de arquiteturas Retrieval-Augmented Generation (RAG) com *Embeddings* Vetoriais como o estado da arte para transformar *outputs* do modelo da Fase 3 da *pipeline* de análise automática em registos pesquisáveis.

A identificação destas tecnologias durante o estudo do estado da arte valida a pertinência da extração de atributos proposta, garantindo que os dados gerados pelo FUSE possuem a riqueza semântica necessária. Tal permite que, numa fase de desenvolvimento posterior ao MVP, o sistema possa evoluir de uma ferramenta reativa de alertas para uma plataforma de investigação forense capaz de suportar pesquisas em linguagem natural.

2.5 Conclusão do Estado da Arte

A revisão sistemática e exploratória da literatura permitiu identificar lacunas críticas e validar a arquitetura proposta para o FUSE em torno de três pilares fundamentais: interoperabilidade, conectividade segura e inteligência artificial distribuída.

Primeiro, relativamente à heterogeneidade de dispositivos (LRRQ1), a literatura [17, 16] demonstra inequivocamente que a integração direta de câmaras IoT é insustentável. A solução validada reside na implementação de uma **Camada de Abstração Modular** baseada em microserviços e no padrão "Digital Twin" [19], que isola a aplicação central da complexidade dos *drivers* proprietários e normaliza o *streaming* de vídeo para formatos web standard [20].

Segundo, no domínio da segurança em redes não controladas (LRRQ2), o estado da arte rejeita as abordagens centralizadas e baseadas em TCP (como OpenVPN) devido à latência excessiva e problemas de "TCP meltdown" [31]. A arquitetura do FUSE alinha-se com a tendência de descentralização, adotando túneis **WireGuard** iniciados em Edge [33, 35] e

topologias P2P (como demonstrado por [36]) para garantir a soberania dos dados e contornar barreiras de NAT sem depender de servidores de retransmissão lentos.

Por fim, a análise das tecnologias de visão computacional confirma a inviabilidade de executar modelos de AI modernos em hardware *legacy* em Edge [45, 46]. Esta limitação valida a decisão arquitetural de centralizar a inteligência: embora a Edge possa atuar teoricamente como um filtro preliminar simples [48], no FUSE tanto a detecção de movimento como a inferência semântica são concentradas no servidor central para simplificar a gestão de dispositivos. Neste servidor, a utilização de modelos de detecção de objetos como o **YOLOv11** fundamenta a concretização da Fase 2 no âmbito do MVP, enquanto o estudo de VLMs, como o Qwen2.5-VL, valida a implementação futura da Fase 3, orientada à extração de atributos específicos (e.g., matrículas) e à evolução para mecanismos de pesquisa semântica. Importa notar que, embora a literatura valide o Qwen2.5-VL, o projeto antecipa a adoção do recém-anunciado **Qwen3.5-VL**, assumindo, até prova em contrário, que a evolução natural da família trará ganhos de desempenho e raciocínio fundamentais para o FUSE.

Capítulo 3

Análise e Desenho

Este capítulo encontra-se organizado em duas partes complementares: a análise e o desenho da solução proposta para o FUSE. A primeira parte centra-se na compreensão e delimitação do problema a resolver, definindo o âmbito do MVP, os requisitos funcionais e não funcionais e os principais conceitos representados no modelo de domínio. A segunda parte traduz essa análise numa proposta arquitetural, descrevendo a organização da solução, a utilização de mecanismos de conexão segura, os componentes principais, a infraestrutura de suporte e as interações entre os diferentes serviços que compõem a plataforma.

3.1 Análise

Na presente secção irá ser analisada a solução proposta, clarificando o que será considerado no âmbito do MVP. Também são descritos os requisitos funcionais e os requisitos não funcionais, que definem comportamentos esperados, restrições e propriedades de qualidade relevantes. Por fim, é introduzido o modelo de domínio, com o objetivo de identificar os principais conceitos, entidades e relações do contexto aplicacional antes da definição da arquitetura técnica da solução.

3.1.1 Âmbito do MVP

O MVP definido para o FUSE tem como objetivo validar, de forma experimental, os principais blocos arquiteturais propostos para responder às questões de investigação: a integração unificada de câmaras tecnologicamente heterogêneas, o acesso seguro a dispositivos em redes externas não controladas e a automatização inicial da análise de vídeo. Assim, o MVP não pretende constituir uma versão final de produção da plataforma, mas sim um protótipo funcional que demonstre a viabilidade técnica da solução proposta.

No âmbito deste MVP serão consideradas, no mínimo, duas câmaras tecnologicamente distintas, expostas através de uma camada de abstração comum. Esta camada deverá permitir normalizar operações essenciais, nomeadamente a visualização em tempo real, o controlo da câmara, denominado de Pan-Tilt-Zoom (PTZ), a gravação e o *playback* das gravações. A comunicação entre as câmaras instaladas remotamente e o servidor central será realizada através de um SBC, como um Raspberry Pi, atuando como *gateway* através de um túnel seguro, neste caso com a ferramenta Tailscale (baseada em Wireguard).

Durante o desenvolvimento e validação, será utilizada a infraestrutura de coordenação disponibilizada pelo Tailscale no seu plano gratuito, operada em modelo SaaS pela entidade responsável pela tecnologia. Esta opção é assumida como solução prática de desenvolvimento de protótipo, permitindo validar a conectividade segura sem introduzir, nesta fase,

a complexidade operacional associada ao alojamento próprio do servidor intermediário da VPN. Num cenário final de implementação, esta dependência deverá ser substituída por uma instância de Headscale, uma solução *open-source* e *self-hosted* do servidor de controlo do Tailscale, alojada em infraestrutura própria [57].

A componente de análise automática de vídeo será validada até à Fase 2 da *pipeline* proposta. Isto inclui uma primeira fase de deteção de movimento e uma segunda fase de deteção de objetos, recorrendo a modelos de visão computacional adequados ao cenário experimental. Esta delimitação permite avaliar a utilidade da plataforma na redução do esforço de análise manual, sem comprometer o trabalho com funcionalidades mais avançadas que exigiriam uma validação mais extensa e um desenvolvimento alongado.

Ficam fora do âmbito imediato do MVP a Fase 3 da *pipeline*, relativa à extração de características através de VLMs, a descoberta automática de câmaras e mecanismos de pesquisa forense avançada sobre o histórico de eventos. Estas funcionalidades são consideradas extensões futuras da arquitetura.

Deste modo, o desenvolvimento concentra-se na validação dos elementos essenciais do FUSE: conectividade segura, abstração de câmaras heterogéneas, disponibilização centralizada de vídeo e deteção automática inicial de eventos. As subsecções seguintes detalham os requisitos e decisões arquiteturais que concretizam este âmbito.

3.1.2 Modelo de Domínio

No modelo de domínio representado na Figura 3.1, observa-se os conceitos fundamentais do sistema, incluindo as entidades, as relações entre elas e os principais atributos de negócio. Trata-se, portanto, de um modelo de domínio conceptual, e não de um modelo lógico ou de dados.

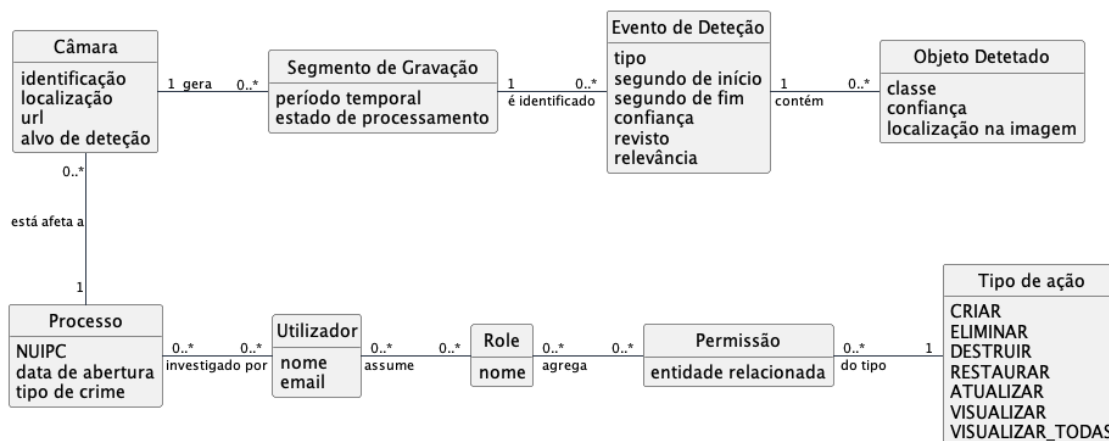


Figura 3.1: Modelo de Domínio Conceptual

A conceptualização da plataforma encontra-se dividida em três grupos semânticos principais: a gestão de processos e controlo de acessos, a gestão de dispositivos e gravação de vídeo, e a análise automática.

Gestão de Processos e Controlo de Acessos

O conceito agregador do domínio operacional é o **Processo**, que representa um Processo de investigação criminal. Este é inequivocamente identificado pelo seu Número Único de Identificação de Processo Crime (NUIPC), e é caracterizado pela sua data de abertura e tipo de crime.

Um **Utilizador** participa em múltiplos processos e, inversamente, um processo pode ter vários investigadores (utilizadores) associados, sendo o objetivo pretendido que um utilizador apenas possa aceder aos processos que lhe foram atribuídos. De forma a suportar um controlo de acessos granular, o modelo de autorização não se baseia num acesso monolítico, mas sim numa estrutura composta por **Role** e **Permissão**. Um utilizador pode assumir várias *roles* (ou perfil) e cada *role* pode agregar múltiplas permissões, permitindo definir perfis de acesso ajustados às responsabilidades de cada interveniente. Contudo, as *roles* funcionam sobretudo como agregadores administrativos de permissões, não substituindo as verificações de autorização realizadas sobre ações e recursos concretos. Assim, as decisões sensíveis de acesso deverão considerar a permissão aplicável, o recurso acedido e o contexto operacional, nomeadamente a associação entre utilizador, processo e câmara. Esta abordagem está alinhada com princípios de Secure by Design, evitando perfis excessivamente permissivos e promovendo validações granulares ao nível da aplicação. A Permissão tem ainda um associado um **Tipo de Ação** (e.g. VISUALIZAR, CRIAR, ELIMINAR, etc.) e um atributo referente à sua entidade relacionada (e.g. Processo, Utilizador, etc.).

Dispositivos e Gravação de Vídeo

A **Câmara** atua como a entidade física e lógica central, afeta a um Processo. Para além da sua identificação e do local de instalação, a câmara é caracterizada pelo seu endereço de acesso (url), bem como pelo alvo de deteção configurado. O vídeo capturado pela Câmara não é tratado como uma entidade contínua infinita, mas sim particionada por segmentos. Assim, uma Câmara gera múltiplos **Segmentos de Gravação**, que consistem em unidades temporais de vídeo persistido. Estes segmentos possuem um período temporal definido que os posiciona temporalmente e um estado de processamento da deteção automática, servindo como base para a posterior análise assíncrona.

Análise Automática e Triagem

Este grupo foca-se na deteção e extração de conhecimento. Um Segmento de Gravação é analisado para identificar **Eventos de Deteção**. Cada evento representa uma ocorrência detetada num determinado instante ou período de tempo, possuindo um grau de confiança gerado pelo modelo de inteligência artificial. Para justificar a ocorrência, o Evento de Deteção contém pelo menos um **Objeto Detetado**, que inclui a sua própria classe, grau de confiança e localização espacial na imagem. Esta localização na imagem será descrita pelo seu nome técnico Bounding Box (BBOX) no decorrer deste documento. Uma vez que a validação da prova exige supervisão humana, o Evento de Deteção contempla atributos adequados para o efeito, nomeadamente o estado de revisão e a sua relevância. Isto permite aos investigadores assinalar quais as deteções automáticas que constituem efetivamente interesse para o Processo, bem como reconhecer facilmente os eventos por rever.

3.1.3 Requisitos

Esta subsecção apresenta os requisitos que orientam o desenho e a validação do MVP. Considerando que a solução proposta assume a forma de um protótipo arquitetural, os requisitos são formulados ao nível das capacidades essenciais a demonstrar, e não como uma especificação exaustiva de produto. Assim, são distinguidos requisitos funcionais e requisitos não funcionais.

Requisitos Funcionais

Os requisitos funcionais identificam as capacidades observáveis que o MVP deve demonstrar para validar a solução proposta. A Tabela 3.1 apresenta o conjunto de requisitos funcionais definidos para orientar o desenho do protótipo.

Tabela 3.1: Requisitos funcionais do MVP

ID	Descrição
RF-01	A solução deve permitir integrar câmaras tecnologicamente heterogêneas, expondo-as através de uma interface funcional comum.
RF-02	A solução deve permitir aceder, através da plataforma central, a câmaras localizadas em redes externas não controladas, sem exigir exposição direta dos dispositivos à Internet e evitando a dependência de mecanismos proprietários de acesso remoto dos fabricantes.
RF-03	A solução deve permitir organizar as câmaras por processos de investigação, garantindo que o acesso e a operação sobre esses recursos são delimitados por utilizador ao âmbito de cada processo.
RF-04	A solução deve disponibilizar visualização em tempo real do <i>streaming</i> das câmaras.
RF-05	A solução deve permitir o controlo remoto de câmaras com capacidades PTZ, quando suportado pelo dispositivo.
RF-06	A solução deve permitir a gravação e reprodução de vídeo, mantendo a associação das gravações à respetiva câmara e contexto operacional.
RF-07	A solução deve permitir a análise automática de segmentos de vídeo até à Fase 2 da <i>pipeline</i> descrita anteriormente, que inclui deteção de movimento e deteção/classificação de objetos, guardando os resultados para verificação humana posterior.

Requisitos Não Funcionais

Os requisitos não funcionais focam-se nas propriedades de qualidade que condicionam diretamente o desenho arquitetural da solução. Foram selecionados apenas os requisitos transversais mais relevantes para validar a proposta do FUSE, distinguindo-os das orientações tecnológicas da entidade de acolhimento, já apresentadas no Capítulo 1. A Tabela 3.2 apresenta os requisitos não funcionais definidos para orientar as principais decisões arquiteturais do MVP.

Tabela 3.2: Requisitos não funcionais do MVP

ID	Categoria	Descrição
RNF-01	Segurança, privacidade e rastreabilidade	A solução deve proteger o acesso remoto às câmaras, evitando a exposição direta dos dispositivos à Internet, garantindo comunicações seguras e mantendo registos de auditoria das ações relevantes realizadas na plataforma.
RNF-02	Implementação <i>self-hosted</i> e soberania dos dados	A solução deve ser concebida para poder ser implementada em infraestrutura controlada pela entidade destinatária ou pelos OPCs, garantindo que o armazenamento e processamento de dados sensíveis permanecem sob controlo da entidade responsável.
RNF-03	Modularidade e extensibilidade	A solução deve ser modular, permitindo a evolução ou substituição independente dos principais componentes, nomeadamente as camadas de abstração de câmaras (<i>media-parser</i> e <i>digital-twin</i>) e a análise automática de vídeo.
RNF-04	Performance operacional adequada ao MVP	A solução deve apresentar performance suficiente para suportar a visualização remota de vídeo, o controlo operacional da câmara e a análise assíncrona de segmentos no contexto experimental do MVP.

Para evitar que estes requisitos permaneçam apenas como declarações abstratas, foram definidos critérios de validação associados a cada requisito não funcional. Alguns destes critérios são qualitativos, por dependerem de evidência arquitetural e de implementação, enquanto outros são quantitativos, por poderem ser medidos no cenário experimental. Estes critérios não representam níveis de serviço finais de uma solução em produção, mas sim limiares e evidências suficientes para avaliar a viabilidade do MVP.

Tabela 3.3: Critérios de validação dos requisitos não funcionais

ID	Tipo de validação	Critério de validação
RNF-01	Qualitativa e experimental	Considera-se satisfeito se o acesso remoto às câmaras for realizado sem exposição direta dos dispositivos à Internet, sem recurso a um servidor do fabricante, e se existirem evidências de registo de ações relevantes, como visualização, controlo, exportação ou operações de gestão.
RNF-02	Qualitativa	Considera-se satisfeito se a solução puder ser implementada em infraestrutura controlada pela entidade final, recorrendo a <i>containers</i> , sem dependência de hardware proprietário específico nem de serviços <i>cloud</i> para armazenamento ou processamento de dados.
RNF-03	Qualitativa	Considera-se satisfeito se os principais componentes comunicarem através de interfaces bem definidas, permitindo a evolução ou substituição independente de módulos como o <i>media-parser</i> , o <i>digital-twin</i> e o detetor. Esta validação deverá ser suportada por separação de responsabilidades, injeção de dependências e ausência de acoplamento forte entre componentes.
RNF-04	Quantitativa	Considera-se satisfeito, no cenário experimental do MVP, se o atraso de uma ponta a outra do <i>streaming</i> se mantiver abaixo de 6 segundos e se um evento detetado pela <i>pipeline</i> automática ficar disponível para consulta ou alerta em menos de 5 minutos após a sua ocorrência no segmento analisado.

3.2 Arquitetura da Solução

Esta secção apresenta a arquitetura conceptual proposta, resultante da análise dos requisitos definidos anteriormente e das conclusões retiradas do estado da arte. O objetivo não é ainda descrever detalhes de implementação, configurações específicas ou decisões tecnológicas de baixo nível, mas sim estabelecer a organização lógica da solução e a forma como os seus principais blocos se articulam para responder ao problema identificado.

Assim, a arquitetura aqui descrita deve ser entendida como uma vista teórica e orientadora do sistema, servindo de ponte entre a análise do domínio e a implementação do MVP. São apresentados os mecanismos de conectividade segura, os componentes responsáveis pela abstração e disponibilização das câmaras, a infraestrutura de suporte ao *streaming* e à gravação, e a integração da análise automática de vídeo. A concretização técnica destes elementos será aprofundada no Capítulo 4.

3.2.1 Conexão Segura aos Dispositivos

Como introduzido anteriormente, para responder ao requisito RF-02, a solução propõe a introdução de uma *gateway* segura, colocada junto da câmara, que permite estabelecer ligações através de um túnel VPN. A Figura 3.2 apresenta uma vista conceptual da proposta da solução para a conectividade segura.

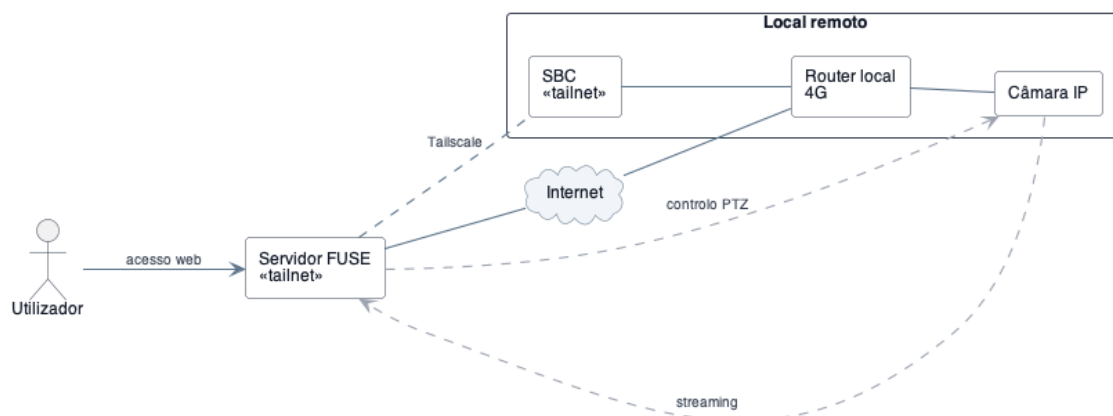


Figura 3.2: Cenário de conectividade segura proposto para o FUSE com Tailscale.

Conceptualmente, a *gateway* segura é representada por um SBC, instalado na mesma rede local da câmara. Em vez de abrir portas no *router* local ou encaminhar o tráfego através de servidores P2P do fabricante, o SBC inicia uma ligação segura para uma rede privada controlada, permitindo que o servidor do FUSE alcance a rede remota e, sucessivamente, a câmara através desse canal estabelecido. Desta forma, a câmara permanece isolada da Internet pública, enquanto o acesso operacional ao seu *streaming* e, quando aplicável, ao controlo PTZ, passa a ser mediado pela infraestrutura da plataforma. O utilizador interage apenas com o servidor FUSE.

Esta solução pressupõe a existência de três elementos no local remoto: a câmara, um *router* 4G e o SBC que atua como *gateway*. Alternativamente, poderia ser utilizado um Raspberry Pi com um *hat* de 4G, acumulando as funções de acesso à Internet e de *gateway* segura. Numa evolução futura, caso a câmara já possua conectividade 4G, poderá ainda ser estudada uma integração mais direta com o seu módulo de comunicação, acoplando os dois componentes através de soldagem, de modo a reduzir hardware adicional e custos. Estas alternativas, contudo, introduzem maior complexidade técnica e ficam fora do âmbito do presente MVP.

3.2.2 Componentes da Solução

Antes de apresentar a vista de infraestrutura e os diagramas de interação, importa clarificar os principais componentes lógicos que compõem a solução e a nomenclatura utilizada ao longo do documento.

No lado do utilizador, a solução é acedida através de uma interface web. Esta constitui o ponto de entrada para consulta de câmaras, gravações e resultados de deteção. No restante

documento, este componente será designado por **browser**. Entre este componente e os restantes componentes internos existe uma entrada responsável por centralizar o acesso à plataforma e encaminhar os pedidos para os módulos adequados, desempenhando a função de *reverse proxy*. Este componente funciona como fronteira aplicacional da solução, evitando que os serviços internos sejam expostos diretamente. No restante documento, este componente será designado por **nginx**.

O núcleo aplicacional da solução é responsável por concentrar a lógica de negócio do FUSE, incluindo a gestão de utilizadores, processos e as restantes entidades de domínio. Este componente coordena as restantes partes da solução e será designado por **app**. Esta informação necessita de ser persistida de forma consistente, de maneiras que, para o efeito, a arquitetura contempla uma base de dados relacional. Esta base de dados será designada por **database**.

A gestão do fluxo de vídeo é isolada num componente próprio, responsável por abstrair a complexidade associada à receção, normalização e disponibilização de *stream*. Este componente serve também de base à gravação segmentada utilizada posteriormente pela análise automática, sendo designado por **media-parser**. A solução necessita de armazenar os ficheiros de vídeo produzidos por este componente. É então necessária uma área de armazenamento que terá que ser alcançada pelos restantes componentes que possam necessitar de acesso aos ficheiros de vídeo. Tecnicamente não se trata de um componente e sim de um volume, e este será designado por **recordings-volume**.

As operações sobre a câmara física são representadas através de uma camada de abstração operacional. Esta camada permite expor as capacidades da câmara de forma normalizada, reduzindo a dependência direta das particularidades de cada modelo. No documento, este componente será designado por **digital-twin**. No ambiente remoto, existe a câmara física responsável pela captação do vídeo, representada por **camera**. Encontra-se também presente o Raspberry Pi introduzido anteriormente, que atua como *gateway* de acesso seguro e será designado por **sbcc**. Por fim, o *router* 4G responsável por disponibilizar conectividade externa ao local remoto será designado por **router**.

A análise automática de vídeo é tratada como um módulo autónomo face ao núcleo aplicacional. Este módulo recebe segmentos de vídeo para análise e executa a *pipeline* de deteção definida para o MVP. No seu conjunto, este módulo será designado por **detector**. Internamente, é composto por um subcomponente responsável por receber pedidos de análise, designado por **detector-api**, por um subcomponente responsável pelo processamento efetivo dos segmentos, designado por **detector-worker**, e por um componente auxiliar de suporte ao processamento assíncrono, designado por **detector-cache**.

Estes componentes são definidos a um nível lógico e não impõem, por si só, uma distribuição física específica. A forma como são agrupados, distribuídos fisicamente e interligados é descrita nas secções seguintes.

3.2.3 Infraestrutura e Conectividade

A Figura 3.3 apresenta uma vista híbrida da solução, combinando a distribuição física dos componentes com as principais interfaces de comunicação entre eles. Esta representação permite compreender não apenas quais os módulos que compõem a arquitetura, mas também onde estes são alojados, através de que interfaces interagem, e quais os protocolos de comunicação definidos em cada interação.

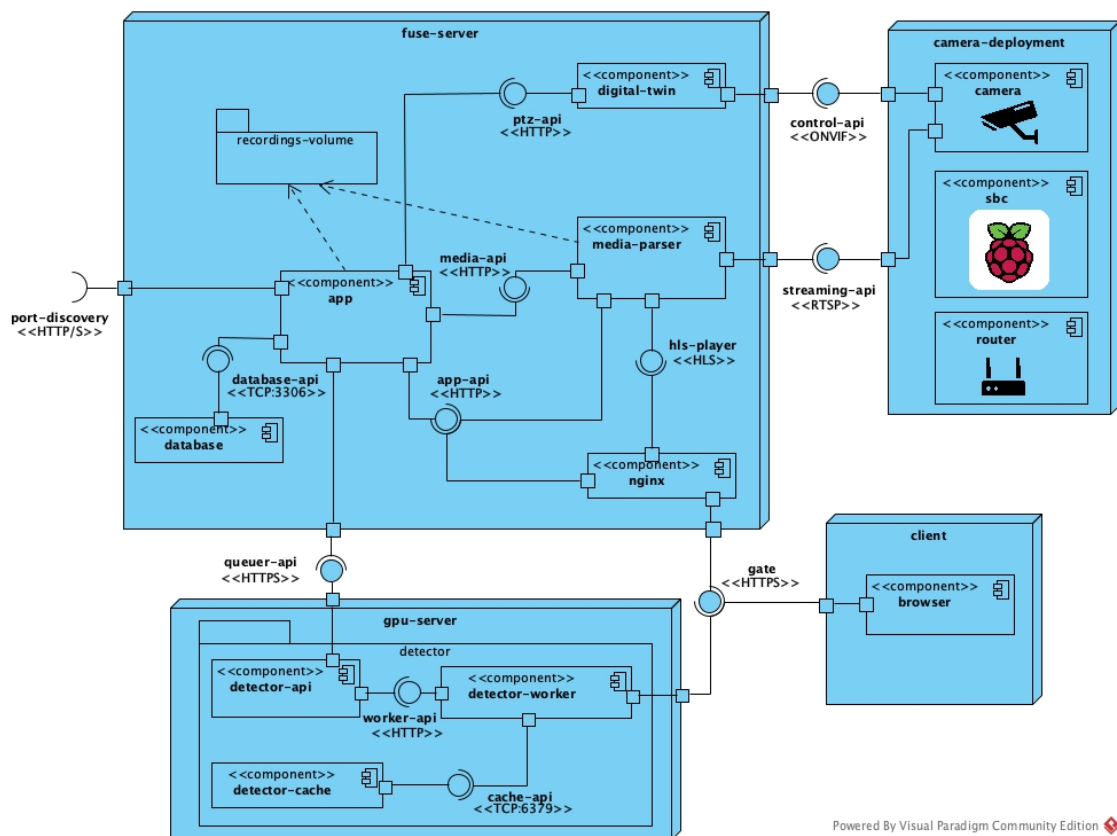


Figura 3.3: Vista híbrida da infraestrutura e conectividade da solução

Ao nível físico, a arquitetura encontra-se organizada em três zonas principais. O **fuse-server** representa o servidor central da plataforma, onde são alojados os componentes *nginx*, *app*, *database*, *media-parser*, *recordings-volume* e *digital-twin*. Esta organização concentra a coordenação principal da solução num ambiente controlado, permitindo que os serviços internos comuniquem entre si sem necessidade de exposição pública direta.

O **gpu-server** isola os componentes associados à análise automática de vídeo. Neste servidor são representados o *detector-api*, o *detector-worker* e o *detector-cache*. A sua separação face ao *fuse-server* permite distinguir a lógica principal da plataforma das tarefas de visão computacional, por poder dispor de hardware gráfico dedicado.

Por fim, o **camera-deployment** representa o ambiente remoto onde a câmara se encontra instalada. Este ambiente inclui a *camera*, o *sbc* e o *router*, correspondendo à extremidade operacional da solução. Para além destas zonas, a figura representa ainda o **client**, correspondente ao ambiente do utilizador onde é executado o *browser*.

A conectividade entre estas zonas é definida através de interfaces explícitas. O *nginx* disponibiliza a interface **gate**, acessível por Hypertext Transfer Protocol Secure (HTTPS), que funciona como ponto controlado de entrada na plataforma e é consumida pelo *browser* para utilização da aplicação, e pelo *detector-worker* quando este, por exemplo, necessita de responder com os resultados da análise. A partir deste ponto, os pedidos aplicacionais são encaminhados para a *app*, que disponibiliza a **app-api** através de HTTP. Esta interface é consumida pelo *nginx* para as suas funções de *proxy* e pelo *media-parser*.

O componente *database* disponibiliza a **database-api** através de TCP na porta 3306, consumida pela *app*. O *media-parser* disponibiliza a **media-api** por HTTP, consumida pela *app*, e disponibiliza ainda a interface **hls-player** por HLS, consumida pelo *nginx* para entrega do *streaming* ao utilizador. A *app* e o *media-parser* utilizam o mesmo *recordings-volume* para armazenar e consultar os ficheiros de vídeo. Por sua vez, o *digital-twin* disponibiliza a **ptz-api** por HTTP, consumida pela *app*.

As interfaces expostas pela câmara são consumidas por componentes distintos, de acordo com a sua responsabilidade. A *camera* disponibiliza a **streaming-api**, consumida pelo *media-parser* para obtenção do fluxo de vídeo, e a **control-api**, consumida pelo *digital-twin* para operações de controlo PTZ. No diagrama, estas interfaces aceitam, respetivamente, os protocolos RTSP e ONVIF. No entanto, as camadas de abstração criadas entre a aplicação e a câmara permitem que outros protocolos sejam suportados futuramente, sem alterações estruturais significativas.

Adicionalmente, a *app* pode recorrer a um processo de **port-discovery** por HTTP/HTTPS, usado para identificar automaticamente as portas disponíveis na câmara instalada.

A comunicação com o servidor de deteção é realizada através do *detector-api*, que disponibiliza a **queue-api** por HTTPS e é consumida pela *app* para submissão de segmentos de vídeo para análise. Internamente, o *detector-worker* disponibiliza a **worker-api** por HTTP, consumida pelo *detector-api*, enquanto o *detector-cache* disponibiliza a **cache-api** através de TCP na porta 6379, consumida pelo *detector-worker*.

Deste modo, esta subsecção define a estrutura estática da infraestrutura e das suas ligações principais. A subsecção seguinte complementa esta vista ao descrever a sequência temporal das interações entre componentes.

3.2.4 Interações entre Componentes

Esta subsecção complementa a anterior, descrevendo as interações sequenciais entre os diferentes módulos apresentados. Enquanto a vista anterior caracteriza a estrutura estática da solução, os diagramas seguintes representam os principais fluxos operacionais do MVP.

Os diagramas apresentados devem ser interpretados como representações conceptuais dos principais fluxos de interação entre componentes. O seu objetivo não é listar todas as chamadas internas ou detalhes de implementação, mas sim evidenciar as responsabilidades dos componentes, os pontos de decisão relevantes e a ordem geral das interações. Assim, alguns passos internos são abstraídos quando não alteram a compreensão arquitetural do processo, sendo os detalhes técnicos de implementação aprofundados no Capítulo 4.

Processo de Streaming

A Figura 3.4 representa o processo de acesso ao *streaming* em direto de uma câmara, desde o pedido inicial de visualização até à entrega contínua dos fragmentos de vídeo ao utilizador. Também aborda o tema da gravação e respetivas interações entre os componentes encarregues da gestão da mesma.

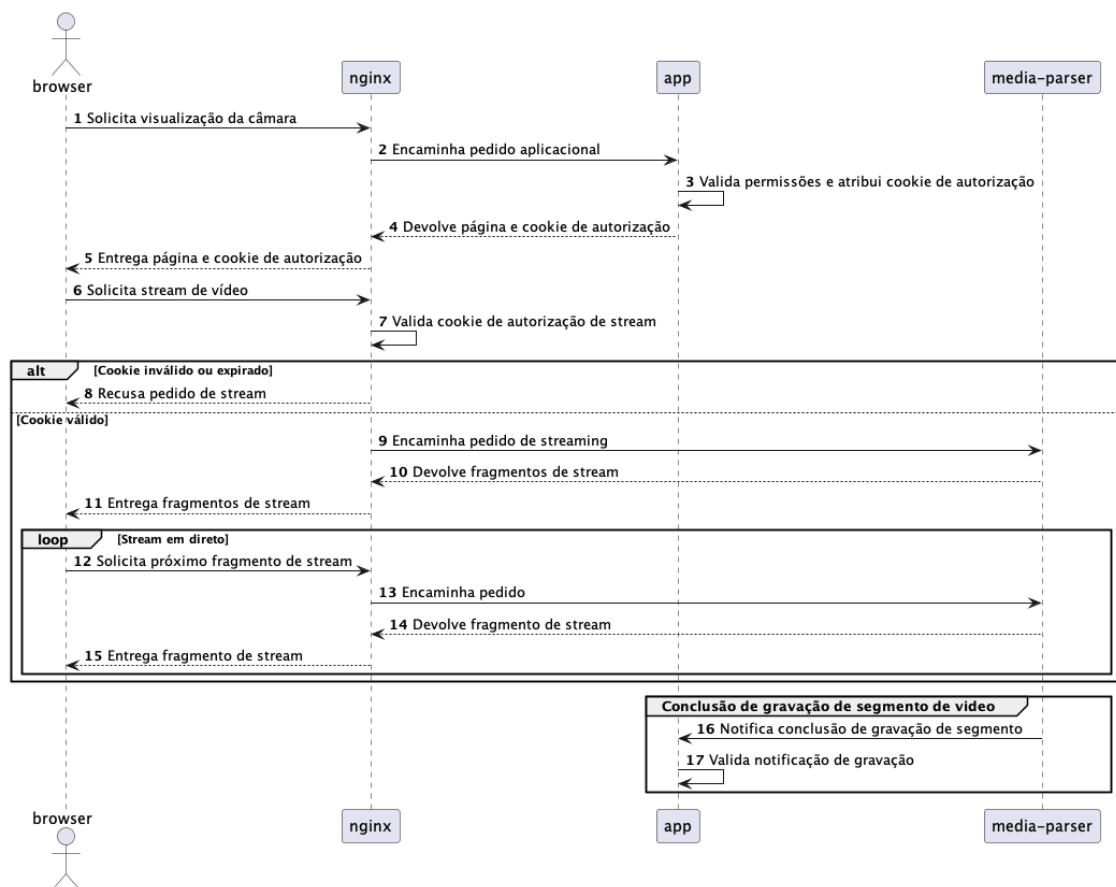


Figura 3.4: Vista de Processo de Streaming

Este fluxo evidencia a separação entre a lógica aplicacional e a entrega do vídeo. O *browser* não acede diretamente ao *media-parser*; em vez disso, todo o acesso passa pelo *nginx*, que atua como ponto controlado de entrada. A *app* valida se o utilizador tem permissões para visualizar a câmara no contexto do processo associado e, quando o acesso é autorizado, emite um mecanismo temporário de autorização utilizado nos pedidos seguintes de *streaming*.

Após essa validação, o *nginx* passa a decidir se os pedidos de fragmentos devem ser aceites ou recusados, encaminhando apenas os pedidos válidos para o *media-parser*. Esta organização evita que o componente responsável pela gestão multimédia tenha de conhecer diretamente o modelo de utilizadores, processos e permissões da aplicação. O ciclo representado no diagrama traduz a natureza contínua do *streaming* em direto. Adicionalmente, a notificação de conclusão de gravação de segmento demonstra a ligação entre a disponibilização do vídeo, a gravação segmentada e os processos posteriores de análise automática. Assim, quando o *media-parser* termina a gravação de um segmento de vídeo, notifica a *app* para que esta possa proceder com os seguintes passos incidentes da gravação.

Processo de Controlo da Câmara

A Figura 3.5 representa o processo de descoberta e execução de comandos de controlo PTZ sobre uma câmara instalada.

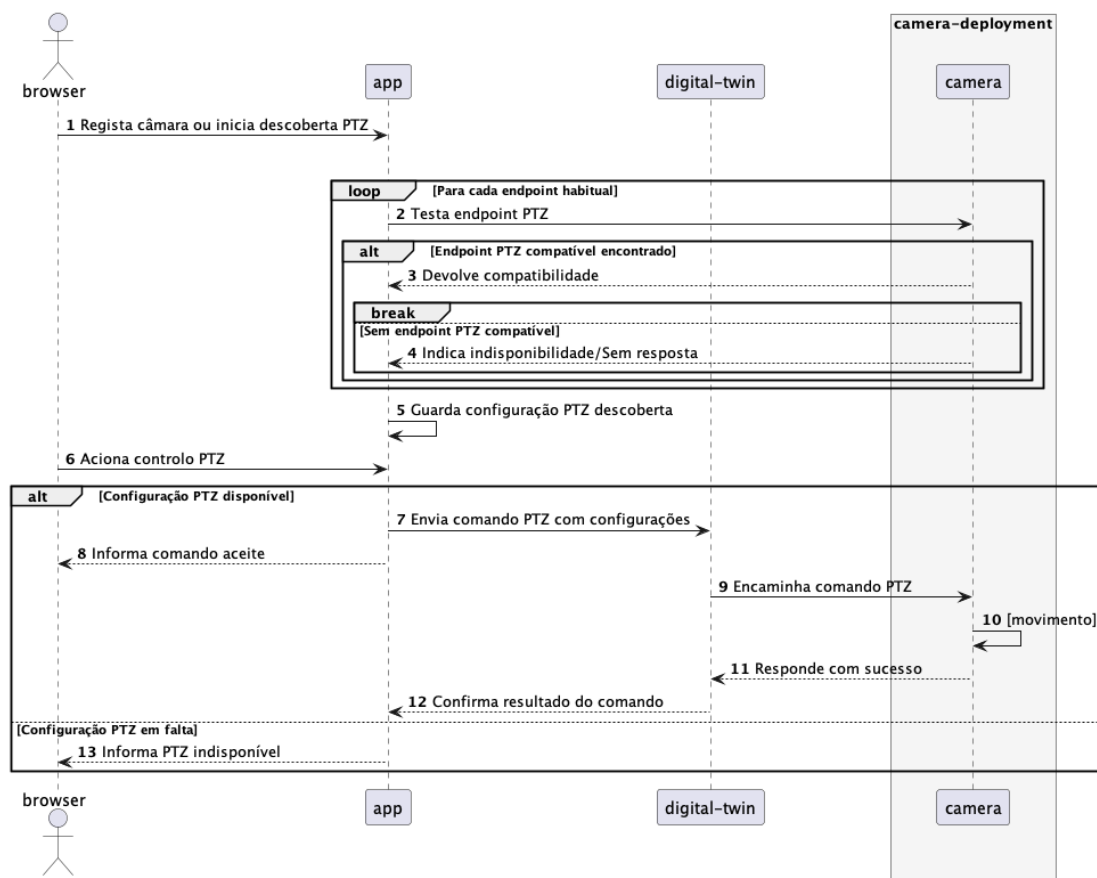


Figura 3.5: Vista do processo de controlo da câmara

Este fluxo encontra-se dividido em dois momentos principais. O primeiro corresponde à descoberta automática de um *endpoint* PTZ compatível, realizada durante o registo ou configuração da câmara. Esta etapa é necessária porque diferentes dispositivos podem disponibilizar o controlo remoto em portas ou interfaces distintas, e porque nem todas as câmaras suportam operações PTZ. A configuração encontrada é posteriormente guardada pela *app*, evitando que este processo tenha de ser repetido antes de cada comando.

O segundo momento corresponde à execução do comando de controlo. O utilizador aciona a operação através do *browser*, mas a câmara continua a não ser controlada diretamente pelo cliente. A *app* verifica se existe configuração PTZ disponível e, caso exista, encaminha o comando e o *endpoint* compatível para o *digital-twin*. Este componente atua como camada de abstração operacional, convertendo a intenção da aplicação num comando adequado à câmara física. Caso a configuração não esteja disponível, a plataforma informa o utilizador de que o controlo PTZ não pode ser utilizado.

Importa notar que a confirmação enviada ao *browser* indica apenas que o comando foi aceite e encaminhado pela plataforma, não significando necessariamente que o movimento físico da câmara já tenha sido concluído. Esta opção evita bloquear a interação do utilizador durante o intervalo que pode existir entre a emissão do comando e a execução efetiva do movimento pelo dispositivo. O resultado da operação permanece, assim, tratado internamente entre o *digital-twin*, a *camera* e a *app*.

Processo de Detecção

A Figura 3.6 representa o processo de detecção automática desencadeado após a gravação de um segmento de vídeo.

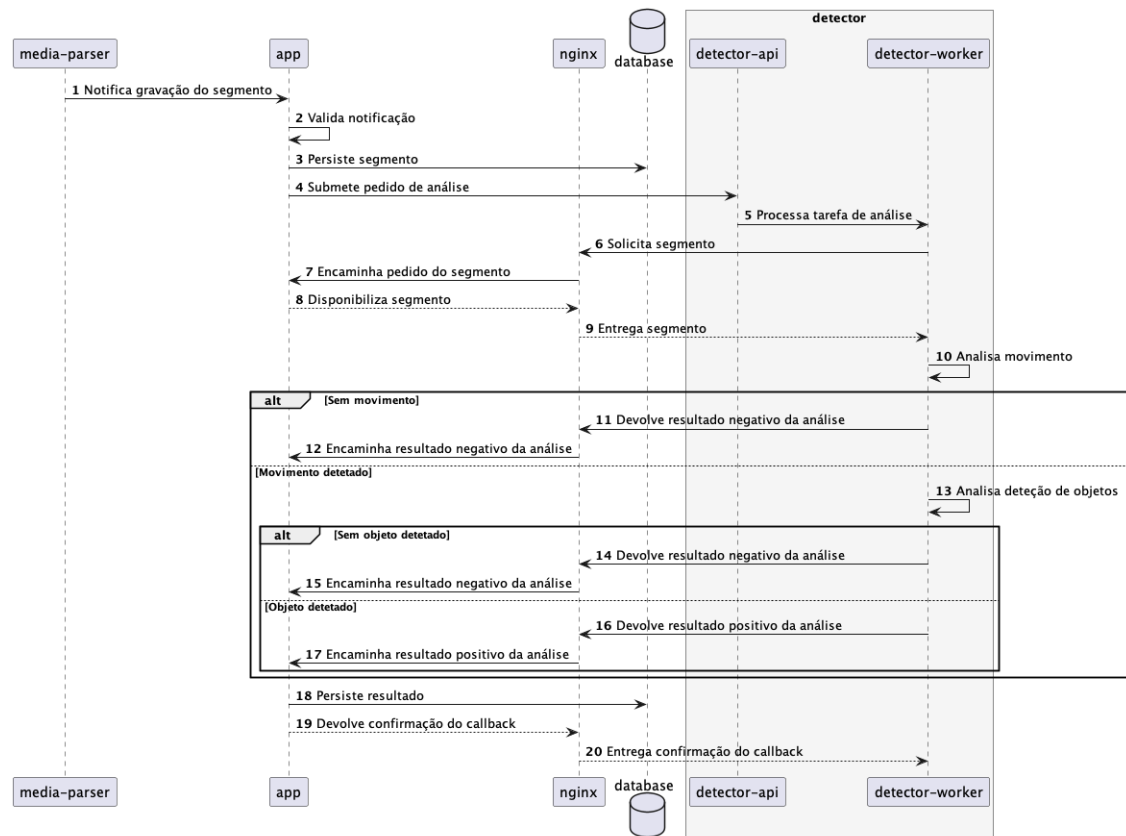


Figura 3.6: Vista do processo de detecção automática

Este fluxo evidencia que a análise automática não ocorre diretamente sobre o *streaming* em direto, mas sim sobre segmentos de vídeo já gravados. Quando o *media-parser* conclui a gravação de um segmento, notifica a *app*, que valida essa notificação, regista o segmento na *database* e submete posteriormente um pedido de análise ao *detector-api*. Desta forma, o processo de detecção é acionado fora do caminho crítico da visualização em tempo real e em segundo plano.

O detetor automático é tratado como um módulo externo ao núcleo aplicacional. A *app* coordena a submissão do pedido, mas o processamento efetivo é realizado pelo *detector-worker*, permitindo que as tarefas de visão computacional decorram de forma assíncrona e separada da lógica principal da plataforma. Quando necessita de obter o ficheiro de vídeo a analisar, o *detector-worker* solicita o segmento através do *nginx*, mantendo uma fronteira controlada entre o módulo de detecção e os recursos geridos pela aplicação.

No interior do detetor, o diagrama representa apenas as decisões principais da *pipeline*: a existência de movimento relevante e, quando aplicável, a detecção de objetos de interesse. Os detalhes internos desta *pipeline*, incluindo os algoritmos e modelos utilizados, são tratados no Capítulo 4. Após a análise, o resultado é devolvido à *app*, novamente através do *nginx*, sendo de seguida persistido na *database*.

3.3 Rastreabilidade das Decisões Arquiteturais

Após a apresentação da arquitetura, importa explicar a origem de algumas das decisões que, sem esta fundamentação, poderiam ser interpretadas como arbitrárias ou insuficientemente justificadas.

A utilização de um SBC como *gateway* remoto responde diretamente ao RF-02 e ao RNF-01, uma vez que permite alcançar a câmara através de um elemento intermédio controlado. A LRRQ2 reforça esta opção ao identificar as limitações associadas a redes móveis, cenários com CGNAT, *port forwarding* e dependência de mecanismos P2P dos fabricantes. Associada a esta decisão está a conectividade segura através de um túnel baseado em WireGuard/Tailscale. Esta opção também se relaciona com o RF-02 e com o RNF-01, por permitir acesso remoto seguro sem exposição pública da câmara. A fundamentação principal encontra-se na LRRQ2, onde os túneis modernos baseados em WireGuard são apresentados como uma alternativa adequada a abordagens mais pesadas ou dependentes de infraestrutura proprietária. No contexto do MVP, o Tailscale é assumido como meio prático para validar esta conectividade, mantendo a possibilidade de evolução futura para uma alternativa *self-hosted*, como o Headscale.

As camadas de abstração de vídeo e abstração operacional da câmara respondem ao RF-01 e concretizam o RNF-03, ao permitir que os componentes responsáveis pela comunicação com as câmaras evoluam ou sejam substituídos de forma independente. Adicionalmente, suportam os requisitos de visualização e controlo, RF-04 e RF-05, ao atribuir responsabilidades distintas ao *media-parser*, para gestão do vídeo, e ao *digital-twin*, para operações de controlo PTZ. Esta opção encontra fundamento na LRRQ1, onde a literatura analisada defende a utilização de camadas intermédias para reduzir a dependência direta do hardware.

A disponibilização do *streaming* através do *media-parser* e do *nginx* concretiza o RF-04 e contribui para o RNF-01. O *browser* não comunica diretamente com a câmara, sendo a complexidade de entrega do *streaming* abstraída pelo componente *nginx*. Esta decisão é coerente com a LRRQ1, que identifica a normalização do *streaming* como requisito relevante para o desacoplamento entre cliente e dispositivo físico.

Por fim, a execução assíncrona da análise automática num servidor de deteção especializado responde ao RF-07 e contribui para o RNF-03 e para o RNF-04. A separação do detetor face ao núcleo aplicacional reforça a modularidade da solução, enquanto a execução assíncrona evita bloquear a aplicação principal ou a visualização em direto. Esta decisão é suportada pelo estudo do estado da arte em visão computacional, que evidencia as limitações de executar modelos de AI em dispositivos *edge* de baixo custo e justifica a centralização do processamento pesado em infraestrutura com capacidade gráfica adequada.

3.4 Síntese

Este capítulo definiu o âmbito do MVP, o modelo de domínio e os requisitos que orientam o desenho da solução. A partir dessa análise, foi proposta uma arquitetura conceptual para o FUSE, estruturada em torno da conectividade segura a câmaras remotas, da abstração de dispositivos heterogéneos, da disponibilização centralizada de *streaming* e gravação, e da integração de análise automática de vídeo.

As decisões arquiteturais apresentadas procuram responder aos problemas identificados nos capítulos anteriores, mantendo o foco na validação experimental. O Capítulo 4 descreve

a concretização técnica desta arquitetura, detalhando os componentes, configurações e mecanismos implementados.

Capítulo 4

Implementação

Este capítulo descreve a concretização técnica do MVP do FUSE, com foco nos mecanismos implementados para integrar câmaras heterogêneas, disponibilizar *streaming* de forma segura, suportar controlo remoto e articular a plataforma central com o detetor automático de eventos. A apresentação parte das decisões de análise e desenho estabelecidas no Capítulo 3 e restringe-se aos aspetos de implementação que condicionam o funcionamento, a segurança e a extensibilidade do sistema.

4.1 Ambiente Tecnológico e Implantação do MVP

A implementação foi organizada em *containers* Docker, de modo a reduzir a dependência face ao ambiente de desenvolvimento e a manter separados os principais blocos funcionais definidos no Capítulo 3. Esta opção permitiu concretizar a arquitetura proposta através de componentes independentes, ligados por uma rede interna controlada e configurados por variáveis de ambiente, reduzindo o acoplamento entre os mesmos.

4.1.1 Tecnologias Utilizadas

A aplicação central do FUSE, anteriormente introduzida como *app*, foi implementada com Laravel e Filament. Ao nível da interface, a solução segue a *stack* TALL, composta por Tailwind CSS, Alpine.js, Livewire e Laravel.

A persistência dos dados foi suportada por MySQL. O componente conceptual *nginx* foi concretizado com Nginx, numa variante leve baseada em Alpine Linux. A gestão dos fluxos de vídeo foi concretizada com MediaMTX, configurado e utilizado como base técnica do componente conceptual *media-parser*. Por sua vez, o componente *digital-twin* foi implementado como um serviço em Python, autónomo, baseado em FastAPI e Uvicorn, recorrendo à biblioteca *onvif-zeep* para descoberta e execução de comandos PTZ através de ONVIF.

Para a conectividade com câmaras instaladas em redes externas não controladas, foi utilizada a tecnologia Tailscale, conforme previsto no desenho da solução. A configuração concreta desta ferramenta é retomada na Seção 4.4 (Segurança e Rastreabilidade).

O módulo de deteção automática foi mantido como subsistema externo ao servidor aplicacional principal. Por esse motivo, a sua *stack* tecnológica e respetiva fundamentação são apresentadas na Seção 4.5 (Detetor Automático e Pipeline de Análise).

Fundamentação de Escolhas

A seleção tecnológica do MVP privilegiou a coerência com os requisitos definidos, a rapidez de prototipagem e a simplicidade de implantação. A utilização de Laravel e Filament resulta diretamente dos requisitos tecnológicos da entidade de acolhimento, apresentados no Capítulo 1. Filament é descrito na documentação oficial como uma framework *Server-Driven UI* para Laravel construída sobre Livewire, Alpine.js e Tailwind CSS [58].

A escolha do MediaMTX foi orientada pelos requisitos de implementação do componente *media-parser*. Este componente deverá receber vídeo proveniente das câmaras e posteriormente disponibilizá-lo à plataforma num formato compatível, bem como suportar a gravação segmentada utilizada nos processos seguintes de análise. O MediaMTX apresenta-se como um servidor *proxy* multimédia com suporte para múltiplos protocolos de entrada e saída de *streaming*, incluindo RTSP e HLS [59]. Assim, a escolha não é apresentada como uma comparação face a alternativas, mas como uma opção adequada aos requisitos concretos do MVP.

No componente *digital-twin*, a biblioteca `onvif-zeep` foi escolhida por disponibilizar uma implementação *open-source* de cliente ONVIF em Python [60]. Esta opção permitiu encapsular as particularidades da comunicação ONVIF num serviço próprio, preservando a aplicação central independente dos detalhes de baixo nível de um protocolo como este. Importa recordar que, para efeitos do MVP, o controlo PTZ implementado se limita a este protocolo ONVIF. Contudo, o facto de este componente estar isolado permite que o suporte a outros protocolos seja acrescentado futuramente através da sua extensão, sem alterações estruturais à aplicação central.

As restantes tecnologias de suporte, nomeadamente Docker Compose, Nginx, MySQL, FastAPI e Uvicorn, correspondem a escolhas comuns e consolidadas na indústria. De forma semelhante, tecnologias como Tailscale e as opções associadas ao detetor são apenas enquadradas nesta secção, uma vez que a sua motivação técnica já foi introduzida no desenho arquitetural e é retomada nas secções específicas de segurança e deteção automática. isto está mal eu não introduzo nada do detetor

4.1.2 Implementação com Docker Compose

O servidor central foi implementado através de Docker Compose, agrupando os serviços principais numa rede interna denominada *fuse-network*. Esta rede permite que os componentes, implementados como *containers* individuais, comuniquem entre si internamente, sem que todas as interfaces tenham de ser expostas diretamente ao exterior. A Tabela 4.1 sintetiza o mapeamento entre os componentes conceptuais apresentados no capítulo anterior e os respetivos *containers* ou volumes.

Tabela 4.1: Mapeamento entre componentes conceptuais e recursos Docker

Componente conceptual	Container ou volume
<i>app</i>	Container fuse
<i>database</i>	Container mysql
<i>nginx</i>	Container nginx
<i>media-parser</i>	Container mediamtx
<i>digital-twin</i>	Container ptz
<i>recordings-volume</i>	Volume ./storage/app/recordings

O Excerto de Código 4.1 apresenta a parte essencial da configuração de implantação, omitindo variáveis sensíveis, detalhes auxiliares e opções repetitivas que não acrescentam informação arquitetural relevante.

```
1 services:
2   fuse:
3     build: ./docker/8.4
4     volumes:
5       - ./storage:/var/www/html/storage
6     networks:
7       - fuse-network
8   mediamtx:
9     image: bluenviron/mediamtx:1.17.0
10    volumes:
11      - ./storage/app/recordings:/recordings
12    networks:
13      - fuse-network
14  nginx:
15    image: nginx:alpine
16    ports:
17      - "443:443"
18    networks:
19      - fuse-network
20  mysql:
21    image: mysql/mysql-server:8.0
22    volumes:
23      - sail-mysql:/var/lib/mysql
24    networks:
25      - fuse-network
26  ptz:
27    build: ./docker/ptz
28    networks:
29      - fuse-network
30 networks:
31   fuse-network:
32     driver: bridge
33 volumes:
34   sail-mysql:
35     driver: local
```

Excerto de Código 4.1: Excerto simplificado da configuração Docker Compose do servidor central (yaml)

Os serviços **mediamtx** (linha 8), **mysql** (linha 20) e **nginx** (linha 14) recorrem a imagens públicas adequadas às respetivas tecnologias, nomeadamente `bluenviron/mediamtx` (linha 9), `mysql/mysql-server` (linha 21) e `nginx:alpine` (linha 15). Em contraste, os serviços **fuse** (linha 2) e **ptz** (linha 26) são construídos localmente a partir dos respetivos **Dockerfiles**, uma vez que incluem código aplicacional desenvolvido especificamente para o MVP.

Todos os *containers* têm a possibilidade de comunicar entre si através da inserção numa rede Docker interna privada, denominada de *fuse-network* (linha 31). O único componente que se expõe para o exterior, de modo a possibilitar conexões externas, é o **nginx** (linha 14), ao qual é atribuída a função de atuar como entrada para qualquer pedido à aplicação. Para o efeito, é exposta a porta externa 443, ligada à porta interna com a mesma numeração (linha 17).

O **recordings-volume** é partilhado pelos *containers* **fuse** (linha 2) e **mediamtx** (linha 8),

concretizando a arquitetura anteriormente desenhada e apresentada no Capítulo 3. A montagem do diretório de gravações (linha 11) permite que o componente *media-parser* grave os segmentos para um volume consultável pelo componente *app*, que vai estar encarregue de disponibilizar estas gravações para permitir o *playback* aos utilizadores.

4.2 Orquestração Central e Modelo Operacional

Esta secção descreve a concretização técnica do componente aplicacional em Laravel. Considerando que o seu papel conceptual foi definido no Capítulo 3, a descrição centra-se nos mecanismos implementados para suportar a operação da plataforma, nomeadamente a persistência dos dados, a aplicação de regras de autorização e a coordenação com os restantes serviços internos. Também explica detalhes necessários à implementação do modelo de domínio.

4.2.1 Concretização do Modelo de Domínio

O modelo de domínio apresentado no Capítulo 3 foi materializado através de migrações de base de dados e modelos Eloquent, o mecanismo de mapeamento objeto-relacional do Laravel.

O conceito de Câmara foi implementado com chave estrangeira para Processo, *process_id*, uma vez que cada câmara se encontra sempre afeta a um processo no contexto operacional da plataforma. O modelo inclui também o alvo de deteção configurado, usado posteriormente para determinar que classe de objetos deve ser analisada. Esta opção pode parecer ambígua se a câmara for entendida apenas como dispositivo físico reutilizável, porém, no contexto da aplicação, uma *Câmara* representa uma montagem ou configuração operacional concreta de um dispositivo num determinado processo. Assim, se o mesmo equipamento for instalado noutro local, associado a outro processo ou configurado com outro alvo de deteção, essa situação passa a corresponder a uma nova instância operacional na aplicação.

O Segmento de Gravação foi modelado com uma chave estrangeira *camera_id*, uma vez que cada segmento de gravação corresponde sempre ao vídeo captado por uma câmara específica. Por sua vez, o Evento de Deteção referencia o segmento onde a ocorrência foi identificada, através de *recording_segment_id*. Como um evento de deteção pode envolver vários objetos detetados no mesmo intervalo temporal, o modelo Objeto de Deteção referencia o evento correspondente através de uma chave estrangeira para Evento de Deteção, *event_id*. Esta organização concretiza, na base de dados, a sequência descrita no modelo de domínio: câmara, segmento de gravação, evento de deteção e objetos detetados.

Pergunta: posso por câmara -> segmento de gravação -> evento de deteção -> objetos detetados ? com setas?

Na componente de processos e acessos aos mesmos, a relação entre *Processo* e *Utilizador* foi implementada através da tabela intermédia *process_user*, permitindo associar utilizadores aos processos em que participam. A modelação de perfis e permissões é retomada na subsecção seguinte *está mal ? referencio?*, onde se descreve o mecanismo de controlo de acessos da aplicação.

4.2.2 Controlo de Acessos na Aplicação

O controlo de acessos na aplicação foi implementado através da combinação de dois mecanismos complementares. O primeiro delimita o âmbito operacional do utilizador, determinando que processos e câmaras ficam visíveis. O segundo define as ações que o utilizador pode executar sobre os recursos a que tem acesso, através de permissões agregadas em *roles* que lhe são atribuídas.

Filtragem por Processo

A associação entre utilizadores e processos é materializada pela tabela pivot `process_user`. Esta tabela representa os utilizadores atribuídos a cada processo e é usada como base para restringir as consultas efetuadas nos recursos operacionais da aplicação. Assim, quando um utilizador sem privilégios de gestão lista processos ou câmaras, a consulta é automaticamente limitada ao seu contexto operacional.

O Excerto de Código 4.2 apresenta um excerto demonstrativo do mecanismo usado quando é necessário listar processos e câmaras na aplicação.

```
1 public static function getEloquentQuery(): Builder
2 {
3     return parent::getEloquentQuery()
4         ->relatedToUser(Auth::user());
5 }
```

Excerto de Código 4.2: Excerto demonstrativo da filtragem por processo e câmara (php)

Tanto na listagem de processos como na de câmaras, a consulta base do recurso chama o método `relatedToUser` (linha 4 do Excerto de Código 4.2), cujo funcionamento é demonstrado no Excerto de Código 4.3:

```

1 public function scopeRelatedToUser(Builder $query, ?User $user): void
2 {
3     $query->when(
4         ! $user?->can(PermissionSeeder::BYPASS_PROCESS_FILTERING),
5         fn (Builder $query) => $query->whereUserIsOperator($user)
6     );
7 }
8
9 public function scopeWhereUserIsOperator(Builder $query, User $user): void
10 {
11     if ($query->getModel() instanceof Process) {
12         $query->whereHas(
13             'operators',
14             fn (Builder $query) => $query->where('user_id', $user->id)
15         );
16         return;
17     }
18
19     $query->whereHas('process', function (Builder $query) use ($user):
20     void {
21         $query->whereUserIsOperator($user);
22     });
23 }

```

Excerto de Código 4.3: Excerto do funcionamento do filtro *relatedToUser* (php)

O método `relatedToUser` (linha 1) apenas aplica a filtragem quando o utilizador autenticado não possui a permissão `bypass-process-filtering` (linha 4). Esta lógica permite não aplicar restrições adicionais e evita consultas desnecessariamente restritivas quando o Utilizador tem associada uma *Role* com permissões para listar recursos sem filtrar pelos processos a que tem acesso.

Caso o utilizador não tenha esta permissão, o método `whereUserIsOperator` (linha 9) concretiza a restrição por processo. Quando a consulta incide sobre processos, a filtragem é feita pela relação `operators` (linha 13), materializada pela tabela `process_user`. Quando a consulta incide sobre câmaras, a filtragem é aplicada através da relação com o respetivo processo (linha 19), reutilizando o mesmo método e a mesma regra de associação entre processo e utilizador (linha 20). Desta forma, a visibilidade dos processos e das câmaras é mantida coerente.

Estes dois métodos são aplicados ao Processo e à Câmara através da utilização do *trait* `HasProcessFiltering`. Desta maneira, futuramente, caso seja necessário estender a aplicação, este filtro funcionará com todos os modelos que usem este *trait* e que, pela lógica de negócio, estejam associados a um processo.

Roles e Permissões

A autorização das ações não depende diretamente das *roles*, mas sim das permissões atribuídas a estas. As *roles* funcionam como agregadores administrativos de permissões, permitindo que a configuração final da aplicação seja ajustada no momento de implantação sem alterar a lógica aplicacional. Assim, uma organização pode criar novos perfis ou alterar as permissões associadas aos perfis existentes, mantendo o mesmo mecanismo de validação.

Ao nível da implementação, esta separação é materializada por tabelas *pivot* distintas. A tabela *user_has_roles* associa utilizadores às respetivas *roles*, enquanto a tabela *role_has_permissions* define o conjunto de permissões agregadas por cada *role*. Assim, a aplicação não precisa de validar diretamente o nome da *role*, mas sim se o utilizador possui a permissão necessária para a ação em causa, através de uma das *roles* a ele atribuídas.

No MVP, foram definidos perfis demonstrativos para representar diferentes níveis de responsabilidade. O perfil *System Admin* possui acesso integral à aplicação, sendo o único com acesso a auditoria e gestão de utilizadores. O perfil *Manager* tem permissões de gestão sobre processos e câmaras, incluindo a capacidade de atribuir operadores a processos e de contornar a filtragem por processo. O perfil *Controller* pode visualizar os processos e câmaras que lhe foram atribuídos e executar operações de controlo remoto sobre essas câmaras. O perfil *Visualizer* não possui permissões de controlo das câmaras, estando restrito a visualização de informação, câmaras e gravações relativos aos processos permitidos.

4.2.3 Recursos Filament e Operação da Plataforma

A camada operacional da aplicação foi concretizada através de recursos Filament, que funcionam como ponto de ligação entre os modelos Eloquent e as tarefas executadas pelos utilizadores. Cada recurso define a forma como uma entidade é apresentada, pesquisada, editada ou acionada na plataforma, através de formulários, tabelas, páginas de visualização e ações contextuais. Desta forma, o modelo de domínio é disponibilizado numa interface orientada à operação do MVP, reutilizando a estrutura já definida nos modelos Eloquent e nas suas relações.

A Tabela 4.2 mostra três exemplos de recursos Filament e suas respetivas funções.

Tabela 4.2: Principais recursos Filament da aplicação central

Recurso	Modelo	Função
<i>ProcessResource</i>	Processo	Gestão dos processos, consulta do seu estado operacional e associação de operadores e câmaras.
<i>CameraResource</i>	Câmara	Configuração das câmaras e acesso às operações de visualização, gravação, reprodução, deteção e controlo remoto.
<i>UserResource</i>	Utilizador	Administração de utilizadores e atribuição de <i>roles</i> .

O caso mais representativo, que será usado como exemplo, é o recurso de Câmara, uma vez que este não se limita às operações genéricas de criação, edição e consulta. A sua definição inclui páginas específicas para gravações e deteções (linhas 8 e 9), como se observa no Excerto de Código 4.4.

```

1 public static function getPages(): array
2 {
3     return [
4         'index' => Pages\ListCameras::route('/'),
5         'create' => CreateCamera::route('/create'),
6         'view' => Pages\ViewCamera::route('/{record}'),
7         'edit' => Pages\EditCamera::route('/{record}/edit'),

```

```
8      'recordings' => Pages\Recordings::route('/{record}/recordings'),  
9      'detections' => Pages\Detections::route('/{record}/detections'),  
10 ];  
11 }
```

Excerto de Código 4.4: Páginas definidas no recurso de Câmara (php)

Esta organização mostra que a Câmara foi tratada como o eixo operacional da interface. Embora existam entidades persistentes próprias para Segmentos de Gravação, Eventos de Detecção e Objetos Detetados, estas não foram expostas como recursos principais independentes. Em vez disso, são acedidas a partir da Câmara a que pertencem, através das páginas de gravações (linha 8) e deteções (linha 9), preservando o contexto operacional do utilizador e evitando que a navegação da plataforma se fragmente em entidades técnicas pouco significativas fora desse contexto.

Para além das páginas específicas, a página de visualização da Câmara agrega elementos associados à operação diária da plataforma, incluindo o *streaming* HLS em direto, a ação de iniciar ou parar gravação, a navegação para *playback* de gravações, a consulta de deteções associadas e o acesso ao controlador PTZ. A existência de indicadores de deteções não lidas permite ainda destacar eventos que requerem revisão pelo utilizador. O processamento que origina estas deteções é descrito na secção dedicada ao detetor automático, enquanto a sua persistência e apresentação ao utilizador são retomadas na secção dedicada aos eventos e artefactos de análise.

Estes recursos funcionam como a camada visível e interativa da aplicação, enquanto a lógica de integração e processamento permanece isolada em serviços e tarefas em segundo plano, descritos nas subsecções seguintes.

4.2.4 Serviços Aplicacionais, Configuração e Integração

Na sequência da descrição dos recursos Filament, importa clarificar como as operações disponibilizadas nesta camada interativa são encaminhadas para a lógica aplicacional quando executadas pelo utilizador. A implementação segue um fluxo simples, em que uma **Action** do Filament representa uma operação disponível na interface. Quando acionada, esta delega a execução num serviço aplicacional. Caso a operação necessite de comunicar com outro componente do sistema, esse serviço recorre a um cliente de integração responsável por encapsular a chamada ao serviço externo.

Este padrão permite que a interface não dependa diretamente dos detalhes dos serviços externos. Assim, a definição visual e operacional de um botão Filament permanece separada da chamada HTTP efetuada ao MediaMTX, ao serviço PTZ ou ao detetor automático.

Actions Reutilizáveis

Como descrito, as *Actions* Filament foram usadas como pontos de entrada reutilizáveis para operações frequentes da plataforma. Estas ações encapsulam elementos da interação com o utilizador, como o rótulo, o ícone, a cor, a confirmação e o comportamento executado quando a ação é acionada. No entanto, a lógica técnica da operação é delegada para classes da camada aplicacional.

O exemplo mais direto é `StartStopRecording`, demonstrado no Excerto de Código 4.5:

```
1 class StartStopRecording extends Action
2 {
3     public static function make(?string $name = 'start-stop-recording'):
4     static
5     {
6         return parent::make($name)
7         ->label(fn (Camera $record) => $record->rec ? __('Stop
8 Recording') : __('Start Recording'))
9         ->icon(fn (Camera $record) => $record->rec ? 'phosphor-record'
10 : 'phosphor-video-camera-slash')
11         ->color(fn (Camera $record) => $record->rec ? 'danger' : 'gray
12 ')
13         ->requiresConfirmation(fn (Camera $record) => $record->rec)
14         ->action(fn (Camera $record) => MediaParserService::
15 startStopRecording($record));
16     }
```

Excerto de Código 4.5: Action reutilizável para iniciar ou parar gravação (php)

Conforme observado, esta *Action* é usada para iniciar ou parar a gravação de uma Câmara. A *Action* decide dinamicamente a sua apresentação visual com base no estado da Câmara, mas a execução da operação é delegada para *MediaParserService*.

Camada de Clientes de Integração

Quando a operação acionada precisa de contactar um componente externo, a aplicação recorre a uma camada de clientes de integração. Esta camada funciona como fronteira entre a aplicação Laravel e os módulos especializados, evitando que os recursos Filament ou os serviços aplicativos conheçam diretamente a estrutura completa das Application Programming Interfaces (APIs) externas.

No caso da gravação, a *Action* apresentada no Excerto de Código 4.5 delega a operação em *MediaParserService*. Este serviço calcula o novo estado pretendido para a Câmara e, por sua vez, chama *MediaParserClient*, responsável por enviar a atualização ao MediaMTX. O Excerto de Código 4.6 apresenta este fluxo de forma simplificada.

```
1 // MediaParserService
2 public static function startStopRecording(Camera $camera): void
3 {
4     $oppositeState = ! $camera->rec;
5     MediaParserClient::setRecording($camera->slug, $oppositeState);
6
7     $camera->rec = $oppositeState;
8     $camera->save();
9 }
10
11 // MediaParserClient
12 public static function setRecording(string $name, bool $record): Response
13 {
14     return Http::withHeaders(['Content-Type' => 'application/json'])
15         ->patch(self::buildAPIUrl('patch', $name), [
16             'record' => $record,
17         ]);
18 }
```

Excerto de Código 4.6: Excerto simplificado de alteração de gravação para o cliente *media-parser* (php)

Esta separação cria uma barreira de integração, onde a camada operacional sabe que existe uma ação de iniciar ou parar gravação, mas é o serviço aplicacional que sabe que essa ação altera o estado da Câmara. Apenas a camada de cliente sabe como traduzir essa alteração numa chamada HTTP concreta ao MediaMTX. O mesmo princípio é utilizado noutras integrações, como o envio de comandos para o serviço PTZ ou a submissão de segmentos ao detetor automático.

Configuração Aplicacional

Para evitar que a camada de clientes fique fortemente acoplada aos componentes externos que concretizam cada integração, a aplicação centraliza as configurações do ambiente no ficheiro `config/fuse.php`. Este ficheiro lê os valores definidos no `.env` e disponibiliza-os ao restante código através da função `config`.

Esta opção é particularmente relevante no componente *media-parser*. Do ponto de vista da aplicação central, o que existe é um serviço responsável por disponibilizar *streaming* e gravação de vídeo das câmaras. No MVP, esse papel é concretizado pelo MediaMTX, mas a aplicação não deve depender diretamente de uma instância desta ferramenta. A ligação concreta ao MediaMTX é, por isso, expressa através da configuração, permitindo que a camada de clientes contacte o serviço atualmente definido como *media-parser*.

O Excerto de Código 4.7 ilustra esta separação.

```
1 // fuse.php
2 'mediaparser' => [
3     'url' => env('MEDIAPARSER_INTERNAL_URL', 'http://mediamtx'),
4     'ports' => [
5         'api' => env('MEDIAPARSER_PORT_API', '9997'),
6     ],
7     'paths' => [
8         'api' => env('MEDIAPARSER_PATH_API', 'v3'),
9     ],
10 ],
11 // MediaParserClient
12 private static function buildAPIUrl(string $operation, string $name):
13     string
14 {
15     $domain = config('fuse.mediaparser.url');
16     $port = config('fuse.mediaparser.ports.api');
17     $path = config('fuse.mediaparser.paths.api');
18
19     return "{$domain}:{$port}/{path}/{operation}/{name}";
20 }
```

Excerto de Código 4.7: Configuração e construção do *endpoint* do *media-parser* (php)

A configuração define o endereço de acesso interno ao *media-parser*, a porta da API e o caminho usado para contactar o componente. O cliente usa esses valores para construir o *endpoint* final. Este mecanismo é usado não apenas para a ligação ao *media-parser*, mas também para o serviço PTZ, para o detetor automático e para outros parâmetros operacionais da plataforma. Deste modo, uma alteração futura da tecnologia usada para concretizar o *media-parser*, ou outro componente, concentrar-se-á na configuração e na respetiva classe cliente, sem exigir que a interface Filament ou os serviços aplicacionais fossem reescritos.

4.2.5 Processamento Assíncrono e Trabalhos em Segundo Plano

Algumas operações de integração não devem bloquear a interação principal do utilizador, sobretudo quando envolvem descoberta de serviços ou chamadas a componentes externos. Por esse motivo, a aplicação recorre a trabalhos em segundo plano para executar processos que podem demorar ou falhar sem comprometer a resposta dos recursos na interface.

O exemplo mais explícito é a descoberta automática de suporte ONVIF após a criação de uma Câmara, como se observa no Excerto de Código 4.8.

```
1 static::created(function (Camera $camera): void {
2     OnvifPortSearcher::dispatch($camera->id);
3 });
```

Excerto de Código 4.8: Excerto simplificado de origem de tarefa após criação de uma Câmara (php)

No registo da Câmara, o utilizador indica o endereço do dispositivo, mas não precisa de conhecer previamente o endpoint de controlo. Após a persistência, a aplicação origina o trabalho *OnvifPortSearcher* (linha 2), que testa combinações comuns de protocolo, porta e caminho ONVIF. Quando encontra uma resposta compatível, persiste o protocolo e a porta detetados.

Esta abordagem reduz a configuração manual necessária para ativar o controlo PTZ. Se a Câmara não responder às combinações testadas, permanece utilizável para visualização e gravação, apenas sem controlo remoto automático. O mesmo padrão permite acrescentar futuramente outras tarefas assíncronas de descoberta de correspondência de protocolo, sem exigir configuração técnica detalhada no momento do registo.

4.3 Streaming, Gravação e Operação Remota

Esta secção concretiza a implementação relativa ao vídeo, que foi desenhada no Capítulo 3, descrevendo como o *media-parser*, o Nginx e o serviço *digital-twin* suportam a receção da *stream*, a disponibilização em HLS e a gravação segmentada. Também é explicada a operação remota PTZ.

4.3.1 Configuração Funcional do *media-parser*

A configuração funcional do componente *media-parser* é definida no ficheiro específico da tecnologia que o concretiza no MVP, neste caso o *mediamtx.yml*. O Excerto de Código 4.9 apresenta um excerto simplificado desta configuração, omitindo valores sensíveis e parâmetros não essenciais, uma vez que não alteram a compreensão do fluxo implementado.

```
1 api: yes
2 apiAddress: :9997
3
4 hls: yes
5 hlsAddress: :8888
6 hlsAlwaysRemux: yes
7 hlsVariant: lowLatency
8
9 pathDefaults:
10   recordPath: ./recordings/%path/%Y-%m-%d_%H-%M-%S-%f
11   recordFormat: fmp4
12   recordSegmentDuration: 30s
13   recordDeleteAfter: 30d
14   runOnRecordSegmentComplete: >-
15     curl --fail --silent --show-error --output /dev/null
16     --header 'X-MTX-Hook-Token: <token>'
17     --data-urlencode 'camera=$MTX_PATH'
18     --data-urlencode 'segment_path=$MTX_SEGMENT_PATH'
19     '${MEDIAMTX_SEGMENT_HOOK_URL}'
```

Excerto de Código 4.9: Excerto simplificado da configuração funcional do MediaMTX (yaml)

A API do MediaMTX é ativada nas linhas 1 e 2 do Excerto de Código 4.9. Esta interface permite que a *app* atualize dinamicamente, por Câmara, os caminhos geridos pelo *media-parser*. Quando uma Câmara é criada, editada ou removida na aplicação, a respetiva configuração de acesso ao vídeo é enviada para o MediaMTX. Desta forma, os caminhos disponíveis no componente multimédia coincidem com a configuração operacional definida na aplicação.

As linhas 4 a 7 configuram a disponibilização do vídeo através de HLS. Esta é a opção de implementação que permite padronizar o *streaming* disponibilizado pelo componente para visualização no *browser*. A opção *hlsAlwaysRemux* mantém a geração HLS disponível mesmo antes do primeiro pedido, reduzindo o tempo inicial de visualização, enquanto *hlsVariant*:

`lowLatency` seleciona uma variante mais leve, adequada à reprodução contínua em direto. O *proxy* destes pedidos HLS através do Nginx é descrito na subsecção seguinte.

As linhas 10 a 13 definem a organização da gravação segmentada. O caminho de gravação inclui um identificador da Câmara e um *timestamp* do segmento. A duração mínima de cada segmento é configurada para 30 segundos e a retenção em disco limitada a 30 dias. Estes parâmetros, como introduzido anteriormente, evitam tratar a gravação como um ficheiro contínuo único, produzindo unidades temporais que podem ser posteriormente registadas, consultadas e processadas pela aplicação.

Por fim, o comando `runOnRecordSegmentComplete`, apresentado a partir da linha 14, define o comando a ser executado pelo MediaMTX quando um segmento de gravação fica completo. Este envia à *app* a identificação da Câmara e o caminho do ficheiro gravado, permitindo que a aplicação crie o respetivo Segmento de Gravação. Os mecanismos de autenticação deste *callback* são tratados na secção de segurança.

4.3.2 Proxy HLS pelo Nginx

Na implementação, o Nginx separa os pedidos aplicacionais dos pedidos de *streaming*: a interface é encaminhada para a *app*, enquanto os pedidos HLS são encaminhados para o MediaMTX. Esta separação evita que a aplicação Laravel intermedie continuamente os fragmentos de vídeo, mantendo-a focada na validação operacional e na coordenação dos serviços internos.

O Excerto de Código 4.10 apresenta um excerto simplificado desta configuração, omitindo a lógica de autenticação e autorização dos pedidos HLS, descrita posteriormente na Secção 4.4.

```
1 server {  
2     listen 443;  
3  
4     location ~* ~/player/(?<camera_name>[~/]+)(?<remaining_path>/.*)?$ {  
5         proxy_redirect ~~/([~/]+)(.*)$ /player/$1$2;  
6         proxy_pass http://mediamtx:8888/$camera_name$remaining_path;  
7     }  
8  
9     location / {  
10        proxy_pass http://fuse:80;  
11    }  
12 }
```

Excerto de Código 4.10: Proxy funcional de pedidos aplicacionais e HLS no Nginx (nginx)

O bloco `/player` captura o identificador da Câmara e o restante caminho do recurso HLS (linha 4), encaminhando o pedido para o servidor HLS interno do MediaMTX (linha 6). O `proxy_redirect` (linha 5) preserva o prefixo público usado pelo Nginx em eventuais referências devolvidas pelo serviço multimédia. Os restantes pedidos são encaminhados para a aplicação Laravel (linha 10), permitindo que, para o utilizador, a interface e o *streaming* sejam servidos pelo mesmo ponto de entrada, embora internamente sejam tratados por componentes distintos.

4.3.3 Operação Remota PTZ

Depois da descoberta do *endpoint* PTZ compatível de uma determinada câmara, como apresentado anteriormente na Subsecção 4.2.5, a aplicação persiste em base de dados a configuração necessária para esta integração. Contudo, a *app* não implementa diretamente os detalhes de controlo dos protocolos suportados, limitando-se a transformar a interação de controlo do utilizador numa intenção operacional e passá-la para o componente especializado *digital-twin*.

O serviço aplicacional *DigitalTwinClient*, similarmente ao *MediaParserClient* apresentado anteriormente, envia estes pedidos para o *container* *ptz*, usando o endereço, credenciais, porta, protocolo e vetor de movimento da Câmara. O Excerto de Código 4.11 apresenta, de forma simplificada, como o componente *digital-twin* valida o protocolo pretendido e traduz a intenção recebida para a implementação correspondente.

```
1 def handle_protocol(protocol, url, port, user, password, action, **kwargs)
2     :
3     if protocol == "onvif":
4         if action == "move":
5             onvif_client.move(
6                 url=url,
7                 port=port,
8                 user=user,
9                 password=password,
10                x=kwargs.get("x", 0.0),
11                y=kwargs.get("y", 0.0),
12                z=kwargs.get("z", 0.0),
13            )
14        elif action == "stop":
15            onvif_client.stop(
16                url=url,
17                port=port,
18                user=user,
19                password=password,
20            )
21        else:
22            raise ValueError(f"Unsupported protocol: {protocol}")
```

Excerto de Código 4.11: Excerto simplificado de encaminhamento de controlo PTZ (python)

Quando a ação recebida é de movimento, é validado o protocolo pretendido e o serviço encaminha os eixos x, y e z para o a câmara, que se trata do cliente ONVIF (linhas 3 a 12). Já quando a ação é de paragem, delega no método correspondente do mesmo cliente (linhas 13 a 19). A conversão final para chamadas concretas *ContinuousMove* e *Stop* fica encapsulada nesse cliente, implementado com *onvif-zeep*. Assim, a variação por protocolo fica concentrada no componente *ptz*, permitindo acrescentar novos clientes de controlo sem alterar a lógica principal da *app*.

4.4 Segurança e Rastreabilidade

Esta secção apresenta os mecanismos de segurança e rastreabilidade relevantes para a solução implementada. São descritas a conectividade segura com Tailscale, a proteção do

streaming HLS, a autenticação de chamadas entre componentes e a auditoria de operações relevante.

4.4.1 Conectividade Segura com Tailscale

Na implementação do MVP, a conectividade segura entre o servidor FUSE e o local remoto é concretizada através do Tailscale. O cliente Tailscale é configurado no servidor FUSE e no SBC instalado junto da Câmara, não diretamente na própria Câmara. Deste modo, como fundamentado no Capítulo 2 (Estudo do Estado da Arte), o SBC assume o papel de *gateway* seguro da rede local onde o dispositivo se encontra, permitindo que a Câmara seja alcançada pelo servidor central sem exposição direta à Internet pública.

O comando seguinte é executado no SBC para associar este equipamento à *tailnet*, isto é, à rede privada formada pelos dispositivos autorizados no Tailscale:

```
tailscale up --auth-key=<auth-key> --advertise-routes=x.x.x.x/24
```

A configuração relevante consiste em usar o parâmetro `-advertise-routes` para tornar a rede alcançável através da *tailnet*. Assim, os componentes alojados no *fuse-server* conseguem alcançar os endereços da rede remota, incluindo a Câmara, através do túnel estabelecido pelo SBC. No comando exemplificado, o valor `x.x.x.x/24` representa o prefixo da rede local a anunciar, e o sufixo `/24` indica o número de bits da máscara de rede. Um exemplo de prefixo seria `192.168.10.0/24`.

A autenticação deste dispositivo na *tailnet* é realizada através do parâmetro `-auth-key`. Este parâmetro espera uma chave de autenticação que é gerada no painel de administração do Tailscale.

No lado do servidor FUSE, o cliente Tailscale foi considerado ao nível do *fuse-server*, em vez de ser configurado individualmente em cada *container*. Esta opção reduz duplicação de configuração e autenticação, uma vez que a *app*, o *media-parser* e o *digital-twin* necessitam de alcançar a rede remota para funções distintas. A aplicação continua a coordenar as operações ao nível dos serviços, mas a conectividade de rede é concentrada no servidor.

A esta configuração junta-se a definição de Access Control Lists (ACLs) no Tailscale, usadas como políticas de acesso. A comunicação pretendida é essencialmente iniciada pelo *fuse-server* para alcançar as redes remotas necessárias, pelo que não é necessário conceder aos SBCs acesso livre ao servidor central. Ao restringir esse sentido de comunicação, o acesso indevido a um SBC em cenário real deixa de implicar, por si só, capacidade de movimento lateral para os serviços internos do FUSE.

4.4.2 Proteção do Streaming HLS

O *proxy* HLS apresentado na Subseção 4.3.2 não pode ser tratado como um recurso aberto. A proteção implementada separa duas responsabilidades: a *app* decide se o Utilizador pode visualizar a Câmara, enquanto o *nginx* valida cada pedido HLS antes de o encaminhar para o *media-parser*. Desta forma, a aplicação central não serve o *streaming*, mas continua a controlar a emissão da autorização necessária para esse acesso.

O Excerto de Código 4.12 apresenta um excerto simplificado da validação aplicada pelo Nginx, omitindo o segredo `STREAM_SECRET`.

```
1 location ~* ~/player/(?<camera_name>[/]+)(?<remaining_path>/.*)?$ {  
2     set $stream_secret "<STREAM_SECRET>";  
3     secure_link $cookie_stream_auth;  
4     secure_link_md5 "$secure_link_expires$camera_name$stream_secret";  
5  
6     if ($secure_link = "") { return 403; }  
7     if ($secure_link = "0") { return 403; }  
8  
9     proxy_pass http://mediamtx:8888/$camera_name$remaining_path;  
10 }
```

Excerto de Código 4.12: Validação simplificada do acesso HLS no Nginx (nginx)

Neste mecanismo, o cookie `stream_auth` funciona como uma autorização temporária para uma Câmara específica. O seu valor contém uma assinatura e um limite temporal de validade, no formato `HASH,EXPIRES`. A assinatura é calculada pela aplicação com base nesse limite temporal, no identificador da Câmara e num segredo partilhado com o Nginx, sendo depois validada pelo `secure_link`. No MVP, a validade configurada é de 30 segundos. Caso o cookie esteja ausente, fora do período de validade ou tenha sido manipulado, o Nginx recusa o pedido com código 403 (linha 6 e 7). Apenas pedidos válidos são encaminhados para o MediaMTX.

Como a autorização é temporária, a interface volta a avaliar o acesso de 10 em 10 segundos enquanto o leitor de vídeo está ativo, através de `wire:poll.10s`. Quando o Utilizador continua autorizado, a aplicação reemite o cookie `stream_auth`. Esta opção permite manter a visualização contínua sem atribuir uma credencial de longa duração ao *streaming*. Caso uma autorização seja reutilizada indevidamente, a sua validade fica limitada à curta janela temporal configurada.

4.4.3 Autenticação de Chamadas entre Componentes

Para além da proteção do acesso ao *streaming*, a implementação inclui validações específicas nas chamadas entre componentes. Cada integração relevante usa tokens, chaves ou assinaturas baseadas em segredos previamente configurados no ambiente. A aplicação valida esses elementos antes de alterar estados ou persistir dados, reduzindo o risco de aceitar notificações ou resultados originados por componentes não autorizados.

O exemplo mais representativo é o callback do detetor automático, uma vez que este devolve resultados de análise que serão posteriormente persistidos. Neste fluxo, o pedido inclui os cabeçalhos `X-Detector-Key` (chave), `X-Detector-Timestamp` e `X-Detector-Signature` (assinatura). A chave identifica o serviço autorizado, enquanto a assinatura é calculada com Hash-based Message Authentication Code (HMAC) SHA-256 sobre a combinação entre o instante temporal indicado e o corpo do pedido. A aplicação valida estes elementos no `EventResultRequest`, permitindo rejeitar resultados sem chave válida, fora do intervalo temporal aceite ou com conteúdo alterado.

O mesmo princípio de acesso limitado é aplicado ao envio de segmentos para análise. Quando o `AnalyzeRecordingSegmentJob` submete um segmento ao detetor, fornece um url temporariamente assinado para descarregar apenas esse ficheiro, com validade própria definida por configuração. O endpoint que serve o ficheiro valida a assinatura do url e a chave do detetor. Assim, o detetor recebe acesso ao segmento necessário para a análise, sem obter acesso livre ao armazenamento interno da aplicação. Este fluxo é detalhado na Seção 4.5.

4.4.4 Auditoria e Rastreabilidade

A rastreabilidade operacional foi implementada ao nível aplicacional, sobre os modelos Eloquent. Para concretizar este mecanismo foi utilizado o pacote `owen-it/laravel-auditing`, que permite registar automaticamente alterações em modelos que implementem o *trait* `HasAudits`. Este *trait* é aplicado na classe `BaseModel`, herdada pelos principais modelos de domínio da aplicação, como se observa no Excerto de Código 4.13.

```
1 abstract class BaseModel extends Model implements Auditable
2 {
3     use HasAudits;
4 }
```

Excerto de Código 4.13: Aplicação do *trait* de auditoria na classe base dos modelos (php)

Desta forma, cada operação de escrita na base de dados, ou seja, operações de criação, atualização ou eliminação, passam a gerar registos de auditoria de forma uniforme nos modelos que estendem desta classe base.

Cada registo de auditoria guarda o Utilizador associado à operação, o tipo de evento, a entidade auditada, os valores antigos e novos e a data e hora da alteração. Também inclui o url, endereço IP, *browser* e sistema operativo, quando disponíveis. Estes registos são consultáveis por administradores na aplicação, através de uma listagem e de uma página de detalhe. Para evitar a exposição indevida de informação sensível, determinados campos, como palavras-passe ou tokens, são mascarados antes de serem apresentados na interface de auditoria.

4.5 Detetor Automático e Pipeline de Análise

O detetor automático concretiza, em termos de implementação, o componente *detector* definido no Capítulo 3. Esta secção descreve o ambiente tecnológico utilizado, a forma como os subcomponentes conceptuais foram implantados, a *pipeline* de análise implementada até à Fase 2 do MVP e o contrato de resultado devolvido à aplicação central.

4.5.1 Ambiente Tecnológico e Implantação do Serviço

A implementação do detetor retoma a divisão interna definida do componente *detector*, composto por *detector-api*, *detector-worker* e *detector-cache*. Nesta subsecção, cada subcomponente é descrito quanto à sua concretização técnica no MVP.

Tecnologias Utilizadas

O *detector-api* foi implementado em Python com FastAPI e Uvicorn. Este subcomponente materializa a interface de entrada do detetor, que recebe pedidos de análise de segmentos, valida o pedido recebido e coloca a tarefa de análise numa fila assíncrona.

O *detector-worker* foi igualmente implementado em Python, mas utilizando Celery para executar as tarefas colocadas previamente na fila. É neste subcomponente que são executadas as operações de obtenção do ficheiro correspondente ao segmento de gravação, e respetivo processamento de vídeo. A filtragem inicial é realizada com OpenCV e a deteção de objetos com o modelo **YOLO11-seg**. A execução deste modelo é suportada por PyTorch e CUDA.

O *detector-cache* foi concretizado com Redis. Na implementação, este subcomponente concretiza a fila necessária ao processamento assíncrono, sendo utilizado pela configuração do Celery como infraestrutura de suporte aos trabalhos de análise.

Fundamentação de Escolhas

A seleção tecnológica do detetor automático foi orientada pela necessidade de concretizar a *pipeline* definida para o MVP, mantendo coerência com as decisões já fundamentadas no Capítulo 2 e no Capítulo 3.

A utilização de Python resulta da maturidade do seu ecossistema para visão computacional e inferência de modelos de aprendizagem automática. Neste contexto, o FastAPI permite definir os *endpoints* de análise e validar de forma estruturada os dados de entrada e saída, enquanto o Uvicorn assegura a execução do serviço HTTP. A utilização de Celery, em conjunto com Redis, concretiza a execução de trabalhos em segundo plano e o suporte à fila de processamento.

Para a Fase 1 da *pipeline*, o OpenCV foi escolhido por disponibilizar operações *standard* de processamento de vídeo e imagem, adequadas à detecção inicial de movimento descrita no Seção 2.4. Para a Fase 2, a escolha do YOLO11-seg parte da análise realizada no Subseção 2.4.3, onde o YOLO11 foi identificado como uma evolução recente da família YOLO adequada a cenários que exigem compromisso entre velocidade e precisão. Na implementação, foi utilizada a sua variante de segmentação, identificada pelo sufixo "-seg", por se adequar à extração das regiões visuais associadas aos objetos detetados. A utilização de PyTorch e CUDA decorre da necessidade de executar essa inferência com aceleração por GPU, quando disponível.

Implantação com Docker Compose

Depois desta concretização tecnológica, os três subcomponentes conceptuais são mapeados para três *containers* distintos, nomeados de acordo com o componente que representam. O Excerto de Código 4.14 apresenta um excerto simplificado desta implantação.

```
1 services:
2   detector-api:
3     build:
4       context: .
5       dockerfile: Dockerfile.gpu
6     ports:
7       - "8080:8080"
8     command: ["python", "-m", "uvicorn", "app.main:app", "--host", "
9       0.0.0.0", "--port", "8080"]
10    detector-worker:
11      build:
12        context: .
13        dockerfile: Dockerfile.gpu
14      gpus: all
15      volumes:
16        - <model-dir>:/models/yolo11:ro
17      command: ["python", "-m", "celery", "-A", "app.queue.celery_app", "
18        worker"]
19    detector-cache:
20      image: redis:7-alpine
```

Excerto de Código 4.14: Excerto simplificado da implantação do serviço de detecção (yaml)

O *detector-api* e o *detector-worker* são construídos a partir do mesmo *Dockerfile* com suporte GPU (linhas 3 a 5 e 10 a 12), partilhando o mesmo ambiente de execução e dependências. O primeiro executa a API HTTP com Uvicorn (linha 8), enquanto o segundo declara acesso à GPU (linha 13) e executa o processo Celery responsável pelos trabalhos assíncronos (linha 16). O *detector-cache* utiliza uma imagem Redis leve (linha 18), suficiente para suportar a fila e o estado operacional dos trabalhos.

O modelo *yolo11s-seg.pt* não é guardado em repositório de versões com o código-fonte da aplicação. No protótipo, é descarregado por um script de apoio a partir do repositório público *Ultralytics/YOLO11*, alojado na plataforma Hugging Face, usada para disponibilização e distribuição de modelos de aprendizagem automática. Esse script configura também o caminho do modelo usado pelo serviço e prepara a montagem de volume indicada nas linhas 14 e 15, permitindo que o *detector-worker* aceda ao modelo em tempo de execução. Assim, o modelo permanece como um artefacto externo ao código aplicacional, podendo ser substituído sem alterar a aplicação.

4.5.2 Pipeline de Análise

A *pipeline* de análise implementada no detetor segue a sequência apresentada na Figura 4.1. Esta figura representa a sequência de ações executadas no MVP.

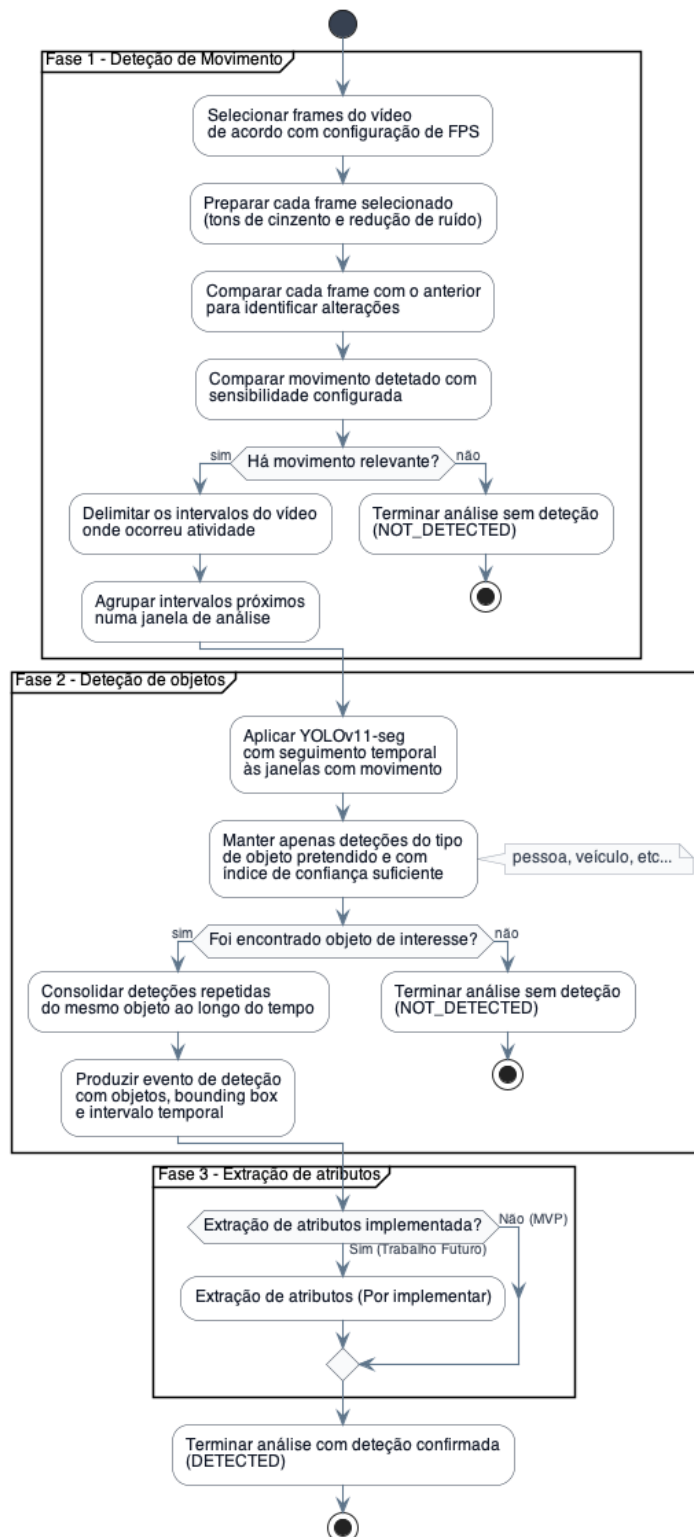


Figura 4.1: Pipeline de análise implementada no detetor automático.

O fluxo representado na figura começa já no interior do detetor. A receção do pedido assíncrono e a obtenção do segmento através do url assinado são partes do fluxo de integração entre componentes que foram introduzidas conceptualmente. Os detalhes relevantes da

implementação das mesmas são explicados na Subsecção 4.4.3, pelo que não são abordados novamente nesta subsecção. Assim, nesta subsecção assume-se que o *detector-worker* já recebeu a tarefa e dispõe do ficheiro a analisar. A partir desse ponto, o objetivo da *pipeline* é decidir se o segmento contém atividade relevante e, caso contenha, se existe algum objeto da classe alvo configurada para a Câmara.

A primeira fase funciona como um *motion gate*. O detetor não compara todos os *frames* do vídeo; em vez disso, seleciona uma amostra de *frames* com base numa taxa de amostragem configurável ao nível da aplicação. Esta taxa é combinada com o FPS original do vídeo para calcular o intervalo entre os *frames* analisados.

Cada *frame* selecionado é depois preparado para comparação, convertendo para tons de cinzento e aplicando redução de ruído, tornando a comparação menos sensível a variações pequenas. Em seguida, cada *frame* preparado é comparado com o anterior para medir a alteração visual ocorrida entre dois instantes consecutivos da amostra.

O valor de movimento obtido é comparado com a sensibilidade configurada para a análise. Se a alteração medida igualar ou ultrapassar este valor, o segmento passa a ser considerado como contendo movimento relevante. Caso contrário, a análise termina, sem avançar, com o estado NOT_DETECTED.

Quando existe movimento relevante, a *pipeline* delimita os intervalos temporais do vídeo onde essa atividade ocorreu, guardando os segundos de início e fim de cada intervalo. Intervalos muito próximos são depois agrupados numa janela de análise maior, o que evita fragmentar a mesma ocorrência em múltiplos intervalos demasiado curtos.

A segunda fase aplica a versão de segmentação do modelo YOLO11, **YOLO11-seg**, apenas às janelas com movimento. A deteção de objetos passa então de incidir sobre a totalidade do segmento a concentrar-se apenas nas partes onde a Fase 1 indicou atividade visual.

Após a inferência, são mantidas apenas as deteções compatíveis com a classe alvo configurada para a Câmara e com confiança suficiente. Quando o mecanismo de seguimento temporal associa observações sucessivas ao mesmo objeto, estas são agregadas através de um respetivo identificador único.

A terceira fase de extração de atributos, que não foi implementada no MVP, está incluída na representação. A sua presença no fluxo tem apenas função conceptual, indicando onde seriam integrados mecanismos futuros de análise de características.

O Excerto de Código 4.15 apresenta uma representação simplificada da função que orquestra a passagem entre a deteção de movimento e a deteção de objetos.

```
1 def analyze_segment(payload, local_video_path, job_id):
2     motion_score, has_motion, _ = detect_motion_score(local_video_path)
3
4     if not has_motion:
5         return CallbackPayload(status="NOT_DETECTED", events=[])
6
7     motion_segments = detect_motion_segments(local_video_path)
8
9     events = detect_objects(
10         local_video_path,
11         payload.target_class,
12         motion_score,
13         motion_context={"segments": motion_segments},
14     )
15     return CallbackPayload(
16         status="DETECTED" if events else "NOT_DETECTED",
17         events=events,
18     )
```

Excerto de Código 4.15: Representação simplificada da orquestração interna da *pipeline* de análise (python)

A chamada a `detect_motion_score` corresponde à primeira decisão da *pipeline*; se não houver movimento, o resultado é imediatamente produzido como `NOT_DETECTED`. Apenas quando há movimento são calculados os intervalos de atividade e invocado o detetor de objetos. A decisão final entre `DETECTED` e `NOT_DETECTED` depende da existência de eventos devolvidos pela Fase 2.

4.5.3 Contrato de Resultado

Depois de concluída a análise, o detetor devolve à aplicação um resultado estruturado. O Excerto de Código 4.16 apresenta um exemplo simplificado do conteúdo relevante desse resultado, focado na estrutura `events[]`.

```
1 {
2   "segment_uid": "seg_01HZX7QCVS9ANR",
3   "status": "DETECTED",
4   "events": [
5     {
6       "event_type": "object_detection",
7       "start_second": 1.2,
8       "end_second": 3.8,
9       "confidence": 0.91,
10      "motion_score": 0.35,
11      "metadata": {
12        "engine": "yolo11_seg_tracker"
13      },
14      "objects": [
15        {
16          "label": "person",
17          "confidence": 0.91,
18          "track_id": "trk_001",
19          "bbox_samples": [
20            {
21              "t": 1.2,
22              "bbox": { "x": 0.12, "y": 0.20, "w": 0.30, "h": 0.50 }
23            },
24            {
25              "t": 1.6,
26              "bbox": { "x": 0.15, "y": 0.21, "w": 0.30, "h": 0.50 }
27            }
28          ]
29        }
30      ]
31    }
32  ]
33 }
```

Excerto de Código 4.16: Exemplo simplificado da estrutura de eventos devolvida pelo detetor (json)

O campo `segment_uid` (linha 2) referencia o Segmento de Gravação a que pertence a resposta do *callback*, permitindo à aplicação associar o resultado recebido ao segmento previamente submetido para análise. O campo `status` (linha 3) indica o resultado global da análise. Os estados `DETECTED` e `NOT_DETECTED` já foram descritos na *pipeline*. Em caso de erro técnico pode ainda ser devolvido `FAILED`. Quando existe deteção, `events[]` (linha 4) contém as ocorrências identificadas no Segmento de Gravação.

Em cada evento, `event_type` (linha 6) identifica a ocorrência, `start_second` (linha 7) e `end_second` (linha 8) delimitam o intervalo relativo ao início do segmento, `confidence` (linha 9) indica a confiança do evento, numa escala de 0 a 1. O `motion_score` (linha 10) preserva a pontuação de movimento associada, atribuída na deteção de movimento. O campo `metadata` (linha 11 a 13) fica reservado para informação técnica adicional do motor de deteção.

A lista `objects[]` (linha 14) contém os objetos que justificam o evento. Cada objeto inclui a classe e a confiança da deteção, podendo também incluir `track_id` (linha 18) para seguimento temporal. O campo mais relevante para a utilização posterior é `bbox_samples` (linha 19), que representa amostras temporais da localização do mesmo objeto ao longo do

evento. Cada amostra contém o instante relativo t e uma BBOX normalizada, permitindo descrever a evolução espacial do objeto durante o intervalo detetado.

4.6 Persistência dos Eventos e Artefactos de Análise

basicamente aqui descrever como a informação do callback enviado pelo detector é recebida, processada e mapeada para cada entidade ou entidade nova dentro da app e depois persistida.

ir ao project/ e explorar como isso é feito

incluir aqui também um pedaço de código que permite a notificação do investigador quando entra e existe uma deteção nova.

4.6.1 artefactos que vao ser visuais

explicar aqui uma pequena sub seccao sobre o bbox . esta informacao da detecao é passada da area em que foi detetado o objeto e depois explicar como no frontend é visualizada esta parte.

4.7 Compromissos e Limitações da Implementação

4.8 Síntese

Capítulo 5

Validação, Avaliação e Resultados

5.1 Plano de Avaliação

5.2 Cenário Experimental

5.3 Validação Funcional

5.4 Avaliação de Segurança e Dependências

5.4.1 Software Composition Analysis

Pendente SCA.

5.5 Avaliação da Conectividade e Streaming

5.6 Avaliação do Controlo PTZ

5.7 Avaliação do Detetor

5.8 Avaliação Qualitativa

5.9 Discussão dos Resultados

5.10 Síntese

Capítulo 6

Considerações Finais

falta atualizar este capítulo todo

O presente capítulo tem como objetivo sintetizar os resultados obtidos durante a fase de preparação da dissertação, consolidando as decisões tomadas com base na investigação realizada. Apresentam-se, de seguida, as principais conclusões retiradas do estudo do estado da arte e do planeamento, bem como a rota detalhada para o desenvolvimento futuro do projeto FUSE.

6.1 Conclusões

O estudo do estado da arte, detalhado no Capítulo 2, foi determinante para as decisões arquiteturais do projeto. A análise literária confirmou que a interoperabilidade entre dispositivos heterogêneos exige uma camada de abstração robusta baseada em microserviços. No domínio da segurança, a investigação apontou para a obsolescência das VPNs tradicionais em cenários de *streaming* de vídeo, validando a adoção de túneis modernos como o WireGuard e topologias P2P para garantir a conectividade segura sem comprometer a performance.

Adicionalmente, a revisão sobre visão computacional demonstrou a inviabilidade atual do processamento complexo em dispositivos Edge de baixo custo, fundamentando a opção por uma arquitetura de inteligência centralizada que tira partido de modelos avançados como o YOLO e VLMs para uma análise semântica aprofundada.

O planeamento apresentado no Seção 1.7, suportado por uma análise de riscos e cronograma detalhado, demonstra a possibilidade da execução técnica e temporal do projeto, estabelecendo uma rota clara para a fase de implementação que se segue.

secção a dizer que isto foi uma das soluções, mas com o decorrer do trabalho concluiu-se que a deteção em edge computing tem lugar a ser considerada etc etc

6.2 Trabalho Futuro

falta atualizar

Com a conclusão da fase de preparação (PREPD), o trabalho transita agora para a execução técnica no âmbito da unidade curricular de dissertação (DIMEI). Os passos imediatos e futuros, alinhados com o plano de trabalhos estabelecido, focam-se na materialização do FUSE e compreendem:

- **Design Arquitetural:** Formalização da arquitetura do sistema através do modelo de Vistas 4+1, detalhando os componentes e as suas interações.
- **Implementação do MVP:** Desenvolvimento iterativo dos módulos críticos, começando pela infraestrutura de comunicações seguras, seguido pela camada de abstração de câmaras e, finalmente, a *pipeline* de análise de vídeo.
- **Validação em Cenário Real:** Instalação e teste do protótipo em ambiente operacional, permitindo recolher métricas de desempenho e feedback qualitativo dos utilizadores finais.
- **Escrita da Dissertação:** Documentação contínua do processo de desenvolvimento e resultados, culminando na produção do documento final de dissertação que reportará as conclusões do projeto.

Bibliografia

- [1] Honghai Liu, Shengyong Chen e Naoyuki Kubota. «Intelligent Video Systems and Analytics: A Survey». Em: *IEEE Transactions on Industrial Informatics* 9.3 (2013), pp. 1222–1233. doi: 10.1109/TII.2013.2255616.
- [2] Vlado Damjanovski. *CCTV: From light to pixels*. 3rd. Oxford: Butterworth-Heinemann, 2014. isbn: 978-0124046078.
- [3] W. Daniel Kissling et al. «Development of a cost-efficient automated wildlife camera network in a European Natura 2000 site». Em: *Basic and Applied Ecology* 79 (2024), pp. 141–152. issn: 1439-1791. doi: <https://doi.org/10.1016/j.baae.2024.06.006>. url: <https://www.sciencedirect.com/science/article/pii/S1439179124000458>.
- [4] Nozomi Networks. *New Reolink P2P Vulnerabilities Show IoT Security Camera Risks*. Acedido em: 12/2025. Jul. de 2021. url: <https://www.nozominetworks.com/blog/new-reolink-p2p-vulnerabilities-show-iot-security-camera-risks>.
- [5] Kaushik Ragothaman et al. «Access Control for IoT: A Survey of Existing Research, Dynamic Policies and Future Directions». Em: *Sensors (MDPI)* 23.4 (2023), p. 1805. doi: 10.3390/s23041805. url: <https://www.mdpi.com/1424-8220/23/4/1805>.
- [6] Diário de Notícias. *Falta de polícias e de viaturas e instalações e equipamentos em mau estado. Relatório aponta as falhas da PSP e GNR*. Baseado em relatório da IGA. Jun. de 2024. url: <https://www.dn.pt/sociedade/falta-de-policias-e-de-viaturas-e-instalacoes-e-equipamentos-em-mau-estado-relatorio-aponta-as-falhas-da-psp-e-gnr>.
- [7] Briony J. Oates. *Researching Information Systems and Computing*. SAGE Publications, 2006. isbn: 9781412902236. url: <https://us.sagepub.com/en-us/nam/researching-information-systems-and-computing/book226687>.
- [8] Politécnico do Porto. *Despacho P.PORTO/P-040/2020 - Código de Boas Práticas e de Conduta do P.PORTO*. 2020. url: <https://www.ipp.pt/comunidade/menu-comunidade/missao-equidade-diversidade-inclusao/DespachoP.PORTOP0402020CodigodeBoasPraticasedeConduta.pdf>.
- [9] Association for Computing Machinery. *ACM Code of Ethics and Professional Conduct*. 2018. url: <https://www.acm.org/code-of-ethics/software-engineering-code>.
- [10] European Union. *Regulation (EU) 2024/1689 laying down harmonised rules on artificial intelligence (Artificial Intelligence Act)*. 2024. url: <https://digital-strategy.ec.europa.eu/en/policies/regulatory-framework-ai>.
- [11] jamditis. *Academic Writing Skill*. GitHub repository. Acedido em: 15/05/2026. 2026. url: <https://github.com/jamditis/claude-skills-journalism/tree/master/research-toolkit/skills/academic-writing>.
- [12] Gustavo Caiano. *FUSE Dissertation Prompts*. GitHub repository. Acedido em: 15/05/2026. 2026. url: <https://github.com/gustavocaiano/fuse/tree/main/docs/prompts>.

- [13] Alberto Sampaio. «Improving Systematic Mapping Reviews». Em: *SIGSOFT Softw. Eng. Notes* 40.6 (nov. de 2015), pp. 1–8. issn: 0163-5948. doi: 10.1145/2830719.2830732. url: <https://doi.org/10.1145/2830719.2830732>.
- [14] David Moher et al. «Preferred reporting items for systematic reviews and meta-analyses: the PRISMA statement». Em: *International Journal of Surgery* 8.5 (2010). PII: S1743-9191(10)00040-3, pp. 336–341. url: <https://www.sciencedirect.com/science/article/pii/S1743919110000403>.
- [15] Mateus Rocha Resende. «Sistema de Videomonitoramento Seguro com Acesso Remoto Utilizando VPN, DNS Dinâmico e Software Livre». por. Em: (mai. de 2025). Accepted: 2025-07-23T13:28:52Z Publisher: Universidade Federal de Uberlândia. url: <https://repositorio.ufu.br/handle/123456789/46462> (acedido em 28/12/2025).
- [16] Yasser Mesmoudi et al. «Design and implementation of a smart gateway for IoT applications using heterogeneous smart objects». Em: *2018 4th International Conference on Cloud Computing Technologies and Applications (Cloudtech)*. Nov. de 2018, pp. 1–7. doi: 10.1109/CloudTech.2018.8713348.
- [17] Vinícius A. Barros et al. «An IoT multi-protocol strategy for the interoperability of distinct communication protocols applied to web of things». Em: *Proceedings of the 25th Brazillian Symposium on Multimedia and the Web*. WebMedia '19. event-place: Rio de Janeiro, Brazil. New York, NY, USA: Association for Computing Machinery, 2019, pp. 81–88. isbn: 978-1-4503-6763-9. doi: 10.1145/3323503.3349546. url: <https://doi.org/10.1145/3323503.3349546>.
- [18] Jürgen Dobaj et al. «A Microservice Architecture for the Industrial Internet-Of-Things». Em: *Proceedings of the 23rd European Conference on Pattern Languages of Programs*. EuroPLoP '18. event-place: Irsee, Germany. New York, NY, USA: Association for Computing Machinery, 2018. isbn: 978-1-4503-6387-7. doi: 10.1145/3282308.3282320. url: <https://doi.org/10.1145/3282308.3282320>.
- [19] Rachid Mafamane et al. «Study of the heterogeneity problem in the Internet of Things and Cloud Computing integration». Em: *2020 10th International Symposium on Signal, Image, Video and Communications (ISIVC)*. Abr. de 2021, pp. 1–6. doi: 10.1109/ISIVC49222.2021.9487539.
- [20] Christopher Schwarzer, Andrei Günter e Matthias König. «IAL: Information Abstraction Layer to Include Multimedia in Building Automation Systems». Em: *2021 17th International Conference on Intelligent Environments (IE)*. ISSN: 2472-7571. Jun. de 2021, pp. 1–8. doi: 10.1109/IE51775.2021.9486461.
- [21] Silvio Barra, Ferdinando D'Alessandro e Oleksandr Sosovskyy. «Exploring Architectural Choices and Emerging Challenges in Data Management for IoT: A Focus on Digital Innovation and Smart Cities». Em: *Adjunct Proceedings of the 32nd ACM Conference on User Modeling, Adaptation and Personalization*. UMAP Adjunct '24. event-place: Cagliari, Italy. New York, NY, USA: Association for Computing Machinery, 2024, pp. 429–436. isbn: 979-8-4007-0466-6. doi: 10.1145/3631700.3665238. url: <https://doi.org/10.1145/3631700.3665238>.
- [22] Anjus George e Arun Ravindran. «Distributed Middleware for Edge Vision Systems». Em: *2019 IEEE 16th International Conference on Smart Cities: Improving Quality of Life Using ICT & IoT and AI (HONET-ICT)*. ISSN: 1949-4106. Out. de 2019, pp. 193–194. doi: 10.1109/HONET.2019.8908023.
- [23] David L. Gomez et al. «Strategies for Assuring Low Latency, Scalability and Interoperability in Edge Computing and TSN Networks for Critical IIoT Services». Em: *IEEE Access* 11 (2023), pp. 42546–42577. issn: 2169-3536. doi: 10.1109/ACCESS.2023.3268223.

- [24] Roger Immich et al. «Multi-tier Edge-to-Cloud Architecture for Adaptive Video Delivery». Em: *2019 7th International Conference on Future Internet of Things and Cloud (FiCloud)*. Ago. de 2019, pp. 23–30. doi: 10.1109/FiCloud.2019.00012.
- [25] Matheus HS Muniz et al. «Pragmatic interoperability in IoT: a systematic mapping study». Em: *Proceedings of the 25th Brazilian Symposium on Multimedia and the Web. WebMedia '19*. event-place: Rio de Janeiro, Brazil. New York, NY, USA: Association for Computing Machinery, 2019, pp. 73–80. isbn: 978-1-4503-6763-9. doi: 10.1145/3323503.3349561. url: <https://doi.org/10.1145/3323503.3349561>.
- [26] Joseph Bugeja, Désirée Jönsson e Andreas Jacobsson. «An Investigation of Vulnerabilities in Smart Connected Cameras». Em: *2018 IEEE International Conference on Pervasive Computing and Communications Workshops (PerCom Workshops)*. Mar. de 2018, pp. 537–542. doi: 10.1109/PERCOMW.2018.8480184.
- [27] Suppakorn Boonprasert, Kittiphan Techakittiroj e Naebboon Hoonchareon. «Low-Cost Sky Camera with Centralized Storage Systems». Em: *2024 21st International Conference on Electrical Engineering/Electronics, Computer, Telecommunications and Information Technology (ECTI-CON)*. ISSN: 2837-6471. Mai. de 2024, pp. 1–4. doi: 10.1109/ECTI-CON60892.2024.10594931.
- [28] Daniel-Florin Hrițcan e Doru Balan. «Exposing IoT Platforms Securely and Anonymously Behind CGNAT». Em: *2024 23rd RoEduNet Conference: Networking in Education and Research (RoEduNet)*. ISSN: 2247-5443. Set. de 2024, pp. 1–4. doi: 10.1109/RoEduNet64292.2024.10722287.
- [29] Jinpo Fan, Zhiqiang Wang e Changchun Li. «Design and Implementation of IoT Gateway Security System». Em: *2019 International Conference on Artificial Intelligence and Advanced Manufacturing (AIAM)*. Out. de 2019, pp. 156–162. doi: 10.1109/AIAM48774.2019.00039.
- [30] Stefania Guarino et al. «Data Acquisition System for Thermal Monitoring in High-Voltage Step Down Substation using Raspberry Pi and IoT Sensors». Em: *2025 IEEE International Conference on Environment and Electrical Engineering and 2025 IEEE Industrial and Commercial Power Systems Europe (EEEIC / I&CPS Europe)*. ISSN: 2994-9467. Jul. de 2025, pp. 1–6. doi: 10.1109/EEEIC/ICPSEurope64998.2025.11169145.
- [31] Muhammad Asim et al. «SecT: A Zero-Trust Framework for Secure Remote Access in Next-Generation Industrial Networks». Em: *IEEE Journal on Selected Areas in Communications* 43.6 (jun. de 2025), pp. 2293–2311. issn: 1558-0008. doi: 10.1109/JSAC.2025.3560015.
- [32] Diwen Xue et al. «OpenVPN is Open to VPN Fingerprinting». Em: *Commun. ACM* 68.1 (dez. de 2024). Place: New York, NY, USA Publisher: Association for Computing Machinery, pp. 79–87. issn: 0001-0782. doi: 10.1145/3618117. url: <https://doi.org/10.1145/3618117>.
- [33] Stephan Hohmann, Tobias Mueller e Marius Stübs. «Bridge Me If You Can! Evaluating the Latency of Securing Profinet». Em: *2021 International Conference on Information Networking (ICOIN)*. ISSN: 1976-7684. Jan. de 2021, pp. 621–626. doi: 10.1109/ICOIN50884.2021.9333897.
- [34] Yongchao Dang et al. «EWDC: Integrating DECT-2020 With Wi-Fi for Enhanced Wireless Direct Connectivity». Em: *IEEE Internet of Things Journal* 12.15 (ago. de 2025), pp. 31783–31796. issn: 2327-4662. doi: 10.1109/JIOT.2025.3574407.
- [35] Mehmet Bezenk e Hayriye Korkmaz. «Remote Access Solution for Industrial Devices with VPN». Em: *2024 Innovations in Intelligent Systems and Applications Conference*

- (ASYU). ISSN: 2770-7946. Out. de 2024, pp. 1–8. doi: 10.1109/ASYU62119.2024.10757061.
- [36] Kit-Lun Tong et al. «DAIoTtalk: A Data-Decentralized Pub-Sub AIoT Platform». Em: *2025 IEEE 101st Vehicular Technology Conference (VTC2025-Spring)*. ISSN: 2577-2465. Jun. de 2025, pp. 1–6. doi: 10.1109/VTC2025-Spring65109.2025.11174754.
- [37] Ander Garcia et al. «Containerized Edge Architecture for Centralized Industry 4.0 Fleet Management». Em: *2023 IEEE 9th World Forum on Internet of Things (WF-IoT)*. ISSN: 2768-1734. Out. de 2023, pp. 1–5. doi: 10.1109/WF-IoT58464.2023.10539473.
- [38] Sabarishraj K et al. «Efficient Implementation of Firmware Over-the-Air Update using Raspberry Pi 4 and STM32F103C8T6». Em: *2024 IEEE International Conference on Electronics, Computing and Communication Technologies (CONECCT)*. ISSN: 2766-2101. Jul. de 2024, pp. 1–6. doi: 10.1109/CONECCT62155.2024.10677143.
- [39] Roberta Avanzato, Francesco Beritelli e Corrado Rametta. «Enhancing Perceptual Experience of Video Quality in Drone Communications by Using VPN Bonding». Em: *IEEE Embedded Systems Letters* 15.1 (mar. de 2023), pp. 1–4. issn: 1943-0671. doi: 10.1109/LES.2022.3182466.
- [40] Rahma Trabelsi, Ghofrane Fersi e Mohamed Jmaiel. «A fog and blockchain-based distributed Virtual Private Networks (VPN)». Em: *2024 20th International Conference on Distributed Computing in Smart Systems and the Internet of Things (DCOSS-IoT)*. ISSN: 2325-2944. Abr. de 2024, pp. 130–137. doi: 10.1109/DCOSS-IoT61029.2024.00028.
- [41] Marwa Qaraqe et al. «PublicVision: A Secure Smart Surveillance System for Crowd Behavior Recognition». Em: *IEEE Access* 12 (2024), pp. 26474–26491. issn: 2169-3536. doi: 10.1109/ACCESS.2024.3366693.
- [42] Deiescart D’Mitrio C. Maceda e Glenn V. Magwili. «Scada Web-Based Instrumentation and Control Laboratory with Real-Time Remote Monitoring and Control System for Columban College». Em: *2022 IEEE 14th International Conference on Humanoid, Nanotechnology, Information Technology, Communication and Control, Environment, and Management (HNICEM)*. ISSN: 2770-0682. Dez. de 2022, pp. 1–6. doi: 10.1109/HNICEM57413.2022.10109507.
- [43] Syed Rafiul Hussain, Shahriar Nirjon e Elisa Bertino. «Securing the Insecure Link of Internet-of-Things Using Next-Generation Smart Gateways». Em: *2019 15th International Conference on Distributed Computing in Sensor Systems (DCOSS)*. ISSN: 2325-2944. Mai. de 2019, pp. 66–73. doi: 10.1109/DCOSS.2019.00032.
- [44] Maria J. Grant e Andrew Booth. «A typology of reviews: an analysis of 14 review types and associated methodologies». Em: *Health Information and Libraries Journal* 26.2 (2009), pp. 91–108. doi: 10.1111/j.1471-1842.2009.00848.x. url: <https://onlinelibrary.wiley.com/doi/full/10.1111/j.1471-1842.2009.00848.x>.
- [45] Tong Bai et al. «A Decade of Video Analytics at Edge: Training, Deployment, Orchestration, and Platforms». Em: *IEEE Communications Surveys & Tutorials* 28 (2025), pp. 2127–2162. doi: 10.1109/COMST.2025.3563377.
- [46] Sebastian Pecolt et al. «Personal Identification Using Embedded Raspberry Pi-Based Face Recognition Systems». Em: *Applied Sciences* 15.2 (2025). issn: 2076-3417. doi: 10.3390/app15020887. url: <https://www.mdpi.com/2076-3417/15/2/887>.
- [47] Mingzhao Lu e Yonggang Xu. «Real-Time Analysis of Dangerous Actions in Video Surveillance Systems Based on Cloud Computing». Em: *2025 5th International Conference on Neural Networks, Information and Communication Engineering (NNICE)*. 2025, pp. 664–667. doi: 10.1109/NNICE64954.2025.11064503.

- [48] Tanin Sultana e Khan A. Wahid. «IoT-Guard: Event-Driven Fog-Based Video Surveillance System for Real-Time Security Management». Em: *IEEE Access* 7 (2019), pp. 134881–134894. doi: 10.1109/ACCESS.2019.2941978.
- [49] Juan Terven e Diana Cordova-Esparza. «A Comprehensive Review of YOLO Architectures in Computer Vision: From YOLOv1 to YOLOv8 and YOLO-NAS». Em: *Machine Learning and Knowledge Extraction* 5.4 (2023), pp. 1680–1716. issn: 2504-4990. doi: 10.3390/make5040083. url: <https://www.mdpi.com/2504-4990/5/4/83>.
- [50] Rahima Khanam, Tahreem Asghar e Muhammad Hussain. «Comparative Performance Evaluation of YOLOv5, YOLOv8, and YOLOv11 for Solar Panel Defect Detection». Em: *Solar* 5.1 (2025). issn: 2673-9941. doi: 10.3390/solar5010006. url: <https://www.mdpi.com/2673-9941/5/1/6>.
- [51] Glenn Jocher e Jing Qiu. *Ultralytics YOLO11*. 2024. url: <https://docs.ultralytics.com/models/yolo11/>.
- [52] Wang Xia et al. «Enhancing Visual Analysis in Person Reidentification With Vision-Language Models». Em: *IEEE Computer Graphics and Applications* 45.6 (2025), pp. 44–60. doi: 10.1109/MCG.2025.3593227.
- [53] Minjae Bang et al. «Reasoning-augmented Vision-Language Models for Improved Competency in Scene Text Recognition». Em: *2025 IEEE/IEIE International Conference on Consumer Electronics-Asia (ICCE-Asia)*. 2025, pp. 1–6. doi: 10.1109/ICCE-Asia67487.2025.11263773.
- [54] Jiuhai Chen et al. «Florence-VL: Enhancing Vision-Language Models with Generative Vision Encoder and Depth-Breadth Fusion». Em: *2025 IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*. 2025, pp. 24928–24938. doi: 10.1109/CVPR52734.2025.02321.
- [55] Dinh-Lam Pham et al. «Deep Learning and Graph-Based Indexing for Context-Aware Video Retrieval Systems». Em: *2025 27th International Conference on Advanced Communications Technology (ICACT)*. 2025, pp. 318–322. doi: 10.23919/ICACT63878.2025.10936725.
- [56] Jeba Sonia J e Priyadarsshini S. «Advanced Object Detection Using Semantic and Temporal Query-Based Frame Retrieval in Surveillance Videos». Em: *2025 3rd International Conference on Inventive Computing and Informatics (ICICI)*. Jun. de 2025, pp. 166–172. doi: 10.1109/ICICI65870.2025.11069872.
- [57] Headscale Project. *Headscale*. Acedido em: 03/05/2026. 2026. url: <https://headscale.net/stable/>.
- [58] Filament. *What is Filament?* Acedido em: 31/05/2026. 2026. url: <https://filamentphp.com/docs/4.x/introduction/overview>.
- [59] MediaMTX. *MediaMTX Introduction*. Acedido em: 01/06/2026. 2026. url: <https://mediamtx.org/docs/kickoff/introduction>.
- [60] Python Package Index. *onvif-zeep*. Acedido em: 01/06/2026. 2026. url: <https://pypi.org/project/onvif-zeep/>.
- [61] Aubrey L. Mendelow. «Environmental Scanning: The Impact of the Stakeholder Concept». Em: *ICIS 1981 Proceedings* (1981), p. 20. url: <https://aisel.aisnet.org/icis1981/20/>.
- [62] Rensis Likert. «A technique for the measurement of attitudes». Em: *Archives of Psychology* 22.140 (1932), pp. 1–55. url: <https://psycnet.apa.org/record/1933-01885-001>.

Apêndice A

Questionário e Entrevistas

O conteúdo deste apêndice refere-se ao questionário em formato de entrevista realizado a agentes de investigação criminal da GNR e da PSP para validação do problema identificado.

A.1 Síntese de Resultados

A recolha de dados teve um carácter exploratório, focando-se na identificação de problemas práticos no acesso e análise de prova adquirida a partir de sistemas CCTV durante a investigação criminal. As respostas obtidas junto dos OPCs validam os desafios descritos na Seção 1.2, destacando-se as seguintes conclusões gerais:

- **Conectividade e Acesso:** Confirma-se a utilização frequente de câmaras com cartão SIM/4G em redes não controladas e a dependência de dispositivos fora da rede institucional para o acesso inseguro às mesmas.
- **Fragmentação:** A generalidade dos inquiridos reporta a necessidade de utilizar múltiplas plataformas ou aplicações incompatíveis para aceder a diferentes equipamentos.
- **Ineficiência Operacional:** A análise de vídeo é descrita como um processo moroso (frequentemente demorando largos períodos de tempo para localizar e exportar eventos relevantes), assentando quase exclusivamente na revisão manual de gravações e *playback* integral.
- **Restrições Institucionais:** Existe um forte consenso sobre a presença de restrições internas rigorosas quanto à instalação de novo software ou abertura de acessos diretos nas redes dos OPCs.
- **Recetividade a Soluções Unificadas:** Os inquiridos demonstraram uma elevada receptividade a uma aplicação única que centralize o acesso às câmaras, integrando alertas e mecanismos de deteção automática.

A.2 Questionário (ficheiro de respostas)

Para consulta e análise detalhada, o ficheiro Excel com as respostas do questionário encontra-se disponível numa versão estruturada e apresentável:

- **Ficheiro (XLSX):** Disponível online em: https://github.com/gustavocaiano/fuse/blob/main/docs/questionary/questionary_presentable.xlsx

Apêndice B

Project Charter

O conteúdo deste apêndice refere-se ao Project Charter, documento inicial que formalizou a proposta de dissertação junto da empresa acolhedora e da instituição académica. Conforme referido no Apêndice D, este artefacto serviu como ponto de partida para a definição do problema, objetivos e identificação de stakeholders.

B.1 Project Charter (documento completo)

O documento completo do Project Charter encontra-se disponível para consulta:

- **Ficheiro (PDF):** Disponível online em: <https://github.com/gustavocaiano/fuse/blob/main/docs/project-charter.pdf>

Apêndice C

Work Breakdown Structure

O âmbito deste projeto foi decomposto através de um WBS. Pelo mesmo, ilustrado na Figura C.1, estão organizados os entregáveis previstos por cinco fases principais, sendo estas:

1. **Planeamento e Análise:** Focada na definição do problema e estado da arte. Inclui entregáveis como o Project Charter, o próprio WBS, um Gantt Chart, um questionário e respetivas respostas dos OPCs, este Extended Abstract e a revisão da literatura incluída no mesmo.
2. **Design:** Dedicada à modelação da solução. Os principais entregáveis previstos são a documentação da Arquitetura de Software (Vistas 4+1) e o Modelo de Domínio, assegurando que a estrutura do FUSE é robusta antes da implementação.
3. **Desenvolvimento:** Fase central do projeto, onde será implementado o código da aplicação. Divide-se em três módulos críticos: Comunicações Seguras, Camada de Abstração e Módulo de Análise de Vídeo.
4. **Testes:** Validação da solução através de testes unitários e funcionais, em conjunto com a validação do protótipo em ambiente controlado ou cenário real.
5. **Conclusões:** Considerações finais do projeto, incluindo a redação final da dissertação, a interpretação dos resultados e a apresentação final.

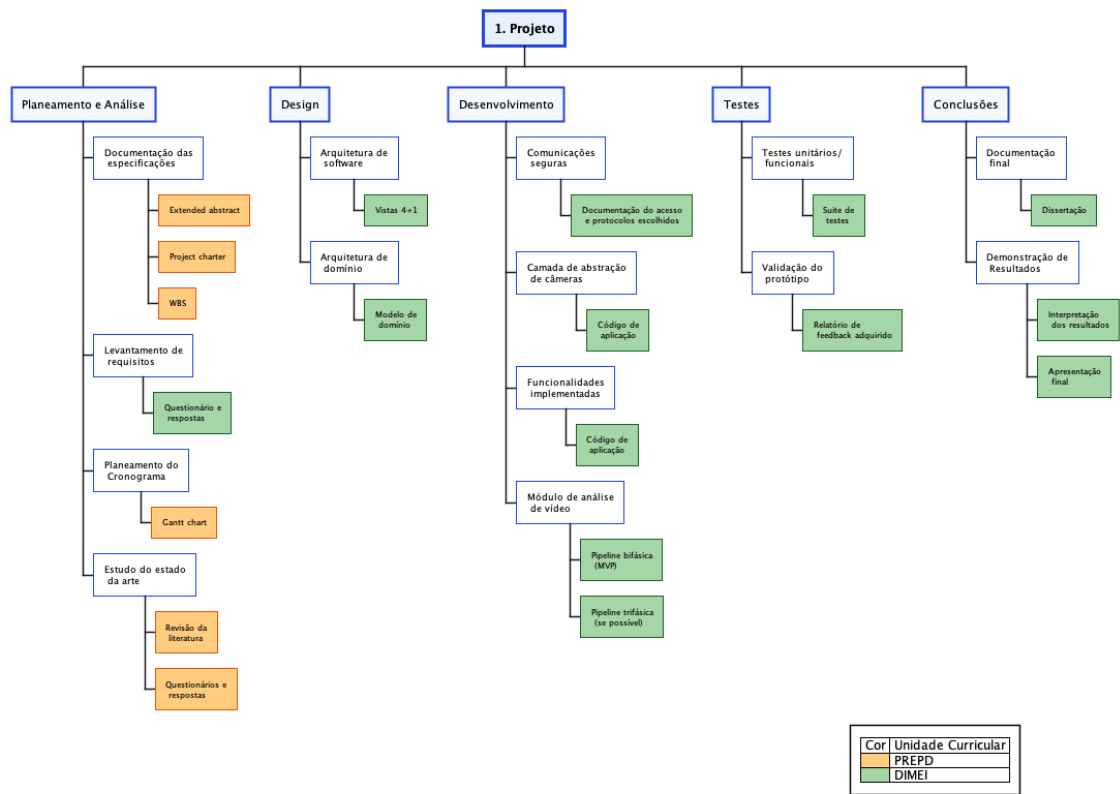


Figura C.1: WBS do projeto.

Apêndice D

Identificação de Stakeholders e Gestão de Riscos

O Project Charter (Apêndice B) constituiu o artefacto inicial deste projeto, servindo como primeiro documento para a formalização da proposta de dissertação junto da empresa acolhedora e da instituição académica. Dada a sua natureza preliminar, este documento apresentou uma visão inicial do problema e dos objetivos que, embora alinhados com a missão do FUSE, se revelaram amplos, tendo sido posteriormente refinados e concretizados no WBS e no plano de trabalhos apresentado na Seção 1.7. As datas e entregáveis originais constantes no Charter foram, por conseguinte, ajustados para refletir um mais correto e detalhado planeamento.

D.1 Identificação de Stakeholders

Apesar dos ajustes realizados, o Project Charter mantém-se como a referência central para a identificação das partes interessadas (Stakeholders). A Tabela D.1 apresenta o mapeamento atualizado dos Stakeholders, classificando-os quanto ao seu poder de influência e nível de interesse no sucesso do projeto. Como principais diferenças entre a identificação do presente documento e a do Project Charter, destaca-se a inclusão da StabilityBubble, cuja posição estratégica evolui para fornecedor potencial do software pós-desenvolvimento, e a exclusão de entusiastas de Home Automation, inicialmente projetados como possível público-alvo. Esta exclusão dá-se devido à grande apropriação do FUSE a entidades e OPCs, dado que se torna uma solução demasiado desenvolvida para projetos caseiros e pequenos.

Para a classificação dos Stakeholders, utilizou-se como base a "Mendelow's Matrix" [61]. A atribuição dos níveis ('Alto', 'Médio', 'Baixo') obedeceu aos seguintes critérios:

- **Poder:** Capacidade da entidade em influenciar decisões, alocar recursos ou bloquear o andamento do projeto.
- **Interesse:** Grau de impacto que os resultados do projeto terão nas operações ou estratégia da entidade.

Tabela D.1: Identificação de Stakeholders com atribuição de Poder e Interesse

Nome	Poder	Interesse	Justificação
StabilityBubble / NovaForensic	Alto	Alto	Entidade promotora e futura fornecedora/exploradora comercial da solução FUSE.
OPCs	Alto	Alto	Utilizadores finais críticos; o seu feedback valida a utilidade operacional e requisitos forenses.
Paulo Baltarejo Sousa (Orientador)	Alto	Médio	Orientação académica e validação científica da metodologia e resultados.
Francisco Loureiro (Supervisor)	Alto	Alto	Supervisão empresarial e alinhamento do produto com a estratégia de mercado.
Gestores de Segurança	Médio	Médio	Potenciais clientes empresariais que beneficiam da centralização de sistemas CCTV.
Fornecedores de Câmaras	Baixo	Baixo	O FUSE visa a eliminação da dependência de um só fornecedor, que tanto pode ser um novo fator decisivo para a escolha do fabricante.

D.2 Gestão de Riscos

Relativamente à gestão de riscos, trata-se de um processo contínuo que se iniciou com o levantamento preliminar no Project Charter. Embora a análise inicial focasse riscos mais genéricos (e.g., dependência de hardware), o planeamento aprofundado permitiu identificar ameaças mais específicas e definir estratégias de mitigação concretas. A Tabela D.2 consolida os riscos, atribuindo-lhes uma probabilidade e um impacto, combinando alguns identificados na fase inicial com novos riscos técnicos decorrentes da complexidade da análise de vídeo e integração de redes. A quantificação do risco segue uma matriz de probabilidade e impacto ($P \times I$), onde ambas as variáveis são classificadas numa escala de Likert [62] de 1 a 5. Esta escala foi definida da seguinte forma:

- **Probabilidade (P):** 1 representando um acontecimento raro ou extremamente improvável, e 5 algo que seja extremamente provável acontecer, possivelmente previsto.
- **Impacto (I):** 1 compreende uma insignificância ou ligeiro atraso, enquanto que 5 impacta criticamente, possivelmente impossibilitando o projeto ou condicionando o resultado do mesmo.

A multiplicação destes fatores resulta no Risco Quantificado, permitindo priorizar as estratégias de mitigação para as ameaças com pontuação mais elevada, conforme demonstrado na Tabela D.2.

Tabela D.2: Identificação e gestão de riscos associados ao projeto

ID	Descrição	Causa	P	I	P*I	Estratégia
1	Indisponibilidade de Hardware (GPU)	A análise de vídeo requer hardware gráfico potente, que é escasso ou não disponível	3	4	12	Mitigar
2	Incompatibilidade de Protocolos	Determinada câmara não suporta o protocolo RTSP, impossibilitando a integração prevista.	2	2	4	Aceitar
3	Falsos Positivos na Detecção	<i>Pipeline</i> de AI gera demasiados alertas falsos em condições adversas (chuva, noite).	3	3	9	Mitigar
4	Atraso no Cronograma	A complexidade da integração de múltiplos sistemas excede o tempo previsto para o semestre letivo.	3	5	15	Mitigar

A Tabela D.2 apresenta quatro riscos identificados, dos quais três requerem estratégias de mitigação e um é aceite como parte do âmbito do projeto. Pela análise à tabela, destaca-se os 1º e 4º riscos, Indisponibilidade de Hardware (GPU) e Atraso no Cronograma, com os índices de risco mais elevados sendo eles 12 e 15, respetivamente. Por consequente, conclui-se que grande parte do sucesso do projeto e dos resultados obtidos dependerão intrinsecamente da gestão rigorosa dos recursos computacionais e do cumprimento dos prazos estipulados, bem como o seguimento do planeamento efetuado.

As estratégias de resposta detalhadas para cada risco são as seguintes:

Risco 1 - Indisponibilidade de Hardware (GPU): A estratégia de mitigação consiste em garantir que a integração com a *pipeline* de análise automática seja feita de forma modular e desacoplada, permitindo a flexibilidade da escolha do modelo de AI utilizado conforme o hardware disponível no momento. Esta abordagem assegura que o sistema possa adaptar-se a diferentes configurações de hardware sem comprometer a funcionalidade core.

Risco 2 - Incompatibilidade de Protocolos: A opção pela estratégia de “Aceitação” serve para delimitar claramente o âmbito do projeto. O MVP a ser desenvolvido não pretende ser universalmente compatível com todo o conjunto de câmaras existentes, mas sim provar o conceito através de um protocolo standard habitualmente presente (RTSP), excluindo câmaras que não suportem este protocolo. Esta decisão permite focar o esforço de engenharia na inteligência do sistema e não na compatibilidade de drivers.

Risco 3 - Falsos Positivos na Detecção: A mitigação será realizada através da implementação de um sistema de ajuste de sensibilidade configurável pelo utilizador, permitindo que este defina parâmetros como os segundos necessários de deteção do objeto antes do alerta ser gerado. Esta funcionalidade permite reduzir falsos positivos em condições adversas, como chuva ou condições de pouca luz.

Risco 4 - Atraso no Cronograma: A estratégia de mitigação assenta na adoção de uma abordagem de desenvolvimento incremental. Será assegurada a estabilidade do núcleo de visualização e deteção básica (*pipeline* bi-fásica) como um MVP robusto, antes de iterar para as funcionalidades avançadas de extração de atributos (*pipeline* tri-fásica). Caso o desenvolvimento da *pipeline* tri-fásica se revele incompatível com o tempo disponível, esta será tratada como trabalho futuro, garantindo sempre a entrega de um produto funcional.

A estratégia de mitigação para os riscos prioritários (Riscos 1 e 4) assenta fundamentalmente na modularidade e no desenvolvimento incremental. Ao garantir que a arquitetura do sistema é flexível quanto aos modelos de AI e que as funcionalidades são entregues por fases, assegura-se que, mesmo na eventualidade destes riscos se materializarem, o projeto resultará sempre num produto funcional (MVP), salvaguardando a dissertação. A monitorização destes riscos será realizada periodicamente em cada reunião de ponto de situação com o orientador e supervisor, permitindo o ajuste dinâmico das estratégias de resposta caso a probabilidade ou impacto de algum fator sofra alterações ao longo do ciclo de vida do projeto.